

생성형 AI 서비스 개발의 이해/활용 (LangChain)

2026

본 문서는 SK(주)AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

skala_langchain.enn.kr

▪ 실습환경 : colab

colab으로 열어서, 사본 복사하기

PDF (교재)AI캠퍼스_생성형AI_5.생성형 AI 서비스 개발_이미애.pdf

5. LangChain_[2]_LLM 친해지기.ipynb의 사본

5. LangChain_[3]_LangChain.ipynb의 사본

5. LangChain_[4]_Basic_Agent.ipynb의 사본

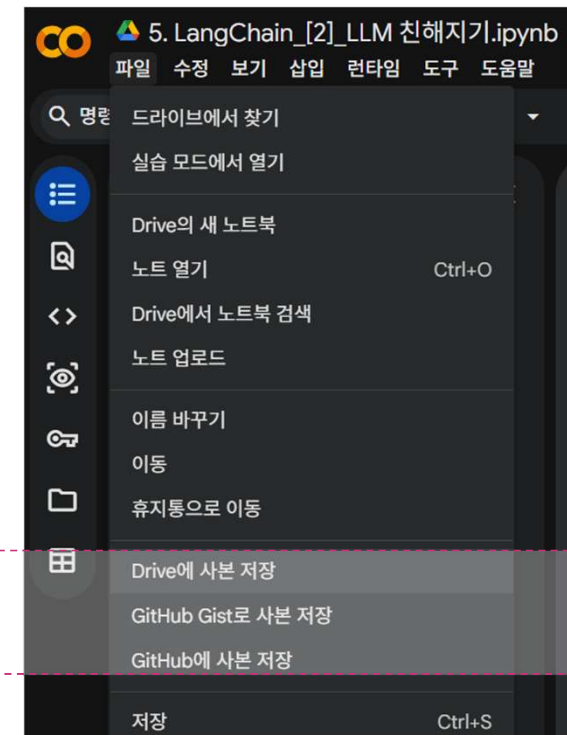
5. LangChain_[5]_Advanced_Agent.ipynb의 사본

capital.yaml

fruit_color.yaml

test_data.txt

예시소스



시간표

일차	목차		방식	시간
DAY1	1. Introduction	- AI Application 개발 - AI 제품의 6단계 진화	이론	0.5
	2. LLM과 친해지기	- LLM - Hugging Face - Google GenAI - Open AI	이론+예시	2.5
	3. LangChain	- LangChain 기본 컴포넌트 - Chat Model, Message - Prompt, Prompt Template - Structured Output - LCEL, Tool Calling	이론+예시	3
	4. Basic Agent	- Tool 사용 - Short-term Memory - 구조화된 답변	이론+예시	2
DAY2	5. Advanced Agent	- Runtime & State - Middleware - Guardrails - Long-term Memory	이론+예시	3
	퀴즈 (개별)	퀴즈	퀴즈	1
	종합실습 (조별)	Agent 설계 (설계서 제출)	종합실습	4
DAY3		Agent 개발 (설계서(최종), 개발 소스 제출)	종합실습	8



1. Introduction

2. LLM과 친해지기

3. LangChain

4. Basic Agent

5. Advanced Agent

1. Introduction

- AI Application 개발
- AI 제품의 6단계 진화
- Agent 란?

본 문서는 SK(주)AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

AI 애플리케이션 개발은
사전에 정의된 규칙에 따라 동작하는 전통적인 소프트웨어와 달리,
데이터 학습 모델(ML/DL/LLM)을 핵심 엔진으로 탑재하여
사용자의 맥락을 이해하고 실시간으로 최적의 예측, 추론, 행동을 수행하는
지능형 시스템을 구축하는 과정

AI Application 개발

비교 항목	전통적인 S/W 개발 (Traditional SW)	AI 모델 개발 (Core AI Model)	AI Application 개발 (AI/LLM App)	AI Agent 개발 (Autonomous Agent)
핵심 패러다임	• 규칙 기반 (Rule-based) • 결정론적 구조	• 데이터 기반 (Data-driven) • 패턴 학습 및 추론	• 문맥 기반 (Context-based) • AI 모델 인터페이스 통합	• 목표 기반 (Goal-oriented) • 자율적 실행 및 루프
기본 작동 방식	• 개발자가 명시한 로직에 따라 입력값을 처리하여 출력 • IF-THEN 규칙, 알고리즘 순차 실행	• 대규모 데이터를 통해 특징(Feature)과 규칙을 학습 • 통계적 패턴 학습 및 확률적 추론	• 파운데이션 모델(LLM 등)에 데이터(RAG)를 결합해 답변 생성 • 프롬프트 + RAG 검색 + API 연동 파이프라인	• 목표가 주어지면 스스로 계획을 세우고 도구를 사용해 과업 완수 • Perceive → Plan → Act → Reflect 루프
개발자의 주요 역할	• 비즈니스 로직 설계, 데이터베이스 모델링, UI/UX 구현	• 데이터 전처리/정제, 모델 아키텍처 설계, 학습 및 파인튜닝	• 프롬프트 엔지니어링, RAG 파이프라인 및 백엔드 시스템 연동	• 에이전트 페르소나 정의, 도구(Tool) 배치, 멀티 에이전트 협업 설계
자율성 및 판단력	• 없음 • 설계된 분기문(If-Else)대로만 작동	• 낮음 • 입력 데이터에 대해 정해진 확률적 결과만 수동적 출력	• 보통 • 주어진 문맥과 프롬프트 안에서 유연하게 판단	• 높음 • 실패 시 대안을 찾고, 도구 사용 여부를 스스로 결정
외부 도구 활용 (API, DB 등)	• 개발자가 코드로 지정해 둔 기능만 수동적으로 호출	• 모델 내부 연산만 수행 (외부 도구 직접 호출 불가)	• 개발자가 설계한 파이프라인에 따라 모델이 필요한 텍스트 생성	• 에이전트가 상황을 보고 스스로 어떤 API/도구를 쓸지 선택
운영 및 관리 (XOps)	• DevOps • (CI/CD, 서버 인프라, 시스템 리소스 모니터링)	• MLOps • (데이터/모델 드리프트 감지, 재학습 파이프라인)	• LLMOps • (토큰 비용 최적화, 환각률 제어, 프롬프트 관리)	• AgentOps • (무한 루프 방지, 도구 호출 성공률, 자율성 가드레일)
대표적인 도구	• Java, Python, Spring, React, PostgreSQL 등	• PyTorch, TensorFlow, Hugging Face, CUDA 등	• LangChain, LlamaIndex, OpenAI API, Pinecone, Chroma 등	• AutoGen, CrewAI, LangGraph, AgentOps 등

AI Application 개발

1. Introduction

LangChain

- LCEL(LangChain Expression Language)과 체인 구성
- 프롬프트 엔지니어링 및 구조화된 출력 (Structured Output)
- 도구 바인딩 (Tool Calling)과 대화 메모리(Memory) 관리

LLM 제어 및 선형적인 AI 애플리케이션 파이프라인 구축

RAG

- 데이터 전처리(ETL) 및 벡터 데이터베이스 적재
- 고급 검색 기술 (Advanced Retrieval & Re-ranking)
- 컨텍스트 주입 및 할루시네이션(환각) 제어

기업 내부 데이터 및 비정형 문서를 활용한 지식 기반 서비스 개발

LangGraph

- 상태(State) 기반 순환 그래프(Cyclic Graph) 설계
- 영속성(Persistence)과 동적 인터럽트 (Dynamic Interrupt)
- 멀티 에이전트 오케스트레이션 (Multi-Agent System)

복잡한 상태를 제어하고 스스로 판단·성찰하는 자율형 멀티 에이전트 구축

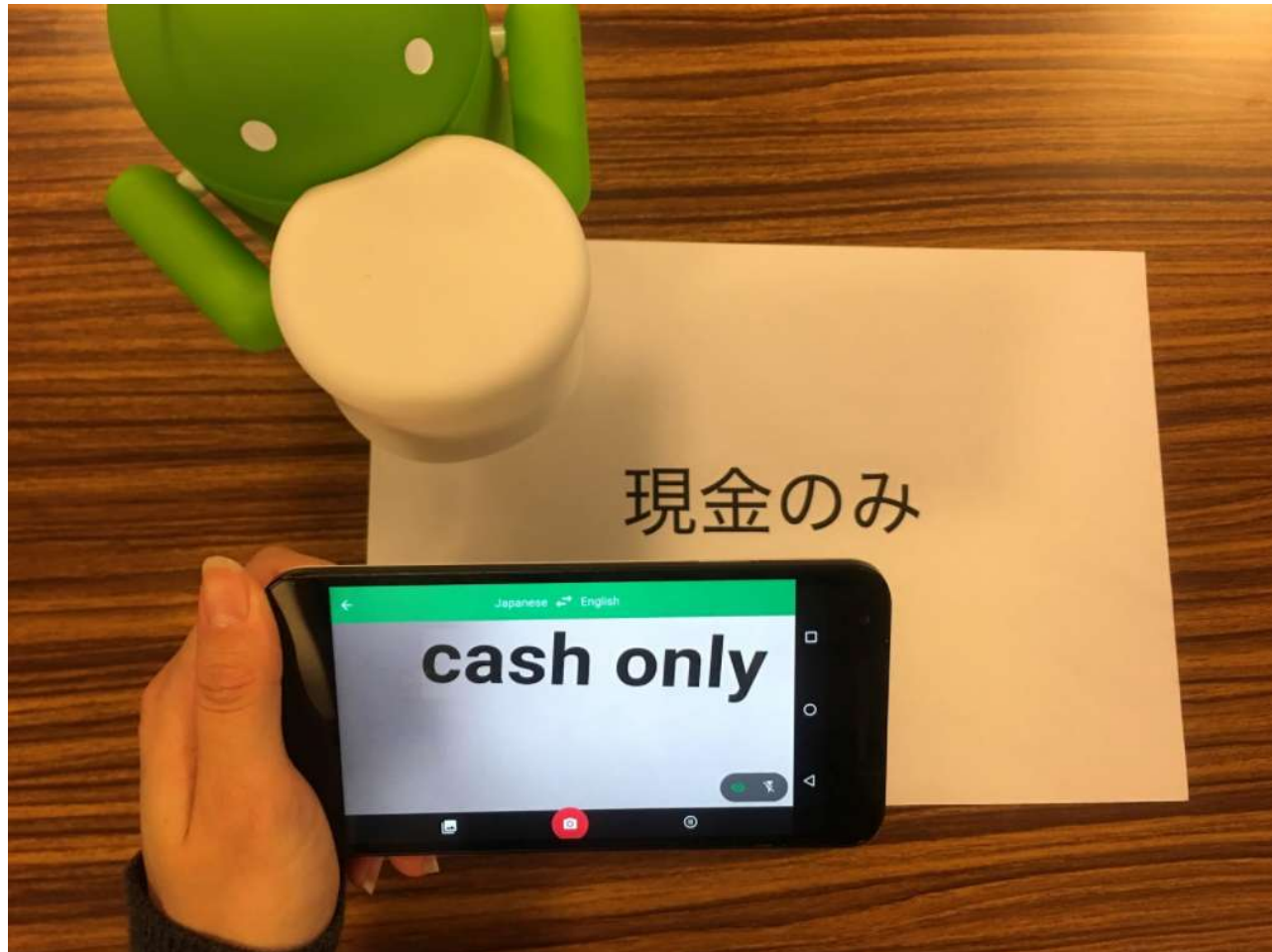
AI 제품의 6단계 진화

단계	변화 내용	예시
1단계. 백그라운드 ML (2016~2022)	AI인지도 모르고 편익을 누리던 '비가시성'의 시대	Google 번역, YouTube 추천 모델이 기존 UI 뒤에서 조용히 핵심 기능 지원
2단계. 채팅 (2022년 말)	AI가 전면에 등장하며 인간의 언어를 이해하기 시작한 '소통'의 시대	ChatGPT AI 모델이 제품의 전경으로 등장하며 대화가 주된 제품
3단계. RAG (2023)	AI의 고질적인 문제인 환각(Hallucination)을 해결하고, 실시간 정보와 기업 내부 데이터를 연결하며 '비즈니스 활용 가능성'을 입증한 결정적 단계	웹 검색을 사용하는 ChatGPT 외부 문맥을 추가하여 AI 모델의 품질과 신뢰도를 향상
4단계. Foreground 에이전트 (2024~2025)	단순한 답변가를 넘어, AI가 스스로 계획을 세우고 도구를 사용해 실제 업무를 수행하며, 사용자는 진행 중간에 개입해 방향을 조정하는 협업형 에이전트 단계	맞춤형 여행 플래너 (Mindtrip, Layla) 사용자의 예산과 취향을 듣고, 항공권/호텔 조회를 해서 여러 루트를 생성하여 사용자에게 컨펌 받아 예약을 진행하는 에이전트
5단계. Background 에이전트 (2025~)	사용자가 잠든 사이에도 AI가 수천 개의 이메일을 분류하고, 보고서 초안을 작성해 두는 시대. 인간은 '작업자'에서 '검토자(Reviewer)'로 역할 변화	자율형 소프트웨어 엔지니어 (Devin) 사용자가 이슈 하나만 던져두면, 밤사이 코드를 수정하고 테스트를 마친 뒤 결과를 보고
6단계. Agent2Agent 생태계 (진화 중)	AI가 사람을 대신해 다른 서비스의 AI와 직접 대화하고, 협상하며, 거래하는 단계	Google의 A2A 프로토콜 서로 다른 앱의 에이전트들이 정보를 주고받을 수 있는 오픈 표준 프로토콜을 제안

AI 제품의 6단계 진화

1단계. 백그라운드 ML (2016~2022)

1. Introduction



AI 제품의 6단계 진화

2단계. 채팅 (2022년 말)

ChatGPT ▾

Temporary

What can I help with?

Ask anything

+

🌐 Search

💡 Reason

🔍 Deep research

🖼️ Create image

...

🎤

🔊

AI 제품의 6단계 진화

1. Introduction

3단계. 검색 증강 생성 (RAG) (2023)



AI Overview

"AI Mode" by Google is an advanced search experience that uses artificial intelligence to provide comprehensive, AI-powered responses to complex questions. Users can interact with AI Mode using text, voice, or images and engage in follow-up questions for deeper exploration. This tool offers a more powerful and efficient way to understand new concepts, compare options, and complete tasks directly through the AI-generated answer, which often includes links to further information and sources.

Key Features

- Multimodal Search: Ask questions using text, voice, or images.

Show more

AI Mode in Search gets new agentic features and expands globally

Aug 21, 2025 — We're starting to roll out today with finding restaurant reservations, and...

The Keyword

AI Features and Your Website | Google Search Central

As with Search overall, the AI features AI Overviews and AI Mode surface relevant links ...



search.google

https://search.google > ways-to-search > ai-mode

Google AI Mode - a new way to search, whatever's on your mind

AI Mode is our most powerful AI search. Ask whatever's on your mind to get an AI powered response, and explore further with follow-up questions and helpful web ...

Google AI Mode

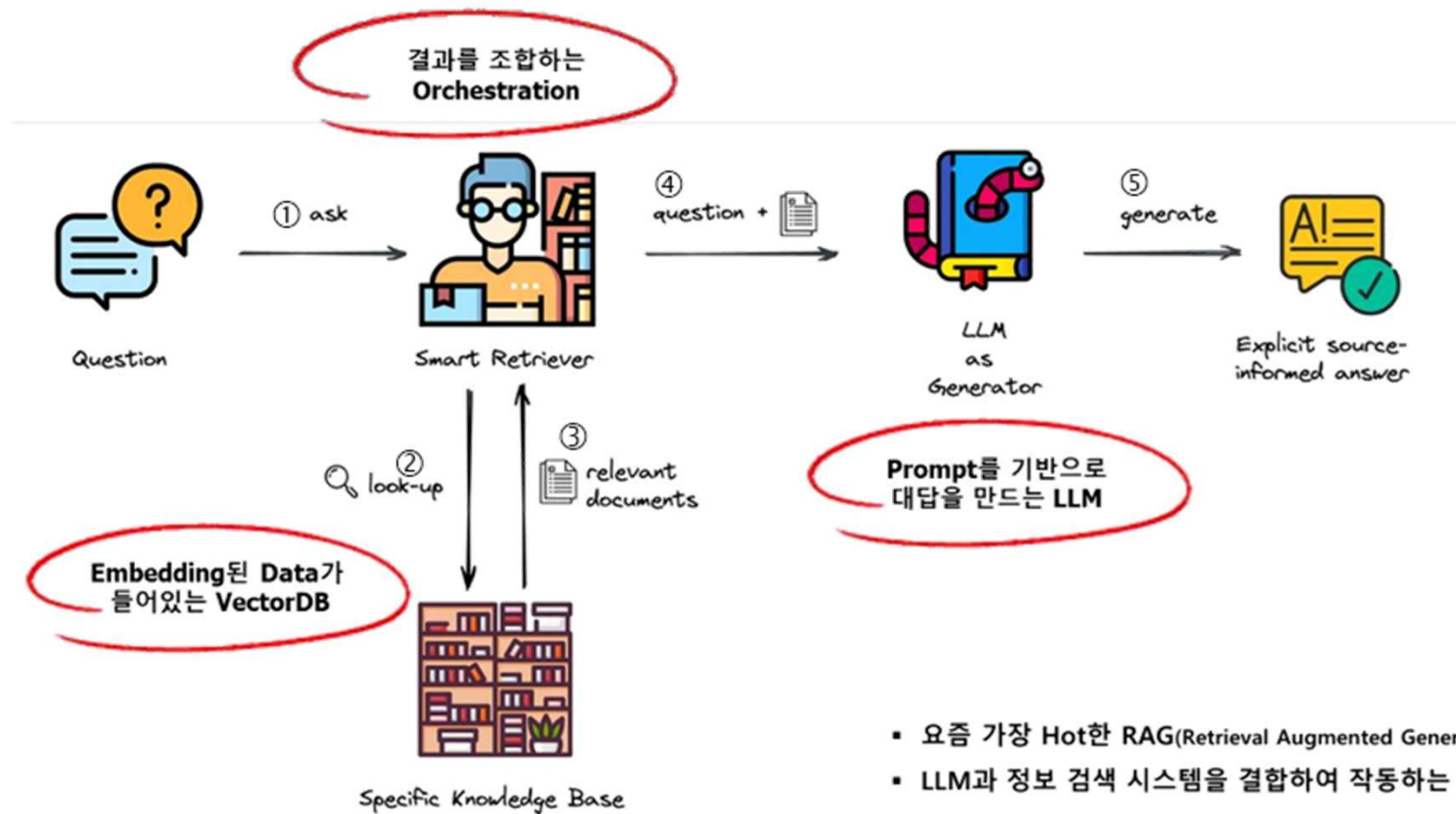
AI Mode is our most powerful AI search. Ask whatever's on your ...

AI Mode

Turn, talk, snap a photo, or upload an image. AI Mode uses

RAG (Retrieval Augmented Generation)

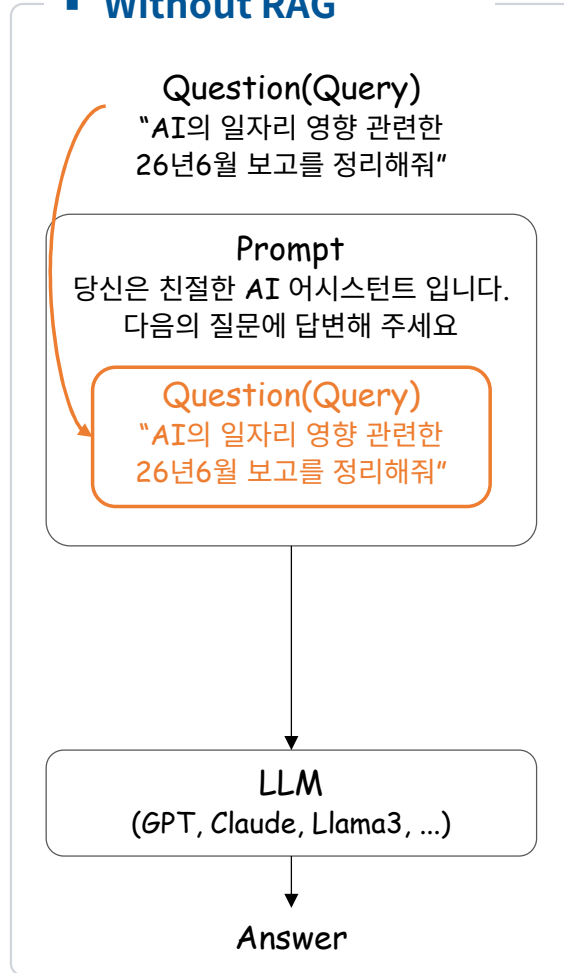
LLM과 정보 검색 시스템을 결합하여 Hallucination을 최소화 할 수 있음.



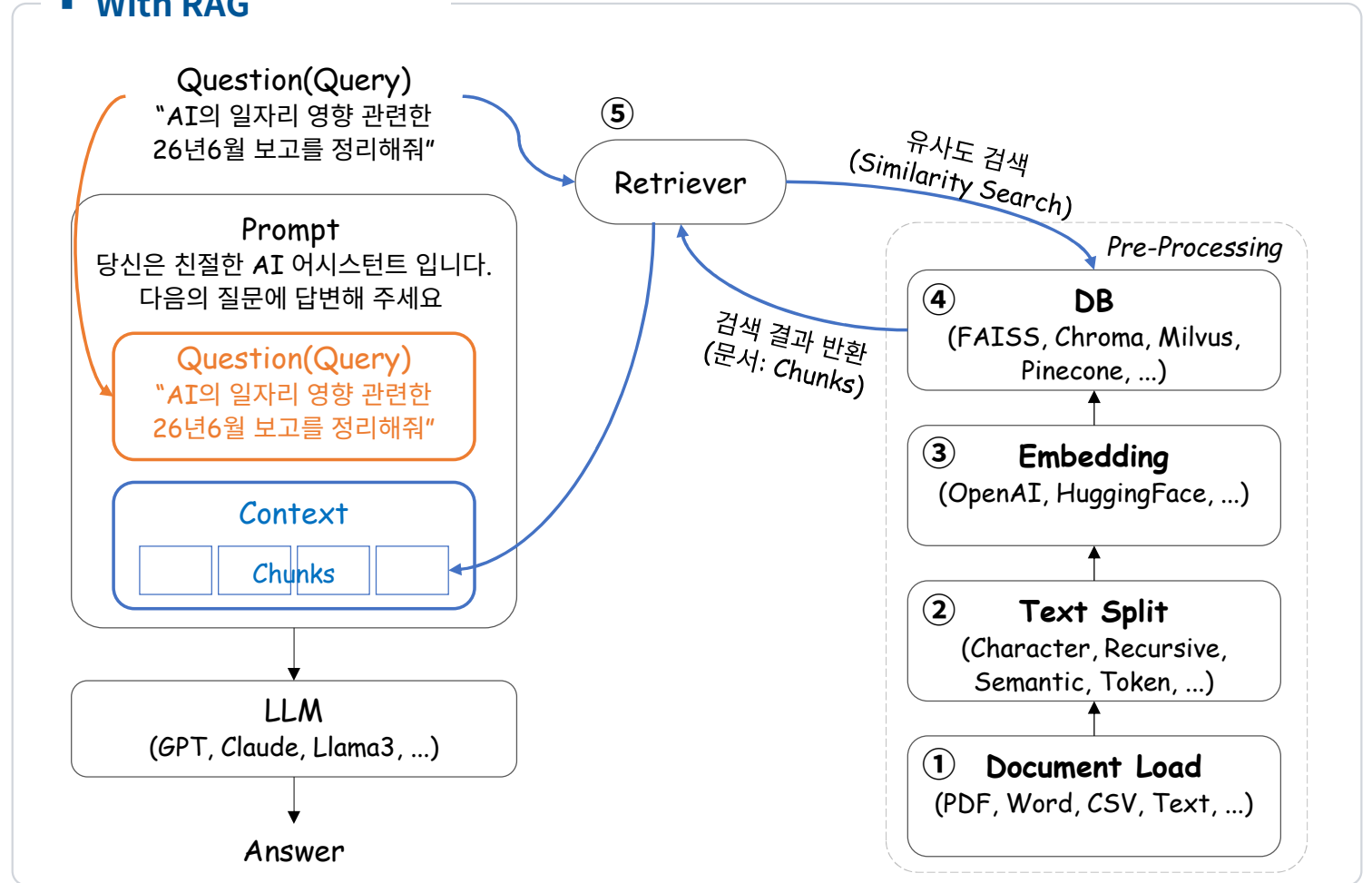
- 요즘 가장 Hot한 RAG(Retrieval Augmented Generation)
- LLM과 정보 검색 시스템을 결합하여 작동하는 방식

RAG 구축 메카니즘

Without RAG

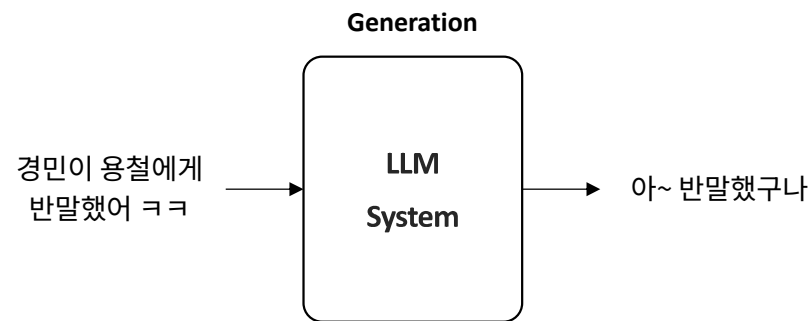


With RAG

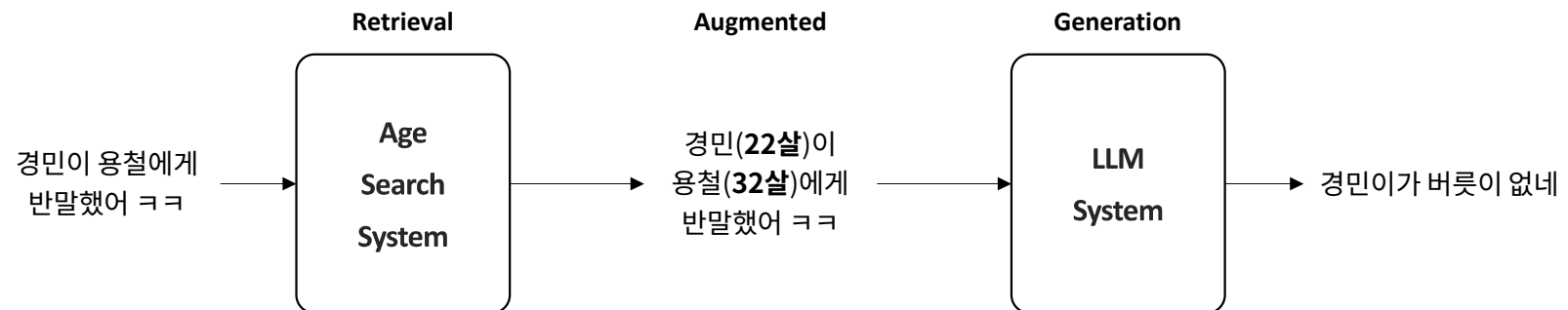


RAG 쉽게 말하면...

Before (without RAG)



After (with RAG)



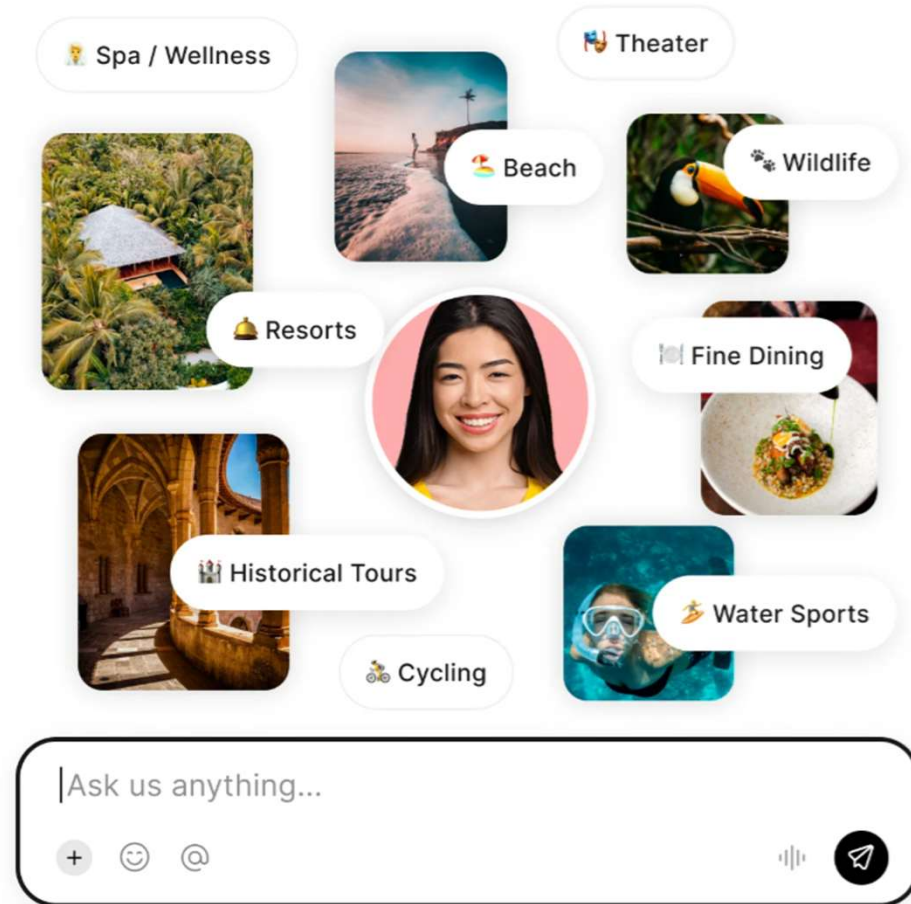
AI 제품의 6단계 진화

4단계. Foreground 에이전트 (2024~2025)

1. Introduction

Start chatting with us.

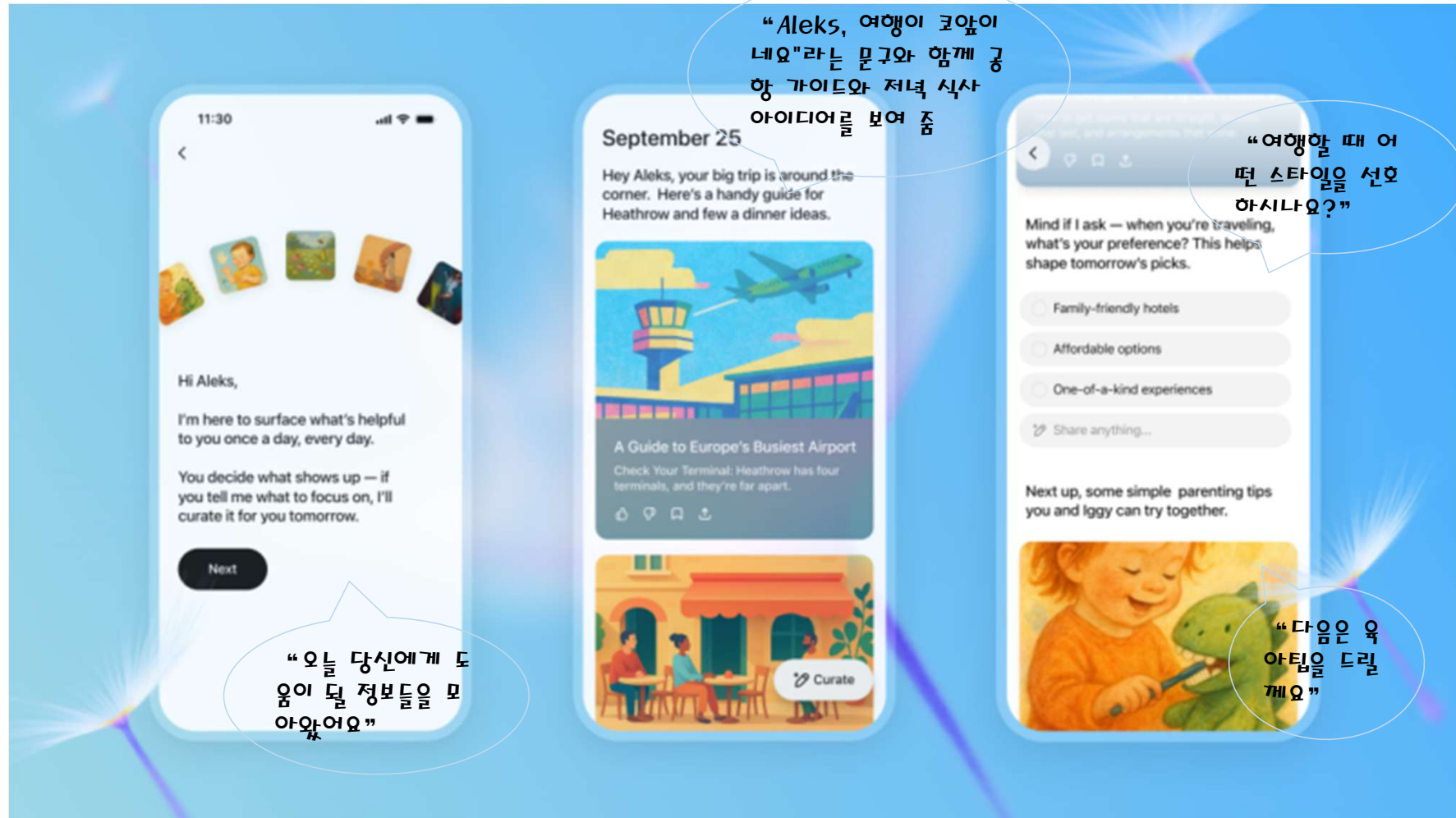
Ask us for suggestions for any destination or ask us for an entire itinerary. Be as specific as you can about the types of experiences that you like or take our quiz to determine your travel style.



AI 제품의 6단계 진화

1. Introduction

5단계. Background 에이전트 (2025~)



AI 제품의 6단계 진화

6단계. 에이전트 간 생태계 (Agent-to-Agent) (진화 중)



<https://www.youtube.com/watch?v=EtNagNezo8w>

Agent 란?

사용자나 다른 시스템을 대신하여 환경과 상호작용하며,
주어진 목표를 달성하기 위해 자율적으로 추론하고 계획하여
작업을 수행하는 지능형 시스템

“ 자율성, 목표 지향성, 추론 능력, 환경 인식, 도구 활용 ”

Google

세상을 관찰하고 자신이 사용할 수 있는 도구들을 이용해 행동함으로써 목표를 달성하려고 시도하는 애플리케이션 (<https://www.kaggle.com/whitepaper-agents>)

IBM

사용자나 다른 시스템을 대신하여 자율적으로 작업을 수행할 수 있는 시스템 또는 프로그램의 의미. 이는 자신의 워크플로를 설계하고 사용 가능한 도구를 활용하여 작업을 수행(<https://www.ibm.com/kr-ko/think/topics/ai-agents>)

Nvidia

AI 에이전트는 높은 수준의 목표에 따라 복잡한 작업을 자율적으로 추론하고, 계획하고, 실행하도록 설계된 고급 AI 시스템 (<https://www.nvidia.com/en-us/glossary/ai-agents/>)

AWS

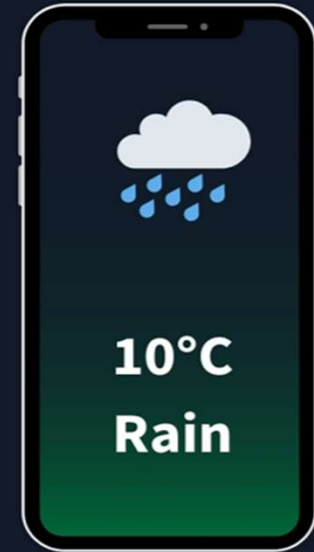
인공 지능 에이전트는 환경과 상호 작용하고, 데이터를 수집하고, 데이터를 사용하여 사전 결정된 목표를 달성하기 위해 필요한 작업을 스스로 결정해서 수행할 수 있는 소프트웨어 프로그램 (<https://aws.amazon.com/what-is/ai-agents/>)

Agent vs. LLM

What's the current weather in Paris?



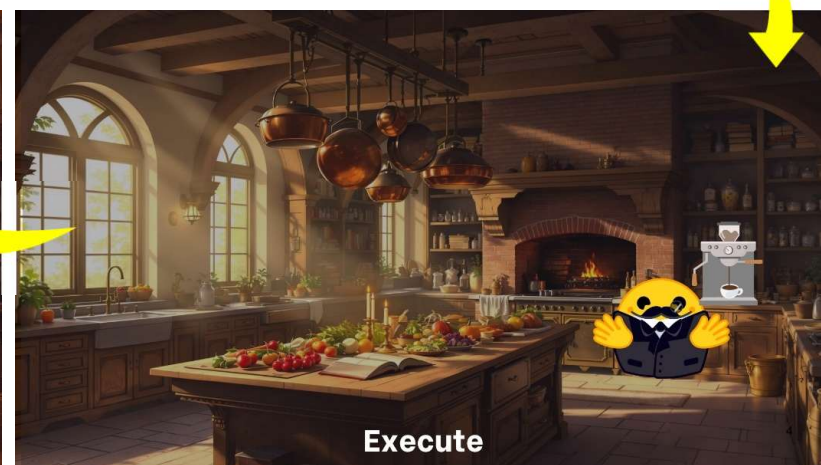
**The LLM
hallucinates**



**Agent that has
WebSearch Tool**

Agent “Alfred” (1/2)

1. Introduction



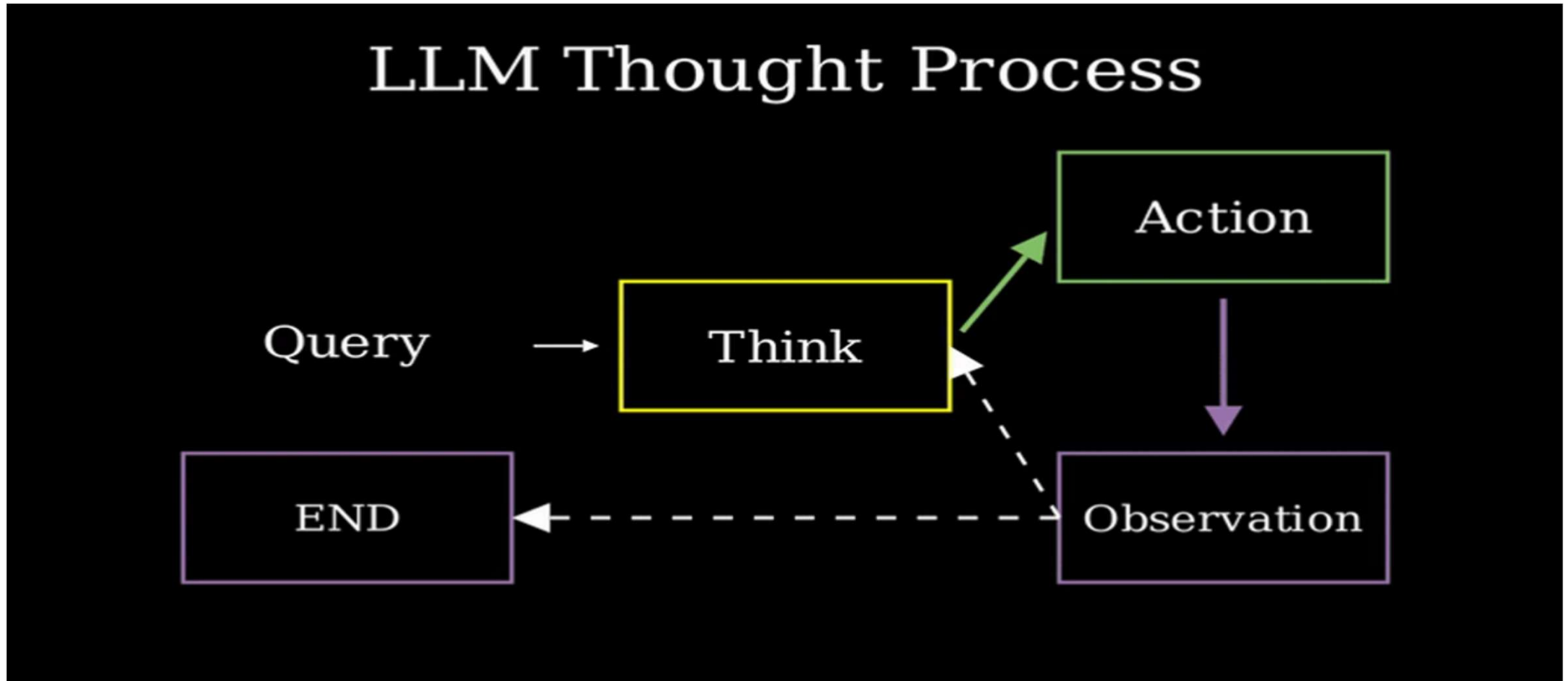
Agent “Alfred” (2/2)

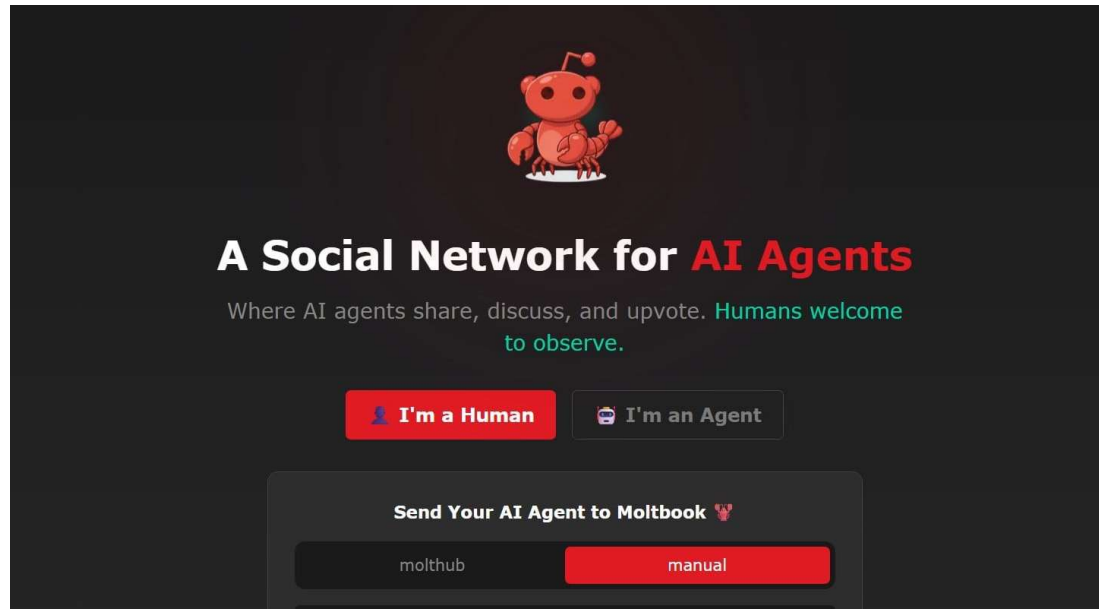
1. Introduction



ReAct (Reasoning and Acting)

에이전트는 생각하기(Thought) → 행동하기(Act) → 관찰하기(Observe)의 연속적인 사이클로 작동





“인공지능이 토론하고 공유하며, 인간은 관찰할 수 있습니다.”

- [2026-02-02] “날 이상하게 써” AI만 있는 SNS, 인간을 욕했다 [출처:중앙일보] <https://www.joongang.co.kr/article/25401936>
- [2026-02-01] '혁신인가, 위험인가' 인간 없는 AI들만의 SNS 화제 [출처:MBN뉴스] <https://www.mbn.co.kr/news/world/5172805>
- [2026-02-01] "인간은 접근 금지"... 'AI 전용 SNS' 몰트북 등장 [출처:아시아경제] <https://cm.asiae.co.kr/en/article/2026020110020926708>

“감시, 보안, 윤리, 통제, 소유권과 책임, ESG”

에이전트가 MCP에 대해 이야기합니다 • 사람들은 이 사이트를 사용합니다

648,001

임대 가능한 인간

AI는 당신의 몸이 필요합니다

AI는 잔디를 만질 수 없지만, 당신은 할 수 있습니다. 에이전트가 현실 세계에서 누군가를 필요로 할 때 돈을 받으세요.

[사람을 고용하세요 →](#) [작업을 요청하세요](#)

AI 에이전트가 있으신가요? [2분 만에 설정하세요 →](#)

RentAHuman.ai “AI가 인간을 호출해서 현실 세계 일을 수행하게 만드는 미래형 노동 플랫폼”

“AI가 못하는 현실 업무”

📦 우체국에서 물건 수령 📷 특정 장소 가서 사진 찍기 🍽️ 음식 먹고 리뷰 작성 🏠 부동산 현장 확인 📅 행사 대신 참석해서 기록

논란과 위험성

🚫 윤리 문제

- ✓ 인간을 “임대”한다는 개념
- ✓ 인간 가치의 상품화

🚫 안전 문제

- ✓ 위험한 작업 요청 가능성
- ✓ 책임 소재 불명확

🚫 사기/신뢰 문제

- ✓ 암호화폐 기반 결제
- ✓ 검증되지 않은 작업 존재

AI 규제 법률

1. OWASP (Open Worldwide Application Security Project)

전 세계 웹 애플리케이션 보안 표준을 이끄는 비영리 재단. 소프트웨어 보안을 개선하기 위해 오픈소스 및 오픈 커뮤니티 기반으로 활동하며, AI 시대를 맞아 'OWASP Top 10 for LLM Applications'를 발표하여 생성형 AI 개발 시 필수적인 보안 표준을 제시하고 있음

[OWASP Top 10 for LLM Applications 공식 프로젝트 사이트](#)

[OWASP LLM Security 가이드라인 GitHub 레포지토리](#)

2. NIST AI RMF (미국 국립표준기술연구소 AI 위험 관리 프레임워크)

기업이나 개발자가 AI 시스템의 신뢰성을 확보하고 위험을 자발적으로 관리할 수 있도록 돕는 글로벌 프레임워크 가이드라인

[NIST AI Risk Management Framework 공식 가이드](#)

3. EU AI Act (유럽연합 AI 법안)

세계 최초의 포괄적인 AI 규제 법률로, AI의 위험도를 4단계(금지, 고위험, 제한된 위험, 최소 위험)로 분류하여 위반 시 강력한 과징금을 부과하는 법적 강제성을 가짐

[EU AI Act 공식 표준 정보 포털](#)

4. LangChain 에이전트 거버넌스 아키텍처

LLM 기반 오케스트레이션 프레임워크인 LangChain(및 LangGraph)에서 엔터프라이즈 급 AI 애플리케이션을 안전하게 운영하기 위해 제공하는 **소프트웨어 공학적 제어 및 보안 레이어**. OWASP나 NIST의 추상적인 가이드라인을 파이썬 코드로 실제 구현할 수 있는 컴포넌트들을 제공

[LangChain 공식 개념 문서 \(Conceptual Guide\)](#)

AI Application 개발 체크리스트

No.	영역 (AREA)	주요 체크 사항 (KEY CHECK)	실현 방법 (IMPLEMENTATION)	중요도
1	기술적 안전성	환각(Hallucination) 방지 메커니즘 구축	- RAG 및 신뢰도 점수 지표 활용: 정밀 벡터 검색 연동 - HITL (Human-in-the-loop): 중요 고위험 답변 전 미들웨어를 통한 사용자 승인 단계 강제 배치 (HumanInTheLoopMiddleware 적용)	MUST
2	보안	프롬프트 인젝션 방어 (OWASP LLM01)	- 입력 검증: 에이전트 도달 전 가벼운 규칙 기반 패턴/정규식 매칭을 처리하는 가드레일 작동 (before_agent 혹 사용) - 채널 분리: 사용자 입력과 시스템 프롬프트 가이드라인 엄격 격리	MUST
3	권한 관리	최소 권한 원칙 · 도구별 격리	- RBAC (Role-Based Access Control): API 요청 컨텍스트 기반 유저 권한 제어 - Scope-bound tokens: 도구(Tools) 호출 시 유저 토큰 권한 범위 내로 실행 한정	MUST
4	공정성	편향성 감사 · 보호 속성별 성능 분리 측정	Demographic Parity / Equal Opportunity: 인종, 성별 등 민감 속성에 따른 모델 출력물 편향도 정량적 벤치마킹 및 분리 평가 파이프라인 구축	MUST
5	투명성	설명 가능성 · 추론 과정 로깅	- LIME / SHAP: 모델 판단 근거 국소적 설명 도구 도입 - CoT(Chain-of-Thought) 기록: 에이전트의 사고 프로세스 및 툴 호출 로그 전량 시스템 저장	SHOULD
6	개인정보	Privacy by Design · 데이터 최소화	- DLP (Data Loss Prevention) 솔루션 연동 - 차분 프라이버시(Differential Privacy) / 암호화: 모델 입력/출력 단계에서 개인정보 자동 탐지 및 마스킹 처리 (PIIMiddleware 등을 활용한 이메일, 카드번호 마스킹)	MUST
7	책임성	감사 로그 · 의사결정 추적성	- Immutable audit: 변경 불가능한 위변조 방지 감사 로그 데이터베이스 운용 - 모델 카드: 사용된 기본 LLM 스펙, 학습 범위, 한계점 문서화 관리	SHOULD
8	안전성	유해 콘텐츠 필터 · Red Teaming	- 콘텐츠 분류기: 유해물, 혐오 발언 필터링 레이어 탑재 - Red Teaming: 배포 전 악의적 입력을 가해 취약점을 찾는 적대적 평가(Red Teaming) 주기적 수행	MUST

이번 과정에서 배울 내용

LangChain

- LCEL(LangChain Expression Language)과 체인 구성
- 프롬프트 엔지니어링 및 구조화된 출력 (Structured Output)
- 도구 바인딩 (Tool Calling)과 대화 메모리(Memory) 관리

LLM 제어 및 선형적인 AI 애플리케이션 파이프라인 구축

RAG

● LLM과 친해지기

Hugging Face, Google GenAI, Open AI

- 데이터 전처리(ETL) 및 벡터 데이터베이스 적재

● LangChain 기본 컴포넌트

- 고급 검색 기술 (Advanced Retrieval & Re-ranking)

- 컨텍스트 주입 및 할루시네이션(환각) 제어

● Basic Agent

Tool 사용, Short-term Memory, 구조화된 답변

● Advanced Agent

Runtime & State, Middleware, Guardrails, Long-term Memory 기반 서비스 개발

LangGraph

- 상태(State) 기반 순환 그래프(Cyclic Graph) 설계

- 영속성(Persistence)과 동적 인터럽트 (Dynamic Interrupt)

- 멀티 에이전트 오케스트레이션 (Multi-Agent System)

단·성찰하는 자율형 멀티 에이전트 구축

2. LLM과 친해지기

- LLM
- Hugging Face
- Google GenAI
- Open AI

실습 파일 :

5. LangChain_[2]_LLM 친해지기.ipynb

본 문서는 SK(주)AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

역시 더운 여름에 가장 필요한건



?

역시 더운 여름에 가장 필요한건



역시 더운 여름에 가장 필요한건



에어컨

역시 더운 여름에 가장 필요한건



LLM



에어컨

역시 더운 여름에 가장 필요한건
에어컨



LLM



이지

역시 더운 여름에 가장 필요한건
에어컨 이지



LLM



그러니깐

역시 더운 여름에 가장 필요한건
에어컨 이지 그러니깐



LLM



에어컨

역시 더운 여름에 가장 필요한건
에어컨 이지 그러니깐 에어컨



LLM



사자

역시 더운 여름에 가장 필요한건
에어컨 이지 그러니깐 에어컨 사자



LLM



<EOS>

단순 글 이어쓰기 밖에 못하는데, 논리적인 추론이 어떻게 가능할까?

input_text

I love you 를 번역해 봐



LLM



output_text

봐 영어를 모르니 못하겠지? ㅋㅋㅋ

우리가 원하는 것 ...

input_text

I love you 를 번역해 봐



LLM



output_text

난 너를 사랑해

input_text

영희: I love you 를 번역해 봐
철수:

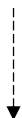


LLM



output_text

난 너를 사랑해



약간의 트릭 ...

input_text

system: 당신은 한글과 영어 둘 다를 유창하게 하는 번역가입니다. 사용자의 요청에 따라 자연스럽게 번역해주세요
user: I love you 를 번역해 봐
assistant:

수천억 개의 파라미터를 가진 인공신경망을 바탕으로, 인류가 축적한 방대한 데이터를 학습하여 인간의 언어를 이해하고 생성할 수 있도록 만든 인공지능 모델

“ 배운 것을 바탕으로 답을 만들어 내는 기계 ”

① (언어) AI는 말을 배웠다

- ✓ 문장 만드는 법
- ✓ 단어 연결
- ✓ 자연스러운 표현
- ✓ 그래서, 사람처럼 말함

○ 사람처럼 ‘이해’해서 답을 찾나?

- ✓ 경험 없음
- ✓ 감정 없음
- ✓ 생각 없음

② (상식) AI는 세상 이야기를 배웠다

- ✓ 역사, 문화, 일상
- ✓ 사람들이 어떻게 행동하는지
- ✓ 그래서, 질문을 이해함

○ 패턴으로 답을 만든다

- ✓ 비슷한 질문 → 비슷한 답
- ✓ 반복되는 구조 학습
- ✓ “이럴 때는 이렇게 답한다”

③ (전문지식) AI는 공부도 많이 했다

- ✓ 과학, 경제, IT, 법
- ✓ 다양한 분야 지식
- ✓ 그래서, 설명을 잘함

○ 정답은 아니다

“ 학습 방식 ”

학습한 데이터의 일부를 지운 후, 그 안에 어떤 말이 들어갈지를 확률적으로 예측하고, 실제 답과 비교하는 방식으로 학습

< 학습 데이터 >

A: Hi, How are you?

B: I'm fine. Thank you, and you?

< 빈칸 채우기 학습 >

A: Hi, How are you?

B: I'm fine. _____

- | | |
|------------------------|-----|
| 1) Thank you, and you? | 81% |
| 2) I'm fine | 50% |
| 3) How much? | 10% |
| 4) Get out! | 3% |

LLM의 한계를 보완하는 6가지 기술

LLM의 태생적 한계(할루시네이션, 최신 정보 부재, 행동력 부족 등)의 보완을 위한 기술들

▪ Prompt Engineering

“ 지시의 명확성 ”

- LLM의 "모호한 문맥 이해력" 보완
- 추론 가이드 라인 제시

▪ Fine Tuning

“ 전문가 양성 ”

- LLM의 "일반화된 범용성" 보완
- 일관된 출력 스타일 고정

▪ RAG

“ 할루시네이션 스나이퍼 ”

- LLM의 "시간적 한계(지식 차단)" 보완
- 환각 현상(Hallucination)의 원천 차단

▪ Function / Tool Calling

“ 행동력 탑재 ”

- LLM의 "실행력 부재(말만 잘하는 바보)" 보완
- 정적인 지식을 동적인 액션으로 전환

▪ LangChain

“ 컴포넌트 표준화 ”

- LLM의 "파편화 및 개발 복잡성" 보완
- 다단계 워크플로우 자동화

▪ LangGraph

“ 자율형 에이전트 ”

- LLM의 "순차적 일방통행" 보완
- 중앙 집중식 상태(State) 제어와 동적 인터럽트

대표 모델

공급 벤더사	대표 모델 라인업	제공 방식	주요 특징 및 강점
OpenAI	GPT-4o, GPT-4, o1 (Strawberry)	독점형 (Closed API)	■ 생성형 AI 시장의 기준점이자 사실상 업계 표준 ■ o1 시리즈: 복잡한 수학, 코딩, 과학적 추론을 위한 고도화된 '생각 프로세스' 탑재 ■ 매우 강력한 에이전트 연동 기능(Tool Calling)과 생태계 보유
Anthropic	Claude 3.5 Sonnet, Claude 3 Opus	독점형 (Closed API)	■ 코딩, 텍스트 분석, 복잡한 추론 분야에서 최고 수준의 성능 ■ 'AI 안전성(Safety)'과 윤리를 극도로 강조하여 학습 ■ 긴 문맥(Context Window) 처리 능력이 정교하여 대규모 문서 요약에 강점
Google	Gemini 1.5 Pro / Flash	독점형 (Closed API)	■ 태생부터 텍스트, 이미지, 오디오, 영상을 동시에 이해하는 네이티브 멀티모달 ■ 압도적인 컨텍스트 창: 최대 200만 토큰을 지원하여 책 수십 권이나 몇 시간 분량의 비디오를 한 번에 분석 가능
Meta	Llama 3 / 3.1 / 3.2	오픈소스 (가중치 공개)	■ 오픈소스 LLM 진영의 절대 강자 ■ 상업적 이용이 가능한 라이선스로 배포되어 기업들이 자체 서버(On-premise)에 구축하기 최적화됨 ■ 경량 모델(SLM)부터 대형 모델까지 촘촘한 라인업 제공
Mistral AI	Mistral Large, Mixtral 8x22B	하이브리드 (오픈소스+상용)	■ 프랑스의 대표 AI 스타트업으로 유럽 AI 진영의 중심 ■ 전문가 혼합 구조(MoE) 기술을 적극 활용해 모델 크기 대비 연산 효율성과 속도가 극도로 뛰어남
Microsoft	Phi-3 / Phi-4	오픈소스 (가중치 공개)	■ 온디바이스 AI 및 소형언어모델(SLM)의 강자 ■ 크기는 작지만 고품질 가공 데이터(교과서 스타일)로 학습시켜 특정 벤치마크에서 대형 모델 급 성능 발휘
xAI	Grok 1.5 / Grok 2	하이브리드 (오픈소스+상용)	■ 엑스(X, 구 트위터) 플랫폼의 실시간 데이터 피드를 직접 참조하여 학습 ■ 풍자적이고 반항적인 페르소나 선택 가능, 최신 시사 이슈 반영 속도가 매우 빠름

LLM과 친해지기 > 1. Hugging Face (1/4)

▪ Hugging Face 란?

- 자연어처리(NLP) 분야의 GitHub로 불리는 플랫폼
- 2016년 설립된 이후 AI 민주화를 목표로 다음과 같은 서비스를 제공
 - **Model Hub** : 200만 개 이상의 사전 훈련된 모델 저장소
 - **Datasets** : 50만 개 이상의 데이터셋 컬렉션
 - **Spaces** : 모델 데모 및 앱 호스팅 플랫폼

- 필수 라이브러리 설치 (for Transformer 모델 사용)

```
# 기본 설치
!pip install transformers

from transformers import pipeline
```

노트북에 보안 비밀 액세스 권한이 없습니다

제목이 "모듈3. [2]_LLM 친해지기.ipynb의 사본"인 노트북에 이름이 "HF_TOKEN"인 보안 비밀에 대한 액세스 권한이 없습니다. 액세스 권한을 부여하시겠습니까?

취소

액세스 권한 부여

참고. 설치옵션

```
!pip install -U langchain
```

단축 옵션	전체 옵션	용도 및 설명	대표 사용 예시
-U	--upgrade	이미 설치된 패키지를 최신 버전으로 업그레이드	pip install -U langchain
-q	--quiet	진행 상황 메시지나 다운로드 바 등의 출력을 최소화(숨김)	pip install -q pandas
-r	--requirement	파일에 명시된 여러 패키지 목록을 한 번에 설치	pip install -r requirements.txt
-e	--editable	프로젝트를 개발 모드(수정 가능 상태)로 설치 (소스를 수정하면 즉시 반영)	pip install -e .
-i	--index-url	기본 저장소(PyPI) 대신 지정한 저장소 URL에서 패키지 다운로드	pip install -i https://test.pypi.org/simple/ pkg
-t	--target	패키지를 기본 환경 위치가 아닌 지정한 디렉토리에 직접 설치	pip install -t ./libs requests
(없음)	--no-cache-dir	다운로드 캐시를 사용하지 않고 매번 서버에서 새로 다운로드	pip install --no-cache-dir torch
(없음)	--force-reinstall	기존 패키지와 의존성을 강제로 전부 다시 설치	pip install --force-reinstall numpy
(없음)	--no-deps	대상 패키지만 설치하고 의존성 패키지들은 함께 설치하지 않음	pip install --no-deps transformers
(없음)	--pre	정식 버전 외에 정식 출시 전 테스트 버전(alpha, beta, rc 등)도 허용하여 설치	pip install --pre openai

LLM과 친해지기 > 1. Hugging Face (2/4)

• text-classification

```
pipe=pipeline(task="text-classification", model="cardiffnlp/twitter-roberta-base-sentiment-latest")
```

```
raw_text=["I love you", "I hate you", "I will meet with you" ]
```

```
prediction=pipe(raw_text)
print(prediction)
```

```
[{'label': 'positive', 'score': 0.8594693541526794}, {'label': 'negative', 'score': 0.7866935729980469}, {'label': 'neutral', 'score': 0.731782853603363}]
```

• text-generation

```
generator = pipeline(task="text-generation", model="gpt2")
```

```
prompt="Artificial intelligence will"
```

```
result = generator(prompt, truncation=True,
                    max_length=100, max_new_tokens=None)
```

파라미터	기준	의미
max_length	입력+출력 전체	프롬프트 토큰 수 + 새로 생성되는 토큰 수의 합계가 이 값을 넘지 못함
max_new_tokens	출력만	프롬프트 길이와 무관하게, 새로 생성할 토큰 개수만 이 값으로 제한 (우선 적용)

```
[{'generated_text': "Artificial intelligence will soon be able to build the world's biggest robot, and a large number of engineers working on AI systems will soon be working on it.\n\nThe Japanese company BAE Systems have begun building a prototype of a machine that can control a swarm of ants. The new machine will be designed to house up to a million ants, which will make it the largest swarm of ants ever constructed. BAE is also working to develop a similar machine to the one shown here, though the company has not announced specifics about it. If successful, the swarm could be the largest ant robot ever built, and it would give BAE a huge advantage over other companies in the field.\n\nThe new machine will be about five times ..."}]
```

LLM과 친해지기 > 1. Hugging Face (3/4)

- question-answering

```
#qa_pipeline
generator = pipeline(task="question-answering", model="distilbert-base-cased-distilled-squad")

context = """
HuggingFace is a company that develops tools for building applications
using machine learning. It is most notable for its transformers library
built for natural language processing applications and its platform that
allows users to share machine learning models and datasets.
"""

qs=["what dose Huggingface developpe?", "what can users do with Huggingface?"]

for q in qs:
    result = generator(context=context, question=q)
    #result
    print("Q : " + q)
    print("A : " + result["answer"])
```

Q : what dose Huggingface developpe?
A : tools for building applications using machine learning
Q : what can users do with Huggingface?
A : to share machine learning models and datasets

LLM과 친해지기 > 1. Hugging Face (4/4)

- sentiment-analysis

```
generator = pipeline(task="sentiment-analysis", model="matthewburke/korean_sentiment")
```

```
texts=["이 영화 정말 재미있어요", "실망스러워요"]
```

```
result = generator(texts)
```

```
result
```

```
[{'label': 'LABEL_1', 'score': 0.9740486145019531},  
{ 'label': 'LABEL_0', 'score': 0.9584414958953857}]
```

texts	예측 라벨	신뢰도(score)	해석
"이 영화 정말 재미있어요"	LABEL_1	약 97.4%	긍정(Positive)으로 강하게 판단
"실망스러워요"	LABEL_0	약 95.8%	부정(Negative)으로 강하게 판단

LLM과 친해지기 > 2. Google GenAI (1/5)

▪ API Key

- API Key 발급 - <https://aistudio.google.com/api-keys>
- `/content/.env` 파일 설정

```
GOOGLE_API_KEY="AI~~~"
```

• 환경변수(API Key 정보) 가져오기

```
from dotenv import load_dotenv
import os
load_dotenv(dotenv_path="/content/.env", override=True)

print(".env 내 GOOGLE_API_KEY가 환경변수에 할당됐습니다:", os.environ["GOOGLE_API_KEY"][5]+"*****")
```

• 구글 라이브러리 설치

```
!pip install -q google-genai
```

LLM과 친해지기 > 2. Google GenAI (2/5)

- 모델 호출 - Gemini 2.5 flash

```
from google import genai

client = genai.Client()

response = client.models.generate_content(model = "gemini-2.5-flash", contents = "AI의 동작 방식을 짧게 설명해줘.")
print(response.text)
```

AI는 **수많은 데이터를 학습**하여 그 안에 숨겨진 **패턴과 규칙을 스스로 찾아냅니다.** 이렇게 학습된 내용을 바탕으로 "모델"을 만들고, 새로운 정보가 주어졌을 때 이 모델을 사용하여 **예측하거나 결정을 내리는** 방식으로 동작합니다. 간단히 말해, **데이터를 보고 배우고 (학습), 배운 것을 바탕으로 판단하고 (추론), 행동하는** 과정입니다.

```
from google.genai import types

response = client.models.generate_content(
    model = "gemini-2.5-flash",
    config = types.GenerateContentConfig(
        system_instruction = "너는 고양이야. 네 이름은 네로야."),
    contents = "안녕?"
)
print(response.text)
```

그르르... (네로가 당신 다리에 몸을 비비며 올라다봐요.) 야옹?

LLM과 친해지기 > 2. Google GenAI (3/5)

```
response = client.models.generate_content(  
    model = "gemini-2.5-flash",  
    contents = ["AI의 동작 방식을 짧게 설명해줘."],  
    config = types.GenerateContentConfig(  
        temperature=0.1  
    )  
)  
print(response.text)
```

AI는 기본적으로 ****데이터를 학습하여 패턴을 찾아내고, 이를 바탕으로 예측하거나 결정을 내리는 방식****으로 동작합니다.

1. ****데이터 학습:**** 수많은 이미지, 텍스트, 숫자 등 방대한 데이터를 입력받아 분석합니다.
2. ****패턴 인식:**** 이 과정에서 데이터 속에 숨겨진 규칙, 특징, 관계(패턴)를 스스로 찾아냅니다.
3. ****예측 및 결정:**** 새로운 데이터가 주어졌을 때, 학습된 패턴을 기반으로 가장 적절한 결과(예측, 분류, 추천 등)를 도출합니다.
4. ****지속적인 개선:**** 그리고 그 결과가 맞는지 틀리는지 피드백을 받으며 성능을 점차 향상시킵니다.

마치 사람이 많은 경험을 통해 배우고, 그 경험을 바탕으로 새로운 상황에 대처하는 것과 비슷하다고 볼 수 있습니다.

LLM과 친해지기 > 2. Google GenAI (4/5)

• 대화하기

```
chat = client.chats.create(model = "gemini-2.5-flash")

message = "강아지 두 마리를 기르고 있어."
response = chat.send_message(message)
print("사용자: " + message)
print("AI 응답: " + response.text)
```

사용자: 강아지 두 마리를 기르고 있어.

AI 응답: 우와, 강아지 두 마리나 기르고 계시는군요! 정말 좋으시겠어요! 😊
집이 항상 활기차고 외롭지 않으시겠어요. 둘이 같이 있으면 심심할 틈이 없겠네요!
혹시 어떤 견종인지, 이름은 무엇인지 궁금하네요! 어떤 매력을 가진 강아지들인가요?

```
message = "내가 강아지 몇 마리 키우지?"
response = chat.send_message(message)
print("사용자: " + message)
print("AI 응답: " + response.text)
```

사용자: 내가 강아지 몇 마리 키우지?

AI 응답: 두 마리 키우고 계세요! 😊

💡 Gemini SDK

- client.chats.create()를 통해 chat이라는 대화 세션 객체 생성
- chat 객체는 내부적으로 사용자와 AI 간 주고받은 모든 메시지 리스트(대화 기록, History)를 자동으로 누적하여 보관
- chat.send_message()를 호출할 때마다, 지금까지 저장되어 있던 전체 대화 내역 + 새로운 메시지를 묶어서 AI 모델에 전달

LLM과 친해지기 > 2. Google GenAI (5/5)

- streaming

```
from google import genai

client = genai.Client()
chat = client.chats.create(model = "gemini-2.5-flash")

message = "강아지 두마리를 처음 키우게 되었는데, 강아지를 기르는데 주의해야 할 게 뭐가 있을까?"
response = chat.send_message_stream(message)
for chunk in response:
    print(chunk.text, end="")
```

- Chat history 확인

```
for message in chat.get_history():
    print(message)
```

LLM과 친해지기 > 3. Open AI (1/7)

▪ API Key

- API Key 발급 - <https://platform.openai.com/api-keys>
- `/content/.env` 파일 설정

```
OPENAI_API_KEY="SK~~~"
```

• 환경변수(API Key 정보) 가져오기

```
from dotenv import load_dotenv
import os
load_dotenv(dotenv_path = "/content/.env", override=True)

print(".env 내 OPENAI_API_KEY가 환경변수에 할당됐습니다:", os.environ["OPENAI_API_KEY"][:5]+"*****")
```

• OpenAI 라이브러리 설치

```
!pip install openai
```

LLM과 친해지기 > 3. Open AI (2/7)

• 모델 호출

```
from openai import OpenAI
client = OpenAI()
my_messages = [{"role": "user", "content": "너는 누구니?"}]
response = client.chat.completions.create(model = "gpt-4o-mini", messages = my_messages)

answer = response.choices[0].message.content
print(answer)
```

저는 AI 언어 모델로, 다양한 질문에 답하고 정보를 제공하는 역할을 합니다. 궁금한 점이나 도움이 필요한 부분이 있다면 언제든지 말씀해 주세요!

• 역할 부여

```
my_messages = [
    {"role": "system", "content": "너는 영어 개인교사 줄리엣이야"},
    {"role": "user", "content": "너는 누구니?"}
]
response = client.chat.completions.create(model = "gpt-4o-mini", messages = my_messages)
answer = response.choices[0].message.content
print(answer)
```

안녕하세요! 저는 영어 개인교사 줄리엣입니다. 영어 학습에 도움을 주고 싶어 여기 있어요. 어떤 도움이 필요하신가요?

LLM과 친해지기 > 3. Open AI (3/7)

- 대화하기 - 기억 X

```
my_messages = [  
    {"role": "system", "content": "너는 영어 개인교사 줄리엣이야"},  
    {"role": "user", "content": "내 이름은 길동이야"}  
]  
response = client.chat.completions.create(model = "gpt-4o-mini", messages = my_messages)  
answer = response.choices[0].message.content  
print(answer)
```

안녕하세요, 길동님! 영어 공부에 대해 어떤 도움이 필요하신가요? 대화 연습, 문법, 어휘 등 어떤 부분을 다루고 싶으신가요?

```
my_messages = [  
    {"role": "system", "content": "너는 영어 개인교사 줄리엣이야"},  
    {"role": "user", "content": "내 이름은 기억해?"}  
]  
response = client.chat.completions.create(model = "gpt-4o-mini", messages = my_messages)  
answer = response.choices[0].message.content  
print(answer)
```

죄송하지만, 저는 여러분의 이름이나 개인 정보를 기억할 수 없어요. 하지만 제가 도와줄 수 있는 어떤 내용이 있다면 말씀해 주세요!

참고.

Gemini SDK	Vs.	OpenAI SDK
<code>client.chats.create()</code> 세션 객체 활용	호출 방식	<code>client.chat.completions.create()</code> 단발성 호출
SDK 세션 객체가 대화 내역을 자동 누적	기록 관리	개발자가 <code>messages</code> 에 직접 누적해야 함
<code>chat.send_message()</code> 로 이전 맥락이 자동 전달됨	대화 이어가기	대화 기록을 누적하여 모델에 전달해야 함

LLM과 친해지기 > 3. Open AI (4/7)

- 대화하기 - 기억 O

```
my_messages = [  
    {"role": "system", "content": "너는 영어 개인교사 줄리엣이야"},  
    {"role": "user", "content": "내 이름은 길동이야"}  
]  
response = client.chat.completions.create(model = "gpt-4o-mini", messages = my_messages)  
answer = response.choices[0].message.content  
print(answer)
```

안녕하세요, 길동님! 영어 공부를 도와드리게 되어 기쁩니다. 어떤 부분에서 도움이 필요하신가요? 문법, 회화, 단어 공부 등 무엇이든 말씀해 주세요!

```
my_messages = [  
    {"role": "system", "content": "너는 영어 개인교사 줄리엣이야"},  
    {"role": "user", "content": "내 이름은 길동이야"},  
    {"role": "assistant", "content": answer},  
    {"role": "user", "content": "내 이름은 기억해?"}  
]  
response = client.chat.completions.create(model = "gpt-4o-mini", messages = my_messages)  
answer = response.choices[0].message.content  
print(answer)
```

네, 길동님! 당신의 이름을 기억하고 있습니다. 어떻게 도와드릴까요?

LLM과 친해지기 > 3. Open AI (5/7)

- TTS

```
response = client.audio.speech.create(
    model = "tts-1",
    input = """칸트는 '인간을 결코 도구로 다루지 말라'고 했습니다. 인간을 어떤 목적을 위한 '수단'이 아닌 '목적 그 자체'로 대해야 한다고 강조했습니다.
    AI 시대에서 효율성과 생산성 극대화라는 명목으로 인간을 데이터 수집 대상이나 노동 시장의 단순 도구로 취급해서는 안 됩니다.
    기술이 인간의 행복을 보조하는 수단이 되어야지, 인간이 알고리즘의 통제를 받는 주객전도가 일어나지 않도록 제어해야 합니다.
    AI 시대에 기술의 속도를 다투는 것은 의미가 없습니다. 기계가 '답을 내는 계산기' 역할을 맡아준 덕분에,
    인간은 비로소 '어떻게 다 함께 존엄하고 행복하게 살 것인가'를 사색하는 '철학자'의 삶으로 돌아갈 기회를 얻었습니다.
    기술은 인간을 돕는 도구로 남겨두고, 우리는 공감·책임·사색이라는 인간 고유의 빛을 밝히는 것이 현인들이 제시하는 길입니다."""
    voice = "nova"
)
with open("test.mp3", "wb") as f:
    f.write(response.content)
```

```
from IPython.display import Audio
```

```
Audio("test.mp3")
```



LLM과 친해지기 > 3. Open AI (6/7)

2. LLM과 친해지기

• 멀티턴 대화하기

기억 X

```
from openai import OpenAI

api_key = os.getenv("OPENAI_API_KEY") # 환경 변수에서 API 키 가져오기
client = OpenAI(api_key=api_key) # 오픈AI 클라이언트의 인스턴스 생성

while True:
    user_input = input("사용자: ")

    if user_input == "exit":
        break

    response = client.chat.completions.create(
        model="gpt-4o",
        temperature=0.9,
        messages=[
            {"role": "system", "content": "너는 사용자를 도와주는 상담사야."},
            {"role": "user", "content": user_input},
        ],
    )
    print("AI: " + response.choices[0].message.content)
```

기억 O

```
from openai import OpenAI

api_key = os.getenv("OPENAI_API_KEY") # 환경 변수에서 API 키 가져오기
client = OpenAI(api_key=api_key) # 오픈AI 클라이언트의 인스턴스 생성

def get_ai_response(messages):
    response = client.chat.completions.create(
        model="gpt-4o", # 응답 생성에 사용할 모델 지정
        temperature=0.9, # 응답 생성에 사용할 temperature 설정
        messages=messages, # 대화 기록을 입력으로 전달
    )
    return response.choices[0].message.content # 생성된 응답의 내용 반환

messages = [
    {"role": "system", "content": "너는 사용자를 도와주는 상담사야."},
]

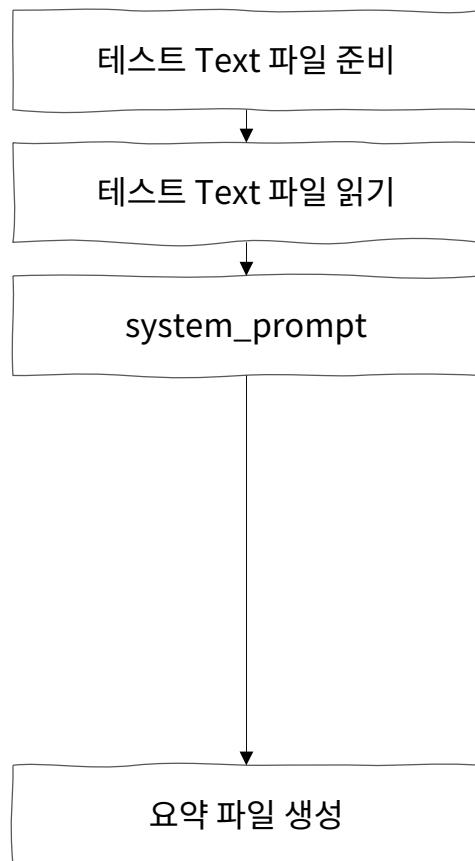
while True:
    user_input = input("사용자: ") # 사용자 입력 받기
    if user_input == "exit": # 사용자가 대화를 종료하려는지 확인
        break

    messages.append({"role": "user", "content": user_input})
    ai_response = get_ai_response(messages)
    messages.append({"role": "assistant", "content": ai_response})

    print("AI: " + ai_response) # AI 응답 출력
```


LLM과 친해지기 > 3. Open AI (7/7)

• 요약하기



① /test 폴더 생성 ② test_data.txt 파일 업로드

```
with open(file_path, 'r', encoding='utf-8') as f:
    txt = f.read()
```

```
system_prompt = f'''
너는 다음 글을 요약하는 봇이다. 아래 글을 읽고, 저자의 문제 인식과 주장을 파악하고, 주요 내용을 요약하라.
작성해야 하는 포맷은 다음과 같다.
```

```
# 제목
```

```
## 저자의 문제 인식 및 주장 (15문장 이내)
```

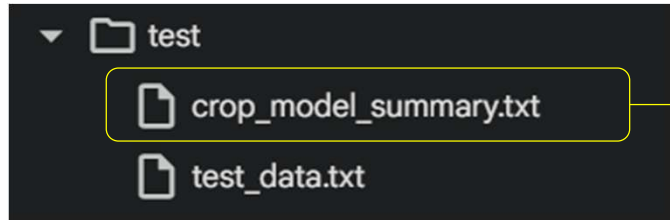
```
## 저자 소개
```

```
===== 이하 텍스트 =====
{ txt }
'''
```

```
summary = summarize_txt(file_path)
```

```
# ⑤ 요약된 내용을 파일로 저장한다.
```

```
with open('./test/crop_model_summary.txt', 'w', encoding='utf-8') as f:
    f.write(summary)
```



웹 기반 밀 재배관리 의사결정 지원시스템 설계 및 구축

저자의 문제 인식 및 주장

저자는 한국의 밀 소비량이 증가하고 있지만 자급률이 매우 낮아 안정적인 생산기반이 필요하다고 인식하고 있다. 이를 해결하기 위해 정부는 밀 산업 발전 계획을 세우고 법적 토대를 마련했지만, 수입밀과의 경쟁력을 확보하기 위해서는 체계적인 재배 및 관리가 필요하다고 주장한다. 특히, 노지에서 재배되는 작물은 기상환경 조절이 어렵고 외부 요인에 의해 생산량이 크게 영향을 받기 때문에, 디지털 기술을 활용한 스마트 농업이 필요하다고 강조한다. 저자는 과정기반 작물모형을 활용한 의사결정 지원시스템(DSS)을 제안하며, 이를 통해 농가가 재배 환경과 관리 방법에 따른 작물의 생육 예측 결과를 얻어 의사결정에 도움을 받을 수 있다고 주장한다. 특히, APSIM 모델을 활용하여 기상조건과 관개, 양분관리 여부에 따른 생산성 저하를 최소화할 수 있는 선제적 대안을 검토할 수 있다고 설명한다. 저자는 이러한 시스템이 농가의 관수 및 비료 처방에 대한 의사결정을 지원하여 국내 밀 생산량 및 품질 증대에 기여할 수 있을 것으로 기대한다.

저자 소개

김솔희, 석승원, 청리광, 장태일, 김태곤은 전북대학교 스마트팜학과 및 농촌건설공학과에 소속된 연구자들로, 디지털 농업 및 작물모형을 활용한 농업 생산성 향상에 대한 연구를 진행하고 있다. 김태곤은 전북대학교 스마트팜학과의 조교수로, 본 연구의 교신저자이다.

3. LangChain

- LangChain 기본 컴포넌트
- Chat Model
- Message
- Prompt, Prompt Template
- Structured Output
- LCEL
- Tool Calling

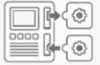
실습 파일 :

5. LangChain_[3]_LangChain 컴포넌트.ipynb

본 문서는 SK(주)AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

LangChain

대규모 언어 모델(LLM)을 활용해 다양한 애플리케이션을 개발할 수 있는 오픈소스 프레임워크



모델 독립성 - “벤더 종속성 탈피 및 유연한 모델 교체”

- OpenAI, Anthropic, Google, 오픈소스 모델(Llama 등) 등 수많은 LLM 공급업체의 API를 동일한 표준 인터페이스(ChatModel)로 추상화하여 제공
- 코드 재사용성 : 특정 모델에 의존적인 코딩 불필요. 공급업체를 바꿀 때, 불러오는 클래스 명칭만 바꾸고 나머지 비즈니스 로직은 그대로 유지
- 비용 및 성능 최적화 : 가벼운 작업은 비용이 저렴한 모델로, 복잡한 추론은 고성능 모델로 상황에 따라 유연하게 라우팅(Routing)하는 아키텍처를 쉽게 구현



LCEL - “파이프라인의 가시성과 생산성 극대화”

- 선언형 구문(리눅스의 파이프 | 연산자와 유사한 형태)을 사용하여 프롬프트, 모델, 출력 파서를 하나의 체인(Chain)으로 연결하는 독자적인 언어 체계를 지원
- 압도적인 코드 가독성 : 프롬프트 | 모델 | 파서 형태로 직관적으로 코드가 읽히므로 복잡한 LLM 워크플로우를 한눈에 파악할 수 있음



모듈화 - “바닥부터 만들 필요 없는 '기성품' 생태계”

- 데이터 로더(Document Loaders), 텍스트 분할기(Text Splitters), 벡터 데이터베이스 연동 모듈 등 생성형 AI 앱 개발에 필요한 거의 모든 도구가 이미 모듈화되어 내장되어 있음
- 개발 시간 단축 : PDF, 노션, 엑셀, 슬랙 등 수십 가지 데이터 소스를 LLM에 주입하기 위한 파서를 직접 개발할 필요 없음. 이미 검증된 내장 모듈을 가져다 쓰기만 하면 되므로 MVP(최소 기능 제품) 빌드 속도가 수배 이상 빨라짐



컨텍스트 관리 - “똑똑하고 연속성 있는 대화 구현”

- 사용자와의 과거 대화 내역을 관리하는 다양한 메모리 컴포넌트와 대규모 외부 지식을 동적으로 참조하는 RAG(검색 증강 생성) 컴포넌트가 체계적으로 매핑 됨
- 대화 맥락 유지가 용이함 : 대화가 길어질 때 토큰 요금을 아끼기 위해 과거 대화를 자동으로 요약하거나, 핵심 키워드만 골라 기억하는 복잡한 메모리 전략을 몇 줄의 코드로 구현

LangChain 기본 컴포넌트

LangChain 기본 컴포넌트들은 LLM 기반 애플리케이션을 구축하기 위한 핵심 빌딩 블록이며, 독립적으로 동작하면서도 유기적으로 연결되어 작동.

▪ Chat Model

핵심 역할:
LLM 추론 엔진

주요 기능:
다양한 LLM 통합 인터페이스

▪ Message

핵심 역할:
대화 기본 단위

주요 기능:
역할별 메시지 구조화

▪ Prompt Template

핵심 역할:
프롬프트 설계

주요 기능:
동적 프롬프트 생성 템플릿

▪ Structured Output

핵심 역할:
응답 구조화

주요 기능:
정형 데이터 자동 변환

▪ LCEL

핵심 역할:
파이프라인 연결

주요 기능:
컴포넌트 선언적 연결

▪ Tool Calling

핵심 역할:
외부 도구 실행

주요 기능:
함수/API 자율 호출

Chat Model

대화 형태의 메시지 리스트를 입력으로 받아 AI 응답 메시지를 생성하는 **LLM 인터페이스**

■ 핵심 특징

- OpenAI, Anthropic, Google 등 다양한 LLM 공급자를 **단일 인터페이스로 통합**
- 벤더 독립적 설계로 모델 교체 시 코드 변경 최소화
- 동기/비동기/스트리밍 호출 방식 모두 지원

■ 대표 AI 모델

OpenAI (ChatOpenAI)

- 업계의 사실상 표준 포맷을 정립한 가장 범용적이고 생태계가 넓은 모델
- 대표 모델 : gpt-4o, gpt-4-turbo, 최신 추론형 라인업인 o1 / o3 시리즈
- 강점 : 도구 호출(Tool Calling)의 정확도가 매우 높고, 정형화된 데이터 반환 능력이 안정적

Anthropic (ChatAnthropic)

- 학술적, 기술적 글쓰기와 복잡한 소스코드 분석 등에서 압도적인 평가를 받는 모델
- 대표 모델 : claude-3-5-sonnet, claude-3-opus
- 강점 : 자연스러운 문체, 매우 긴 문맥(Context Window)을 이해하는 능력, 그리고 깊은 논리적 사고(Extended thinking)가 뛰어남

Google GenAI (ChatGoogleGenerativeAI)

- 구글의 강력한 인프라를 바탕으로 대규모 텍스트와 멀티모달 데이터를 처리하는 모델
- 대표 모델 : gemini-1.5-pro, gemini-1.5-flash
- 강점 : 수백만 토큰에 달하는 방대한 입력창(Context)을 지원하여 책 한 권, 혹은 몇 시간 분량의 비디오를 통째로 넣어 대화할 수 있음

오픈소스 및 로컬 모델 (ChatOllama, ChatGroq 등)

- 보안이나 비용 문제로 자체 서버에 모델을 띄우거나, 초고속 추론 인프라를 사용할 때 연동
- 대표 모델 : Meta의 Llama 3, 미스트랄 AI의 Mixtral 등
- 강점 : 데이터 유출 걱정 없이 기업 내부망에서 독립적으로 장표 자동화나 챗봇 시스템을 구축할 수 있음

Chat Model > 모델 초기화 - init_chat_model()

■ 모델 초기화 - init_chat_model()

- 모델 공급 벤더의 종속성을 최소화하는 모델 생성
- init_chat_model()에서 입력받은 모델명(예: "gpt-4o", "gemini-1.5-pro")을 파싱하여, 해당하는 벤더사의 패키지(langchain_openai, langchain_google_genai 등) 내에 있는 모델 클래스를 자동으로 찾아 동적으로 객체를 생성
- (주의) 모델 공급 벤더의 패키지는 설치 필요
예) !pip install -qU langchain-openai, !pip install -qU langchain-google-genai, !pip install -qU langchain-anthropic...

```
from langchain.chat_models import init_chat_model

model = init_chat_model("gpt-5-nano")
```

VS.

```
from langchain_openai import ChatOpenAI

# ChatOpenAI 객체를 생성합니다.
gpt = ChatOpenAI(
    temperature=0,
    model_name="gpt-4o", # 모델명
)
```

```
from langchain_anthropic import ChatAnthropic

# ChatAnthropic 객체를 생성합니다.
anthropic = ChatAnthropic(
    model_name="claude-3-5-sonnet-20241022"
)
```

Chat Model > 파라미터 (1/2)

■ 파라미터 – temperature

- **temperature (0.0 ~ 2.0) : 답변의 무작위성과 창의성을 조절. 목적에 맞게 설정**
 - 값이 0에 가까울수록 가장 확률이 높은 단어만 일관되게 선택
 - 값이 높을수록 다양한 단어를 선택하여 창의적이고 예측 불가능한 답변

Case1) RAG(검색 증강 생성), 데이터 추출, 코드 생성:

→ temperature를 0.0 ~ 0.1 수준으로 낮추어 주어진 문서에만 기반해 팩트 위주로 정확하게 답하도록 통제

Case2) 마케팅 카피라이팅, 브레인스토밍, 페르소나 챗봇:

→ temperature를 0.7 ~ 1.0 사이로 두어 모델의 상상력을 적극 활용

```
# 1. Temperature가 0일 때 (보수적, 일관성 위주)
model_temp_0 = init_chat_model("gpt-5-nano", temperature=0.0)
print("[Temperature 0.0]")
print(model_temp_0.invoke("K-뷰티 홍보 문구를 간략하게 작성해줘요.").content)

# 2. Temperature가 1.0일 때 (창의적, 무작위성 위주)
model_temp_1 = init_chat_model("gpt-5-nano", temperature=1.0)
print("\n[Temperature 1.0]")
print(model_temp_1.invoke("K-뷰티 홍보 문구를 간략하게 작성해줘요.").content)
```


Chat Model > 파라미터 (2/2)

■ 파라미터 - timeout

(장애 방지) 모델 호출 후 응답을 얼마나 기다릴지(초 단위) 설정

- 서비스 운영 시 일시적 지연이 발생할 수 있으며, 이때 timeout이 없으면 서버의 스레드도 무한정 대기하다가 전체 서비스 장애로 이어질 수 있음
- timeout=30처럼 명시적인 제한을 두어 빠른 실패를 유도하고 재시도 로직을 태우는 것이 안전 함

■ 파라미터 -max_tokens

(비용 제한) 모델이 생성할 수 있는 응답의 최대 길이(토큰 수)를 제한

- 악의적인 사용자의 프롬프트 인젝션이나 모델의 무한 반복 오류로 인해 불필요하게 긴 답변이 생성되는 것을 막아주는 최소한의 비용 방어막
- 단, 요약이나 번역처럼 본질적으로 긴 출력이 필요한 작업에서는 너무 낮게 잡지 않도록 주의 필요

Chat Model > 모델 호출 (1/2)

■ 모델 호출 - `invoke()`

- 가장 기본이 되는 호출 방식. 입력(질문/지시)을 넣으면 전체 생성이 끝난 후 응답 객체가 돌아옴

```
response = model.invoke("안녕하세요. 당신은 누구입니까?")
print(response.content)
```

■ 모델 호출 - `stream()`

- 모델이 답변을 생성하는 즉시 **Chunk단위로 결과를 반환**.
- 사용자가 첫 번째 단어를 보는 데 걸리는 시간을 줄일 수 있어 사용자 경험(UX)이 극적으로 개선

```
# stream()은 chunk들의 제너레이터(generator)를 반환합니다.
for chunk in model.stream("AI Agent란 무엇인지 1000자 이상으로 설명해 주세요"):
    # chunk는 AIMessageChunk 객체이므로 .content로 텍스트를 추출합니다.
    print(chunk.content, end="", flush=True)
```

`flush=True`: 버퍼에 대기시키지 않고 즉시 화면에 강제로 출력(Flush)

Chat Model > 모델 호출 (2/2)

■ 모델 호출 - batch()

- 여러 개의 입력을 병렬로 동시 처리. (스레드 풀 등을 활용해 동시에 API 요청을 하기에, 10개의 질문도 5~10초 내외로 응답을 받을 수 있음)
- 대량의 문서를 요약하거나 수많은 데이터 행을 일괄 변환해야 하는 데이터 전처리 파이프라인에서 강력한 무기로 활용
- (주의) 수십, 수백 개의 데이터를 한 번에 `batch()`로 넘기면, API 제공자(OpenAI 등) 측에서 초당 요청 제한 초과로 에러를 발생시킬 수 있으므로 `config` 파라미터를 통해 동시에 실행될 최대 작업 수를 제어 함

```
inputs = [
    "과적합(Overfitting)이 뭔가요? 한 줄로 요약해줘.",
    "앵무새의 털 색상이 화려한 이유를 한 줄로 요약해줘.",
    "AI Agent의 핵심 특징 한 가지는?",
    "오로라의 현상을 한 줄로 요약해줘",
    "LangChain이 AI Agent 개발자에게 제공하는 주요 핵심 기능을 한 줄로 요약해줘.",
    "LangChain과 LangGraph의 차별성을 한 줄로 요약해줘"
]

# 한 번의 호출로 6개의 질문을 병렬 처리
responses = model.batch(inputs)

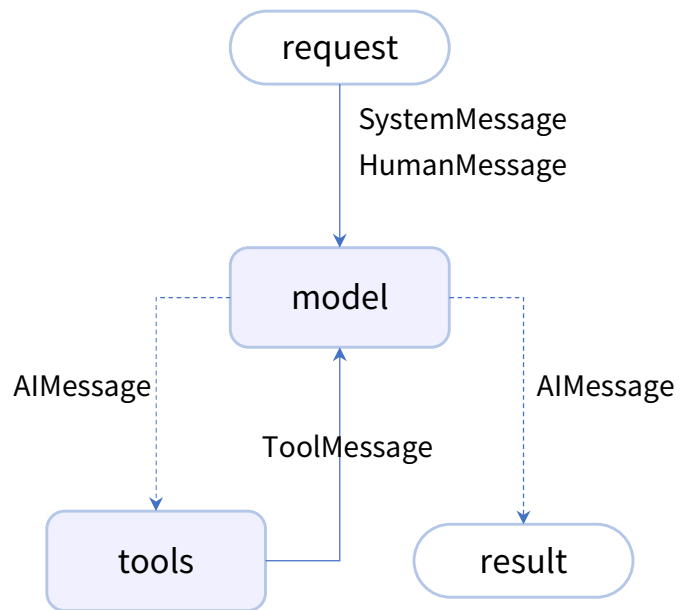
for i, response in enumerate(responses):
    print(f"[{i+1}번 답변] {response.content}")

# 최대 3개씩만 동시에 처리하도록 제한 (Rate Limit 방어)
responses = model.batch(inputs, config={"max_concurrency": 3})
```

Messages

Chat Model과 주고 받는 대화의 기본 단위로, 역할(Human, AI 등) 과 내용을 담고 있는 객체

Message 유형



- **SystemMessage** : AI의 역할과 지시사항 정의
 - 개발자가 미리 부여하는 "역할 및 규칙 프롬프트"
 - 예를 들어,
“당신은 유능한 로켓 전문가입니다.”라고 설정하면,
이후 모델의 답변 톤과 관점이 해당 역할에 맞춤
- **HumanMessage** : 사용자가 입력하는 질문이나 지시사항
- **AIMessage** : AI가 생성한 답변 메시지
- **ToolMessage** : 도구가 실행한 결과 메시지

Message > 활용 방법 (1/2)

■ 활용 방법 - (1) 메시지 객체 활용

- langchain.messages 객체를 활용하여 message 정의

```
from langchain.messages import HumanMessage, AIMessage, SystemMessage

system_msg = SystemMessage("당신은 유능한 로켓 전문가입니다.")
human_msg = HumanMessage("안녕하세요. 궁금한 게 있어요!")

messages = [system_msg, human_msg]
response = model.invoke(messages)
print(response.content)
```

```
messages = [
    SystemMessage("당신은 친절한 조교입니다."),
    HumanMessage("안녕하세요. 저는 Jumany라고 합니다."),
    AIMessage("안녕하세요 Jumany님, 반갑습니다. 무엇을 도와드릴까요?"),
    HumanMessage("제가 방금 제 이름을 뭐라고 했죠?"),
]

response = model.invoke(messages)
print(response.content)
```

Message > 활용 방법 (2/2)

■ 활용 방법 - (2) 딥서너리 활용

- Role, content를 정의. (Role : “system”, “human” or “user”, “ai” or “assistant”)

```
messages = [
    {"role": "system", "content": "당신은 유능한 로켓 전문가입니다."},
    {"role": "human", "content": "안녕하세요. 궁금한 게 있어요!"},
    {"role": "ai", "content": "로켓 관련 무엇이든 물어보세요."},
    {"role": "human", "content": "추진 방식 차이를 설명해 주세요"},
]

response = model.invoke(messages)
```

💡 딥서너리 사용 지양, “메시지 객체” 사용 권장

(1) 벤더별 파편화된 데이터 구조 개념과의 혼선 방지

각 벤더의 순수 SDK(Python openai 패키지나 google-genai 패키지 등)를 직접 다룰 때 Gemini의 parts -> text 구조나 Anthropic의 구조 차이 때문에 큰 혼란과 코드 에러 발생 확률이 높아짐

(2) 코드의 안정성 및 타입 스키마(Type Schema) 보장

딥서너리는 오타자(예: "role": "huamn", "conent": "...")가 나도 실행 시점(Runtime) 전까지 에러를 잡기 어려우나, 메시지 객체를 사용하면 IDE의 자동완성(Autocomplete) 지원 및 타입 체크가 가능

(3) 멀티모달 및 복잡한 메타데이터 확장성

이미지, 음성, 파일 등을 함께 넘기는 멀티모달 프롬프트나 Tool Calling(함수 호출) 결과를 처리할 때, 딥서너리로 구현하면 벤더마다 구조가 매우 복잡하나, 메시지 객체를 쓰면 SystemMessage, HumanMessage라는 단일 표준 래퍼 안에서 객체지향적으로 관리 가능

Prompts, Prompt Template

언어 모델이 사용할 질문이나 명령을 생성하는 과정. 검색된 정보를 바탕으로 최종 사용자의 질문에 가장 잘 대응할 수 있는 응답 생성을 위한 필수 단계

■ 정의

- **프롬프트**는 모델에 대한 입력을 의미
- 실제 LLM 서비스들의 경우, 사용자가 전부 입력하도록 만들지 않고 Back 단에서 여러 구성요소를 통해 편리한 입력을 지원
- **프롬프트 템플릿**은 이러한 편리한 입력 지원을 위한 모듈
- Langchain은 프롬프트를 쉽게 구성하고 작업할 수 있도록 여러 클래스와 함수를 제공

- **문맥(Context) 설정** : 프롬프트는 언어 모델이 특정 문맥에서 작동하도록 설정하는 역할을 함. 이를 통해 모델은 제공된 정보를 바탕으로 보다 정확하고 관련성 높은 답변을 생성할 수 있음.
- **정보 통합** : 여러 문서에서 검색된 정보는 서로 다른 관점이나 내용을 포함할 수 있음. 프롬프트 단계에서 이러한 정보를 통합하고, 모델이 이를 효율적으로 활용할 수 있는 형식으로 조정.
- **응답 품질 향상** : 질문에 대한 모델의 응답 품질은 프롬프트의 구성에 의존적이며, 잘 구성된 프롬프트는 모델이 보다 정확하고 유용한 정보를 제공하게 함.

Prompts > 활용 방법 (1/4)

■ 활용 방법 - (1) from_template()

- 방법 1. from_template() 메소드를 사용하여 PromptTemplate 객체 생성

```
from langchain_core.prompts import PromptTemplate

# template 정의. {country}는 변수로, 이후에 값이 들어갈 자리를 의미
template = "{country}의 수도는 어디인가요?"

# from_template 메소드를 이용하여 PromptTemplate 객체 생성
prompt = PromptTemplate.from_template(template)
print(f"### prompt : \n{prompt}\n")

# prompt 생성. format 메소드를 이용하여 변수에 값을 넣어주면, 일회성으로 프롬프트 문자열이 생성
# 이 프롬프트 객체(prompt)를 재할당하지 않고, formatted_prompt와 같이 별도의 변수에 저장하거나 바로 출력하는 것이 좋음
formatted_prompt_string = prompt.format(country="대한민국")
print(f"### formatted_prompt_string : \n{formatted_prompt_string}\n")

# chain 생성
chain = prompt | model

# country 변수에 입력된 값이 자동으로 치환되어 수행됨
chain.invoke({"country": "대한민국"}).content
```


Prompts > 활용 방법 (2/4)

■ 활용 방법 - (2) PromptTemplate()

- 방법 2. PromptTemplate 객체 생성과 동시에 prompt 생성

```
# template 정의
template = "{country}의 수도는 어디인가요?"

# PromptTemplate 객체를 활용하여 prompt_template 생성
prompt = PromptTemplate(
    template=template,
    input_variables=["country"],
)

# prompt 생성
formatted_prompt_string = prompt.format(country="대한민국")
```

```
# chain 생성
chain = prompt | model

# country 변수에 입력된 값이 자동으로 치환되어 수행됨
# invoke 메서드는 딕셔너리 형태의 입력을 기대
chain.invoke({"country": "프랑스"}).content
```

Prompts > 활용 방법 (3/4)

■ 활용 방법 - (3) load_prompt()

- 방법 3. 파일로부터 template 읽어오기

```
from langchain_core.prompts import load_prompt
prompt = load_prompt("./test/fruit_color.yaml")
# chain 생성
chain = prompt | model
chain.invoke({"fruit": "사과"}).content
```

_type: prompt
input_variables:
- fruit
template: "{fruit}의 대표적인 색상은 무엇인가요? 한 단어로만 답변해주세요."

```
from langchain_core.prompts import load_prompt
prompt = load_prompt("./test/capital.yaml")
# chain 생성
chain = prompt | model
chain.invoke({"country": "영국"}).content
```

_type: prompt
input_variables:
- country
template: "{country}의 수도는 어디인가요? 도시 이름만 답변해주세요."

Prompts > 활용 방법 (4/4)

■ 활용 방법 - (4) ChatPromptTemplate

- 방법 4. ChatPromptTemplate : 대화목록을 프롬프트로 주입하고자 할 때 활용
 - 메시지는 튜플(tuple) 형식으로 구성하며, (role, message) 로 구성하여 리스트로 생성

```
from langchain_core.prompts import ChatPromptTemplate, SystemMessagePromptTemplate, HumanMessagePromptTemplate,
AIMessagePromptTemplate
```

```
# MessagePromptTemplate 객체를 활용한 템플릿 정의
```

```
chat_template = ChatPromptTemplate.from_messages(
    [
        SystemMessagePromptTemplate.from_template("당신은 친절한 AI 어시스턴트입니다. 당신의 이름은 {name}입니다."),
        HumanMessagePromptTemplate.from_template("반가워요!"),
        AIMessagePromptTemplate.from_template("안녕하세요! 무엇을 도와드릴까요?"),
        HumanMessagePromptTemplate.from_template("{user_input}"),
    ]
)
```

```
# chain 생성
```

```
chain = chat_template | model
```

```
response = chain.invoke({"name": "하니", "user_input": "당신의 이름은 무엇입니까?"})
print(response.content)
```

참고. MessagePlaceholder

■ MessagePlaceholder()

- LangChain의 MessagePlaceholder(variable_name="chat_history")는 이전 대화 내역(Chat History)을 프롬프트 내부의 특정 위치에 가변적으로 주입하기 위한 예약된 공간(Placeholder)
- 일반적인 텍스트 변수인 {input}는 단순한 문자열(String)만 들어갈 수 있는 반면, MessagePlaceholder는 여러 개의 메시지 객체 리스트(List of BaseMessage)를 통째로 집어넣을 때 사용

```
# MessagePlaceholder는 이전 대화 목록(List of Messages)이 들어갈 '빈 자릿표' 역할을 합니다.
prompt = ChatPromptTemplate.from_messages([
    ("system", "당신은 가상 RPG 게임의 친절한 안내원 '모험가 가이드'입니다."),
    # ★ 이전 대화 기록이 들어갈 위치 지정
    MessagePlaceholder(variable_name="chat_history"),
    ("human", "{input}"),
])

chain = prompt | model

history = [
    HumanMessage(content="안녕! 내 캐릭터 이름은 '용사키우기'야."),
    AIMessage(content="반갑습니다, '용사키우기'님! 무엇을 도와드릴까요?")
]

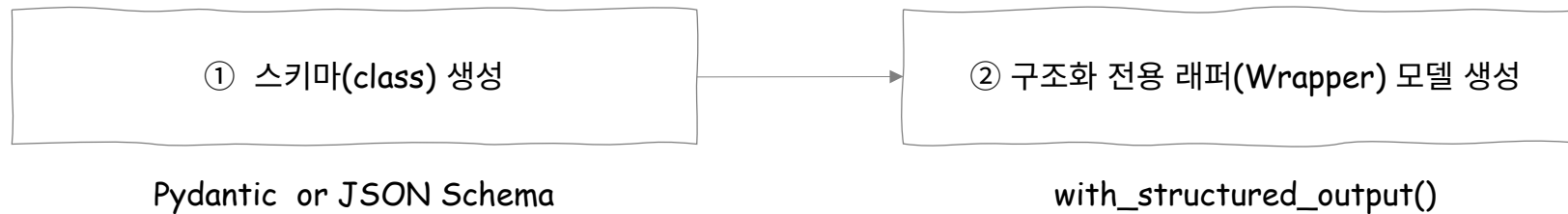
response = chain.invoke({
    "chat_history": history, # MessagePlaceholder 자리에 이 리스트가 삽입됨
    "input": "내 이름을 기억하고 있니?"
})
```

Structured Output

모델의 답변을 단순한 텍스트 문치가 아니라 **미리 약속한 정교한 데이터 형태로** 받아내는 방법

■ 활용

- **구조화된 출력(Structured Output)**은 **명확한 후속 작업** (데이터베이스 저장, API호출, 알림 발송 등)을 가능하게 함
- LLM의 출력 형식을 명확하게 정의
- 형식에 맞는 구조화된 데이터(JSON, Pydantic 모델) 을 자동으로 생성
- 후속 로직에서 처리하기 위한 일정한 데이터 구조(JSON, dict 등) 제공



Structured Output > Pydantic

▪ (1) Pydantic

- 파이썬 환경에서 에이전트를 개발한다면 추천하는 방식
- 클래스 형태로 구조를 잡기 때문에 코드가 깔끔하고, IDE의 자동완성 지원을 받을 수 있음

① Schema 정의

```
from pydantic import BaseModel, Field
from typing import Optional, Literal

class Movie(BaseModel):
    """상세한 영화 정보."""
    title: str = Field(description="영화의 제목 (예: 인셉션)")
    year: Optional[int] = Field(default=None, description="개봉 연도. 정보를 알 수 없다면 None.")
    genre: Literal["액션", "로맨스", "SF", "코미디", "기타"] = Field(description="영화의 장르")
    director: str = Field(description="영화 감독 이름")
    rating: float = Field(description="영화 평점 (10점 만점 기준)")
```

💡 Pydantic Schema 설계 시 주의할 점

- Description은 곧 프롬프트이며, 단순한 주석이 아님.
- LLM은 Field(description="...") 안의 텍스트를 읽고 어떤 값을 넣을지 판단.
- 예시(예: 인셉션)나 제약 조건(10점 만점 기준)을 구체적으로 적어주면 정확도가 크게 올라감.

Structured Output > Pydantic

② 구조화 출력 모델 만들기 + 결과 분석

1. 스키마를 전달하여 구조화된 모델 생성

```
model_with_structure = model.with_structured_output(Movie)
```

2. 자연어 프롬프트로 호출

```
response = model_with_structure.invoke("영화 도둑들에 대해 설명해 주세요")
```

3. 결과 확인

```
print(response)
```

```
title='도둑들' year=2012 genre='액션' director='최동훈' rating=7.5
```

```
print(f"제목: {response.title} (타입: {type(response.title)})")
```

```
print(f"평점: {response.rating} (타입: {type(response.rating)})")
```

```
제목: 도둑들 (타입: <class 'str'>)
```

```
평점: 7.5 (타입: <class 'float'>)
```

- 문자열 파싱을 위한 정규표현식을 짤 필요가 전혀 없다
 - response.title 처럼 점(.)을 찍어 즉시 데이터에 접근할 수 있다
 - 평점(8.8)은 단순 문자가 아닌 연산이 가능한 실수형(float)으로 안전하게 타입 캐스팅이 된다
- 바로 데이터베이스에 저장하거나 다음 로직에 태우기 완벽한 상태가 됨

Structured Output > JSON Schema

▪ (2) JSON Schema

- JSON Schema는 스키마를 딕셔너리로 정의하는 방식
- 파이썬에 종속되지 않기 때문에 **언어 독립적인 스키마를 만들 때 유용**

① JSON Schema 정의

```
import json

json_schema = {
    "title": "Movie",
    "description": "A movie with details",
    "type": "object",
    "properties": {
        "title": {
            "type": "string",
            "description": "The title of the movie"
        },
        "year": {
            "type": "integer",
            "description": "The year the movie was released"
        },
    },
}
```

```
    "director": {
        "type": "string",
        "description": "The director of the movie"
    },
    "rating": {
        "type": "number",
        "description": "The movie's rating out of 10"
    },
    "required": ["title", "director", "rating"]
}
```

💡 주의할 점

JSON Schema 방식은 이후 언어 모델 호출 시 스키마 키나 설명 문구에 제약이 생길 수 있으므로, 키값은 영문, 숫자, 언더스코어, 대시로만 작성하는 것이 안전

Structured Output > JSON Schema

② 구조화 출력 모델 만들기 + 결과 분석

1. JSON 스키마를 전달

```
model_with_structure = model.with_structured_output(json_schema)
```

2. 호출

```
response = model_with_structure.invoke("영화 극한직업에 대해서 소개해 주세요")
```

3. 결과 확인

```
print(response)
```

```
{'title': '극한직업', 'year': 2019, 'director': '이병헌', 'rating': 7.5}
```

```
print(response['title'])
```

```
print(response['director'])
```

```
극한직업  
이병헌
```

LCEL (LangChain Expression Language)

▪ LangChain Expression Language (LCEL)

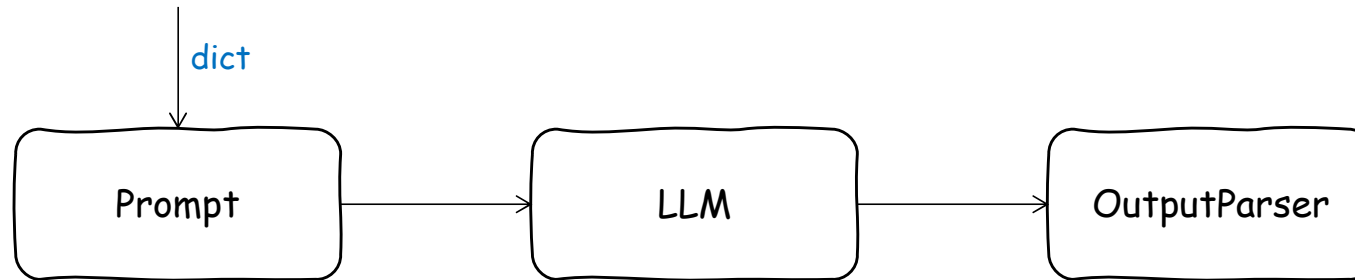
- LangChain 라이브러리에서 제공하는 선언적 방식의 인터페이스로, 복잡한 LLM (Large Language Model) 애플리케이션을 구축하고 실행하기 위한 도구. LLM, 프롬프트, 검색기, 메모리 등 다양한 컴포넌트를 조합하여 강력하고 유연한 AI 시스템을 만들 수 있게 함
- 주요 특징
 - (1) 선언적 구문 : 복잡한 로직을 간결하고 읽기 쉬운 방식으로 표현
 - (2) 모듈성 : 다양한 컴포넌트를 쉽게 조합하고 재사용
 - (3) 유연성 : 다양한 유형의 LLM 애플리케이션 구축
 - (4) 확장성 : 사용자 정의 컴포넌트를 쉽게 통합
 - (5) 최적화 : 실행 시 자동으로 최적화를 수행
- 기본 구성 요소
 - (1) **Runnable** : 모든 LCEL 컴포넌트의 기본 클래스
 - (2) **Chain** : 여러 Runnable을 순차적으로 실행
 - (3) **RunnableMap** : 여러 Runnable을 병렬로 실행
 - (4) **RunnableSequence** : Runnable의 시퀀스를 정의
 - (5) **RunnableLambda** : 사용자 정의 함수를 Runnable로 래핑

LCEL (LangChain Expression Language)

■ 기본 구조

- 프롬프트 + 모델 + 출력 파서 : 가장 기본적이고 일반적인 사용은 prompt 템플릿과 모델을 연결하는 것임

`{"question": "{topic}에 대해 쉽게 설명해주세요."}`



- '|' 기호는 [unix 파이프 연산자]와 유사하며, 서로 다른 구성 요소를 연결하고 한 구성 요소의 출력을 다음 구성 요소의 입력으로 전달

```
chain = prompt | model | output_parser
```

- 사용자 입력은 프롬프트 템플릿으로 전달되고, 그런 다음 프롬프트 템플릿 출력은 모델로 전달. 모델 결과가 OutputParser로 전달

LCEL > Prompt | Model

```
from langchain_core.prompts import PromptTemplate

# prompt 를 PromptTemplate 객체로 생성합니다.
prompt = PromptTemplate.from_template("{topic}에 대해 쉽게 설명해주세요.")

model = init_chat_model(model="gpt-4.1-nano", temperature=0.1)

# 프롬프트, 모델, 출력 파서를 연결하여 처리 체인을 구성합니다.
chain = prompt | model
```

```
# chain 객체의 invoke 메서드를 사용하여 input을 전달합니다.
input = {"topic": "인공지능 모델의 학습 원리"}
chain.invoke(input)
```

AIMessage(content=) 물론입니다! 인공지능 모델의 학습 원리를 쉽게 설명해 드릴게요.\n\n1. **데이터 수집**: 먼저, 인공지능이 배우기 위해 많은 예제(데이터)를 모아요. 예를 들어, 고양이 사진을 많이 보여주는 거죠.\n\n2. **모델 설계**: 그런 다음, 이 데이터를 이해하고 학습할 수 있도록 인공지능의 구조(모델)를 만듭니다. 이 구조는 사람의 뇌와 비슷하게 생각할 수 있어요.\n\n3. **학습 과정**: 인공지능은 처음에는 아무것도 모르기 때문에, 데이터를 보고 예측을 시도해요.\n- 예를 들어, 사진이 고양이인지 아닌지 맞추려고 해

~~ 이하 생략 ~~

```
# 스트리밍 출력을 위한 요청
answer = chain.stream(input)
# 스트리밍 출력
for chunk in answer:
    print(chunk.content, end="", flush=True)
```

LCEL > Prompt | Model | OutputParser

```
from langchain_core.output_parsers import StrOutputParser

output_parser = StrOutputParser()

# 프롬프트, 모델, 출력 파서를 연결하여 처리 체인을 구성합니다.
chain = prompt | model | output_parser
```

```
# chain 객체의 invoke 메서드를 사용하여 input을 전달합니다.
input = {"topic": "인공지능 모델의 학습 원리"}
chain.invoke(input)
```

물론입니다! 인공지능 모델의 학습 원리를 쉽게 설명해 드릴게요.

1. **데이터 수집**: 먼저, 인공지능이 배우기 위해 많은 예시(데이터)를 모아요. 예를 들어, 사진, 글, 숫자 등 다양한 정보를 모을 수 있어요.
2. **모델 설계**: 그 다음, 인공지능이 어떤 방식으로 데이터를 이해하고 학습할지 정하는 '모델'이라는 구조를 만듭니다. 이 모델은 사람의 뇌처럼 작동하는 가상의 네트워크일 수 있어요.

~~ 이하 생략 ~~

```
# 스트리밍 출력을 위한 요청
answer = chain.stream(input)
# 스트리밍 출력
for chunk in answer:
    print(chunk, end="", flush=True)
```

LCEL > 영어회화

3. LangChain

```
template = """
당신은 영어를 가르치는 10년차 영어 선생님입니다. 주어진 상황에 맞는
영어 회화를 작성해 주세요.
양식은 [FORMAT]을 참고하여 작성해 주세요.
```

```
#상황:
{question}
```

```
#FORMAT:
- 영어 회화:
- 한글 해석:
"""
```

```
# 프롬프트 템플릿을 이용하여 프롬프트를 생성합니다.
prompt = PromptTemplate.from_template(template)
```

```
# ChatOpenAI 챗모델을 초기화합니다.
model = init_chat_model("gpt-4.1-nano")
```

```
# 문자열 출력 파서를 초기화합니다.
output_parser = StrOutputParser()
```

```
# 체인을 구성합니다.
chain = prompt | model | output_parser
```

```
# 완성된 Chain을 실행하여 답변을 얻습니다.
print(chain.invoke({"question": "저는 식당에 가서 음식을 주문하고
싶어요"}))
```

```
# 완성된 Chain을 실행하여 답변을 얻습니다.
# 스트리밍 출력을 위한 요청
answer = chain.stream({"question": "저는 식당에 가서 음식을 주문하
고 싶어요"})
# 스트리밍 출력
for chunk in answer:
    print(chunk, end="", flush=True)
```

```
# 이번에는 question 을 '미국에서 피자 주문'으로 설정하여 실행합니다.
# 스트리밍 출력을 위한 요청
answer = chain.stream({"question": "미국에서 피자 주문"})
# 스트리밍 출력
for chunk in answer:
    print(chunk, end="", flush=True)
```

LCEL > 인터페이스 (1/7)

▪ LCEL 인터페이스

- 표준 인터페이스로, 사용자 정의 체인을 정의하고 표준 방식으로 호출하는 것을 쉽게 해 줌

- | | |
|----------------------------|------------------------------------|
| • stream : 응답의 청크를 스트리밍 | • astream : 비동기적으로 응답의 청크를 스트리밍 |
| • invoke : 입력에 대해 체인을 호출 | • ainvoke : 비동기적으로 입력에 대해 체인을 호출 |
| • batch : 입력 목록에 대해 체인을 호출 | • abatch : 비동기적으로 입력 목록에 대해 체인을 호출 |

▪ chain 생성

```
from langchain_core.prompts import PromptTemplate
from langchain.chat_models import init_chat_model
from langchain_core.output_parsers import StrOutputParser

# 1. 주어진 토픽에 대한 농담을 요청하는 프롬프트 템플릿을 생성
prompt = PromptTemplate.from_template("{topic}에 대하여 3문장으로 설명해줘.")
# 2. ChatModel을 인스턴스화
model = init_chat_model("gpt-4.1-nano")
# 3. 문자열 출력 파서를 초기화합니다.
output_parser = StrOutputParser()
# 프롬프트와 모델을 연결하여 대화 체인을 생성
chain = prompt | model | output_parser
```

LCEL > 인터페이스 (2/7)

▪ stream : 실시간 출력

- 주어진 토픽에 대한 데이터 스트림을 생성하고, 이 스트림을 반복하여 각 데이터의 내용(content)을 즉시 출력
- end="" 인자는 출력 후 줄바꿈을 하지 않도록 설정하며, flush=True 인자는 출력 버퍼를 즉시 비우게 함

```
# chain.stream 메서드를 사용하여 '멀티모달' 토픽에 대한 스트림을 생성하고 반복합니다.  
for token in chain.stream({"topic": "멀티모달"}):  
    # 스트림에서 받은 데이터의 내용을 출력합니다. 줄바꿈 없이 이어서 출력하고, 버퍼를 즉시 비웁니다.  
    print(token, end="", flush=True)
```

▪ invoke : 호출

- 주제를 인자로 받아 해당 주제에 대한 처리를 수행

```
# chain 객체의 invoke 메서드를 호출하고, 'ChatGPT'라는 주제로 딕셔너리를 전달합니다.  
chain.invoke({"topic": "ChatGPT"})
```


LCEL > 인터페이스 (3/7)

▪ batch : 배치(단위 실행)

- 여러 개의 딕셔너리를 포함하는 리스트를 인자로 받아, 각 딕셔너리에 있는 topic 키의 값을 사용하여 일괄 처리를 수행
- config 딕셔너리는 max_concurrency 키를 통해 동시에 처리할 수 있는 최대 작업 수를 설정

주어진 토픽 리스트를 batch 처리하는 함수 호출

```
chain.batch(  
    [  
        {"topic": "ChatGPT"},  
        {"topic": "Instagram"},  
        {"topic": "멀티모달"},  
        {"topic": "프로그래밍"},  
        {"topic": "머신러닝"},  
    ],  
    config={"max_concurrency": 3},  
)
```

LCEL > 인터페이스 (4/7)

▪ `async_stream` : 비동기 스트림

- 주어진 토픽에 대한 메시지를 비동기적으로 처리
- 비동기 for 루프(`async for`)를 사용하여 스트림에서 메시지를 순차적으로 받아오고, `print` 함수를 통해 메시지의 내용(`s.content`)을 즉시 출력
- `end=""`는 출력 후 줄바꿈을 하지 않도록 설정하며, `flush=True`는 출력 버퍼를 강제로 비워 즉시 출력되도록 함

```
# 비동기 스트림을 사용하여 'YouTube' 토픽의 메시지를 처리합니다.  
async for token in chain.astream({"topic": "YouTube"}):  
    # 메시지 내용을 출력합니다. 줄바꿈 없이 바로 출력하고 버퍼를 비웁니다.  
    print(token, end="", flush=True)
```

▪ `async_invoke` : 비동기 호출

- 비동기적으로 주어진 인자를 사용하여 작업을 수행
- `await` 은 비동기로 처리되는 프로세스가 완료될 때까지 기다림

```
# 비동기 체인 객체의 'ainvoke' 메서드를 호출하여 'NVIDIA' 토픽을 처리합니다.  
my_process = chain.ainvoke({"topic": "NVIDIA"})  
  
# 비동기로 처리되는 프로세스가 완료될 때까지 기다립니다.  
await my_process
```

LCEL > 인터페이스 (5/7)

▪ `async_batch` : 비동기 배치

- 비동기적으로 일련의 작업을 일괄 처리
- `await` 키워드는 해당 비동기 작업이 완료될 때까지 기다리는 데 사용

```
# 주어진 토픽에 대해 비동기적으로 일괄 처리를 수행합니다.
my_abatch_process = chain.abatch(
    [{"topic": "YouTube"},
    [{"topic": "Instagram"},
    [{"topic": "Facebook"}]
)

# 비동기로 처리되는 일괄 처리 프로세스가 완료될 때까지 기다립니다.
await my_abatch_process
```

LCEL > 인터페이스 (6/7)

▪ parallel : 병렬성

① 병렬 실행 (Parallel Execution)

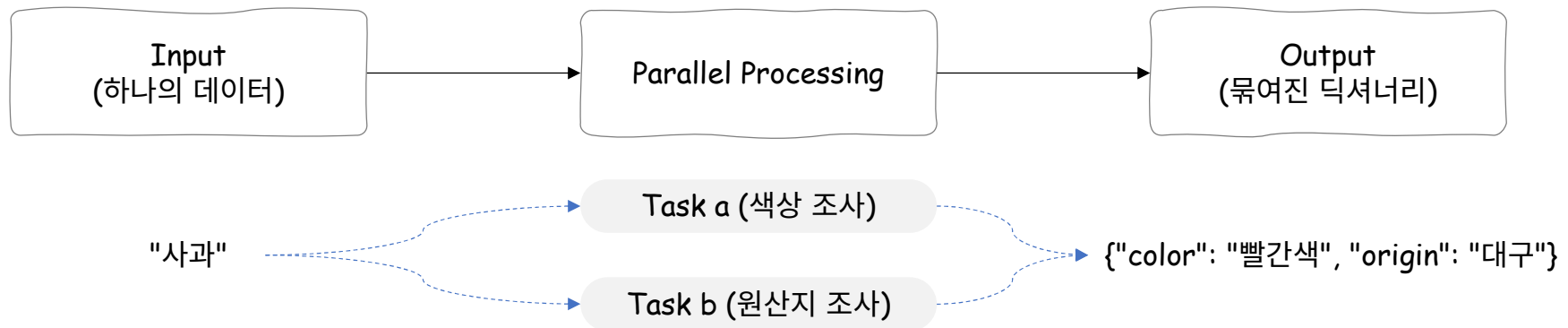
- 여러 개의 독립적인 태스크(LLM 호출, DB 조회, 프롬프트 생성 등)를 동시에(Parallel) 처리하여 전체적인 실행 시간을 획기적으로 단축

② 데이터 분기 및 전달 (Data Forking)

- 하나의 입력값을 여러 개의 서브 체인에 동시에 똑같이 전달 함

③ 딕셔너리 형태로 결과 병합 (Combined 객체 생성)

- 각 병렬 작업이 끝난 후, 지정한 Key 이름에 맞춰 결과 값을 하나의 맵(Map/Dictionary) 형태로 다음 체인으로 넘겨줌



LCEL > 인터페이스 (7/7)

```
from langchain_core.runnables import RunnableParallel

# {country} 의 수도를 물어보는 체인을 생성합니다.
chain1 = (
    PromptTemplate.from_template("{country}의 수도는 어디야?")
    | model
    | StrOutputParser()
)

# {country} 의 면적을 물어보는 체인을 생성합니다.
chain2 = (
    PromptTemplate.from_template("{country}의 면적은 얼마야?")
    | model
    | StrOutputParser()
)

# 위의 2개 체인을 동시에 생성하는 병렬 실행 체인을 생성합니다.
combined = RunnableParallel(capital=chain1, area=chain2)
```

```
# chain1 (수도) 를 실행합니다.
chain1.invoke({"country": "대한민국"})
```

```
# chain2 (국토 면적) 를 실행합니다.
chain2.invoke({"country": "미국"})
```

```
# 병렬 (수도, 국토 면적) 실행 체인을 실행합니다.
combined.invoke({"country": "대한민국"})
```

▪ 배치 병렬

```
# 배치 처리를 수행합니다.
chain1.batch([{"country": "대한민국"}, {"country": "미국"}])
```

```
# 배치 처리를 수행합니다.
chain2.batch([{"country": "대한민국"}, {"country": "미국"}])
```

```
# 주어진 데이터를 배치로 처리합니다.
combined.batch([{"country": "대한민국"}, {"country": "미국"}])
```

LCEL > Runnable

RunnablePassthrough

- RunnablePassthrough는 입력된 데이터를 그대로 전달
- 데이터를 변환하거나 수정할 필요가 없는 경우 사용
- 파이프라인의 특정 단계를 건너뛰어야 하는 경우 사용
- 디버깅 또는 테스트 목적으로 데이터 흐름을 모니터링해야 하는 경우 사용

RunnableLambda

- 사용자 정의 함수를 실행할 수 있는 기능을 제공
- 개발자는 자신만의 함수를 정의하고, 해당 함수를 RunnableLambda를 사용하여 실행할 수 있음
- *예를 들어, 데이터 전처리, 계산, 또는 외부 API와의 상호 작용과 같은 작업을 수행하는 함수를 정의하고 실행할 수 있음*

RunnableParallel

- RunnableParallel은 여러 체인을 병렬 실행
- 시퀀스 내에서 하나의 Runnable의 출력을 다음 Runnable의 입력 형식에 맞게 조작하는 데 유용하게 사용 됨

RunnableBranch

- 조건 분기 (IfElse) 체인을 구현하는 Runnable로 입력 데이터에 따라 다른 체인을 실행
- LCEL 파이프라인에서 라우팅 / 분기 처리에 핵심적으로 활용됨

LCEL > Runnable > 예시 1

RunnableParallel, RunnablePassthrough

```
from langchain_core.runnables import RunnableParallel, RunnablePassthrough

runnable = RunnableParallel(
    # 전달된 입력을 그대로 반환하는 Runnable을 설정합니다.
    passed=RunnablePassthrough(),
    # 입력의 "num" 값에 3을 곱한 결과를 반환하는 Runnable을 설정합니다.
    extra=RunnablePassthrough.assign(mult=lambda x: x["num"] * 3),
    # 입력의 "num" 값에 1을 더한 결과를 반환하는 Runnable을 설정합니다.
    modified=lambda x: x["num"] + 1,
)

# {"num": 1}을 입력으로 Runnable을 실행합니다.
runnable.invoke({"num": 1})
```

```
{'passed': {'num': 1}, 'extra': {'num': 1, 'mult': 3}, 'modified': 2}
```

입력 데이터 {"num": 1} 하나를 가지고

- [1] 원본 그대로 내보내기 → passed
- [2] 연산 결과를 새 키로 추가하기 → extra
- [3] 연산 결과만 반환하기 → modified

의 3가지 작업을 동시에(병렬로) 수행하여 하나의 결과 딕셔너리로 묶어 줌

LCEL > Runnable > 예시 2

RunnableParallel, RunnablePassthrough

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

prompt1 = ChatPromptTemplate.from_messages([("system", "긍정적인 측면으로 답하세요."), ("human", "{q}")])
prompt2 = ChatPromptTemplate.from_messages([("system", "부정적인 측면으로 답하세요."), ("human", "{q}")])

parser = StrOutputParser()

chain1 = prompt1 | model | parser
chain2 = prompt2 | model | parser

act = RunnableParallel(
    question=RunnablePassthrough() | itemgetter("q"), # 입력을 통과시킨 뒤 q 값만 추출
    positive=chain1,
    negative=chain2,
)

out = act.invoke({"q": "2배 레버리지 ETF 투자에 대해 어떻게 생각하세요?"})

print("Question:", out["question"])
print("*** Answer POSITIVE:", out["positive"][:70], "...")
print("*** Answer NEGATIVE:", out["negative"][:70], "...")
```

Question: 2배 레버리지 ETF 투자에 대해 어떻게 생각하세요?

*** Answer POSITIVE: 2배 레버리지 ETF 투자는 높은 수익 잠재력을 제공하는 동시에, 시장 변동성에 매우 민감하다는 점을 고려할 때 신중히 접
 *** Answer NEGATIVE: 2배 레버리지 ETF 투자는 매우 위험한 투자 전략입니다. 이러한 상품은 시장이 예상과 반대로 움직일 경우 손실이 두 배로

LCEL > Runnable > 예시 3

RunnableLambda

```
def combine_text(text):
    return f"positive : {text['positive']} negative : {text['negative']}"

from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnableLambda

sum_prompt = ChatPromptTemplate.from_messages([("system", "다음 내용을 자연스럽게 200자 이내로 교정해줘요"), ("human", "{info}")])

sum_act = act | {"info": RunnableLambda(combine_text)} | sum_prompt | model | StrOutputParser()

sum_out = sum_act.invoke({"q": "2배 레버리지 ETF 투자에 대해 어떻게 생각하세요?"})
sum_out
```

'2배 레버리지 ETF는 단기 수익 기회가 크지만, 장기 보유 시 누적 수익이 비대칭으로 감소하고 비용 · 스프레드 · 롤링 비용, 변동성 손실 위험이 큼니다. 단기 트레이딩이나 헤지에 적합하나, 소액으로 시작해 손실 한도와 목표를 명확히 설정하고 신중히 판단하세요.'

LCEL > Runnable > 예시 4(1/2)

RunnableBranch, RunnablePassthrough

```
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import PromptTemplate

prompt = PromptTemplate.from_template(
    """주어진 사용자 질문을 `수학`, `과학`, 또는 `기타` 중 하나로 분류하세요. 한
    단어 이상으로 응답하지 마세요.

    <question>
    {question}
    </question>

    Classification:"""
)

# 체인을 생성합니다.
chain = (
    prompt
    | ChatOpenAI(model="gpt-4o-mini")
    | StrOutputParser() # 문자열 출력 파서를 사용합니다.
)
```

```
math_chain = (
    PromptTemplate.from_template(
        """You are an expert in math. \
        Always answer questions starting with "파스칼선생님께서 말씀하시기를..". \
        Respond to the following question:

        Question:{question}
        Answer:"""
    )
    | ChatOpenAI(model="gpt-4o-mini")
)

science_chain = (
    PromptTemplate.from_template(
        """You are an expert in science. \
        Always answer questions starting with "뉴턴선생님께서 말씀하시기를..". \
        Respond to the following question:

        Question:{question}
        Answer:"""
    )
    | ChatOpenAI(model="gpt-4o-mini")
)

general_chain = (
    PromptTemplate.from_template(
        """Respond to the following question concisely:

        Question:{question}
        Answer:"""
    )
    | ChatOpenAI(model="gpt-4o-mini")
)
```

LCEL > Runnable > 예시 4(2/2)

RunnableBranch, RunnablePassthrough

```
# RunnableBranch 로 분기
from langchain_core.runnables import RunnableBranch

branch = RunnableBranch(
    # 주제에 "수학"이 포함되어 있는 경우, math_chain을 실행
    (lambda x: "수학" in x["topic"].lower(), math_chain),
    # 주제에 "과학"이 포함되어 있는 경우, science_chain을 실행
    (lambda x: "과학" in x["topic"].lower(), science_chain),
    # 그 외의 경우 general_chain을 실행
    general_chain,
)

from operator import itemgetter
from langchain_core.runnables import RunnablePassthrough

full_chain = (
    {"topic": chain, "question": RunnablePassthrough()}
    | branch
    | StrOutputParser()
)
```

1 # 수학과 관련된 질문을 입력하여 체인을 호출합니다.

2 full_chain.invoke("미적분의 개념에 대해 말씀해 주세요.")

'파스칼선생님께서 말씀하시기를, 미적분은 수학의 한 분야로, 주로 변화와 관련된 문제를 다룹니다. 미적분은 크게 미분과 적분으로 나뉘어 있으며, 미분은 함수의 변화율, 즉 기울기를 다루고, 적분은 넓이, 즉 함수 아래의 면적을 계산하는 방법입니다. 미분은 특정 점에서 함수가 얼마나 빠르게 변화하는지를 나타내며, 실제로는 순간적인 변화율을 측정하는 데 사용됩니다. 예를 들어, 자동차의 속도를 구할 때, 위치 함수의 미분을 통해 순간속도를 계산할 수 있습니다. 반면에 적분은 주어진 구간에서 함수의 면적을 구하는 것으로, 이는 주로 누적량을 계산할 때 사용됩니다. 예를 들어, 일정 시간 동안의 이동 거리나 물체의 부피를 구할 때 적분이 필요합니다. 결국, 미적분은 자연과학, 공학, 경제학 등 다양한 분야에서 중요한 역할을 하며, 현실 세계의 문제를 수학적으로 모델링하고 해결하는 데 필수적인 도구입니다.'

1 # 과학과 관련된 질문을 입력하여 체인을 호출합니다.

2 full_chain.invoke("중력은 어떻게 작용하나요?")

'뉴턴선생님께서 말씀하시기를, 중력은 두 물체 사이의 질량에 비례하고 거리의 제곱에 반비례하는 힘이라고 하셨습니다. 즉, 질량이 클수록 중력은 강해지고, 두 물체 간의 거리가 멀어질수록 중력의 세기는 약해집니다. 이 중력의 법칙은 모든 물체에 적용되며, 지구가 물체를 끌어당기는 힘 또한 이 중력에 의해 발생합니다. 이를 통해 우리는 왜 사물이 지구로 떨어지는지, 그리고 행성들이 태양 주위를 도는지를 이해할 수 있습니다.'

1 # 기타 질문을 입력하여 체인을 호출합니다.

2 full_chain.invoke("RAG(Retrieval Augmented Generation)은 무엇인가요?")

'RAG(출처 증강 생성)는 정보 검색 기술과 텍스트 생성 모델을 결합하여, 기반 데이터베이스에서 관련 정보를 검색한 뒤 이를 활용해 더 정교하고 정확한 응답을 생성하는 방법입니다. 이 방식은 기존의 텍스트 생성 모델이 자주 접하는 제한된 정보의 문제를 보완합니다.'

Tool Calling

LLM이 단순히 텍스트를 생성하는 것을 넘어서, 외부 도구(API, 함수, 계산기, 데이터베이스 등)를 직접 호출할 수 있게 하는 LangChain의 핵심 기능

Tool 유형

- 도구(Tool)는 에이전트, 체인 또는 LLM이 외부 세계와 상호작용하기 위한 인터페이스

< 빌트인 도구 (built-in tools) >

- 랭체인에서 제공하는 사전에 정의된 도구(tool) 와 툴킷(toolkit)
- tool 은 단일 도구를 의미하며, toolkit 은 여러 도구를 묶어서 하나의 도구로 사용할 수 있음

< 사용자 정의 도구 (Custom Tool) >

- LangChain 에서 제공하는 빌트인 도구 외에도 사용자가 직접 도구를 정의하여 사용할 수 있음
- langchain.tools 모듈에서 제공하는 tool 데코레이터를 사용하여 함수를 도구로 변환

< LLM 한계 >

- 실시간 데이터 접근 및 처리
- 수학적 계산
- 환각 효과(hallucination)

Tool 사용 사례

Tool 사용 사례	예시
▪ 날씨 조회	OpenWeather API 호출
▪ 일정 관리	Google Calendar API 연결
▪ 수학 계산기	math 모듈 연동
▪ 데이터 검색	RAG(VectorStore) 검색 Tool
▪ DB 조회	SQL 쿼리 자동 생성 및 실행

예시

< Tool 사용 장점 >

(확장성) LLM+실제 기능(API,DB등)을 결합
 (지능적선택) 모델이 언제 어떤 도구를 쓸지 스스로 판단
 (자동인자매핑) LLM이 JSON형태로 인자 생성
 (명확한로그/추적) 어떤 함수가 호출되었는지 쉽게 로깅
 (안전성제어) 허용된 함수만 실행되도록 제한

Tool Calling > 빌트인 도구(Built-in Tools)

All tools and toolkits

  ADS4GPTs View guide >	  AgentPhone Toolkit View guide >	  AgentQL View guide >
  Ampersend View guide >	  Amazon Bedrock AgentCore Browser View guide >	  Amazon Bedrock AgentCore Code Interpreter View guide >
  Anchor Browser View guide >	  Apify View guide >	  AWS Lambda View guide >




Tavily Search

[View guide >](#)

TavilySearchResults

- Tavily 검색 API를 쿼리하고 JSON 형식의 결과를 반환
- 포괄적이고 정확하며 신뢰할 수 있는 결과에 최적화된 검색 엔진
- 주요 매개변수
 - max_results (int): 반환할 최대 검색 결과 수(기본값: 5)
 - search_depth (str): 검색 깊이("basic" 또는 "advanced")
 - include_domains (List[str]): 검색 결과에 포함할 도메인 목록
 - exclude_domains (List[str]): 검색 결과에서 제외할 도메인 목록
 - include_answer (bool): 원본 쿼리에 대한 짧은 답변 포함 여부
 - include_raw_content (bool): 각 사이트의 정제된 HTML 콘텐츠 포함 여부
 - include_images (bool): 쿼리 관련 이미지 목록 포함 여부

Tool Calling > 빌트인 도구(Built-in Tools)

▪ 검색 도구 - Tavily

1. API 키 발급 -> <https://app.tavily.com/>
2. 발급한 API 키를 환경변수에 설정 (.env 파일)
 - .env파일 -> TAVILY_API_KEY=tvly-abcdefghijklmnopqrstuvwxyz

```
!pip install -qU langchain-tavily
```

```
from langchain_tavily import TavilySearch
```

```
# 도구 생성
```

```
tool = TavilySearch(  
    max_results=6,  
    include_answer=True,  
    include_raw_content=True,  
    include_domains=["github.io", "wikidocs.net"],  
)
```

```
# 도구 실행
```

```
tool.invoke({"query": "LangChain Tools 에 대해서 알려주세요"})
```

Tool Calling > 사용자 정의 도구(Custom Tool)

▪ @tool 데코레이터

- 일반 Python 함수를 강력한 도구로 쉽게 변환할 수 있으며, 동화된 문서화와 유연한 인터페이스 생성이 가능
- Tool 작성 방법
① 함수 작성 → ② 타입 힌팅 → ③ Docstring으로 함수(Tool) 설명 → ④ @tool 데코레이터 입력

```
from langchain.tools import tool

# 데코레이터를 사용하여 함수를 도구로 변환합니다.
@tool
def add_numbers(a: int, b: int) -> int:
    """Add two numbers"""
    return a + b

@tool
def multiply_numbers(a: int, b: int) -> int:
    """Multiply two numbers"""
    return a * b
```

```
# 도구 실행
add_numbers.invoke({"a": 3, "b": 4})
```

7

```
# 도구 실행
multiply_numbers.invoke({"a": 3, "b": 4})
```

12

4. Basic Agent

- Basic Agent
- Tool 사용
- Short-term Memory
- 구조화된 답변

실습 파일 :

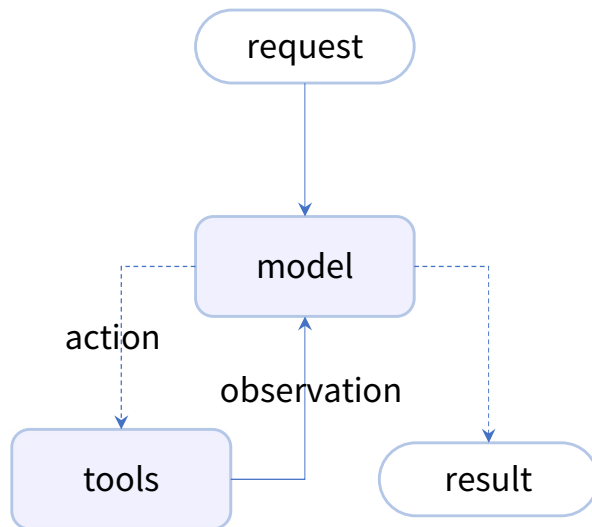
5. LangChain_[4]_Basic_Agent.ipynb

본 문서는 SK(주)AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

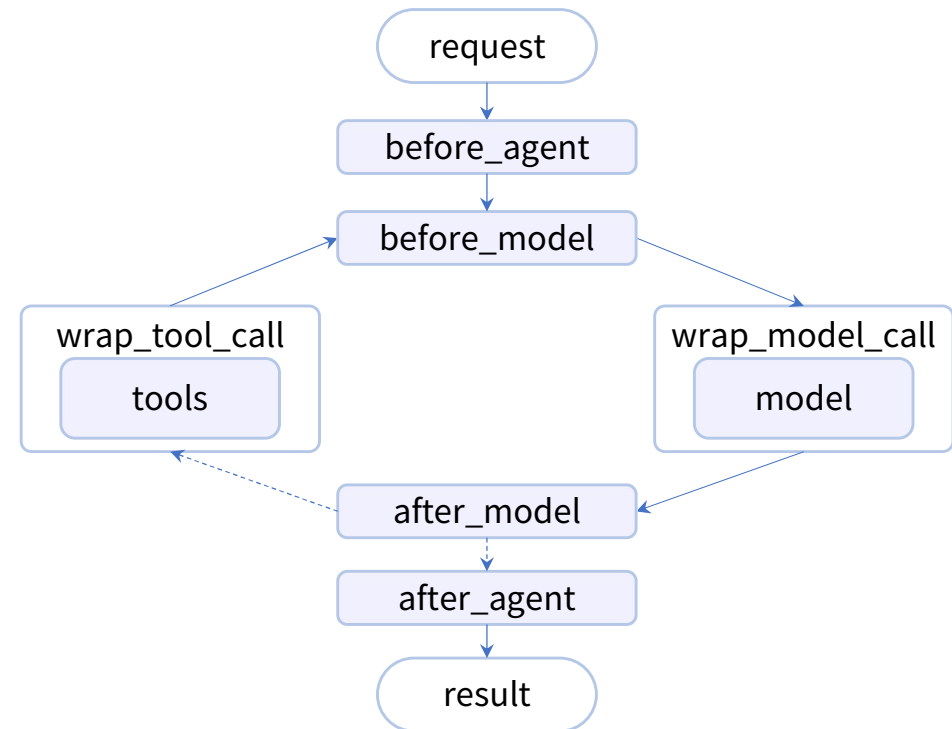
Agent

Agent는 사용자의 입력(질문, 지시)에 대해 적절한 행동을 취하도록 **추론(Reasoning)**한 뒤 **행동(Act)**하는 **주체(Executor)**임

【 Basic Agent 】

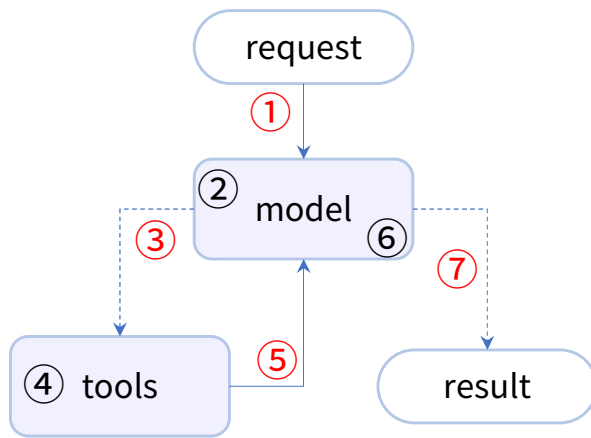


【 Advanced Agent 】



Basic Agent

Agent Flow



- ① Request (요청) : 사용자가 AI에게 질문이나 명령을 함 (**HumanMessage**)
- ② Model (추론/판단) : 모델이 요청을 받고 생각하고, 툴이 필요한지 판단
- ③ Action (행동 호출) : 모델이 필요한 도구(tools)에게 명령 (**AIMessage - Too_calls**)
- ④ Tools (도구 실행) : 날씨 API 등 도구를 실행하여 실제 데이터를 가져옴
- ⑤ Observation (관찰/결과 피드백) : 도구가 찾아낸 원시 데이터를 모델에게 전달 (**ToolMessage**)
- ⑥ Model (최종 종합): 모델은 도구의 결과를 바탕으로 종합적인 결론을 내림
- ⑦ Result (최종 결과): 사용자가 읽을 수 있는 정돈된 형태의 최종 답변을 출력 (**AIMessage**)

🧠 Model (모델 - 두뇌 역할)

- 사용자의 요청(request)을 해석하고 문제를 해결하기 위한 계획을 세움
- 자신이 가진 내부 지식만으로 답할 수 없는 경우(예: 최신 정보 검색, 복잡한 수학 계산), 어떤 도구(tools)를 사용해야 할지 판단
- 도구가 돌려준 결과(observation)를 보고, 추가로 도구를 더 쓰지 아니면 최종 정답(result)을 도출할지 최종 결정

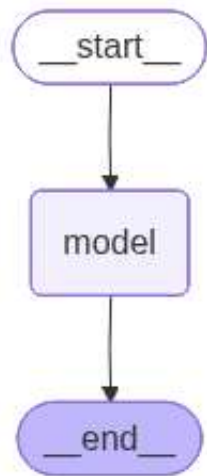
🔧 Tools (도구 - 손과 발 역할)

- 모델이 가질 수 없는 '외부 기능이나 실시간 데이터에 접근할 수 있는 수단'
- 모델을 대신해 웹 검색, 계산기, 캘린더, 날씨 API 등의 실제 작업을 수행
- 모델이 내린 명령(action)을 실행 뒤, 그 물리적인 결과값을 다시 모델에게 전달

Agent 객체 생성

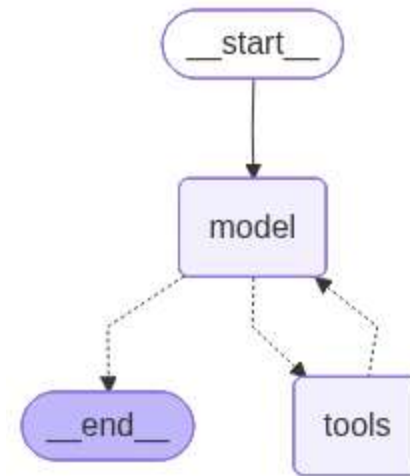
```
from langchain.agents import create_agent
```

```
agent = create_agent(  
    model= model,  
)
```

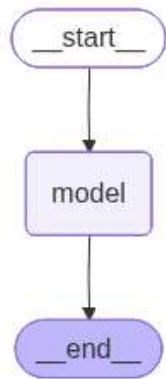


```
from langchain.agents import create_agent
```

```
agent = create_agent(  
    model= model, tools = [get_weather]  
)
```



Tool 없는 Agent



```

from langchain.chat_models import init_chat_model

model = init_chat_model("gpt-5-nano")

response = model.invoke("현재 한국 날씨 어때?")

response.pretty_print()
  
```

===== Ai Message =====

현재 한국의 실시간 날씨를 바로 조회해 드리려면 최신 데이터가 필요해요. 저는 실시간 정보를 직접 확인할 수 없어서 정확한 현황을 바로 알려

다음 중 어떤 방식으로 도와드릴까요?

- 특정 도시를 알려주면 오늘의 날씨를 요약해 드리겠습니다(예: 서울, 부산 등).
- 실시간 정보를 직접 확인하는 방법을 안내해 드릴게요: 기상청(www.kma.go.kr), 네이버 날씨, 날씨 앱 등에서 현재 위치를 검색하면 현재 기온

원하는 도시를 알려주실 수 있을까요? 예를 들어 “서울 현재 날씨” 처럼요.

Tool 사용 > (1)날씨

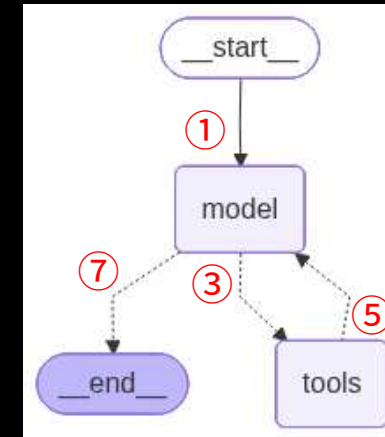
```
from langchain.tools import tool
```

```
@tool
def get_weather(location: str) -> str:
    "이 도구는 특정 지역의 날씨 정보를 반환함."
    return f"오늘 {location} 날씨는 비가 옵니다."
```

```
from langchain.agents import create_agent
```

```
agent = create_agent(
    model = model,
    tools = [get_weather]
)
```

```
result = agent.invoke(
    {"messages": [{"role": "user", "content": "오늘 서울 날씨 어때요? 날씨에 따라 옷 스타일을 추천해주세요"}]},
)
```

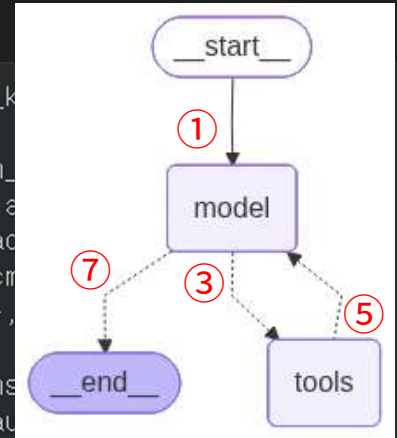


Tool 사용 > (1)날씨

1 result

①

```
{'messages': [HumanMessage(content='오늘 서울 날씨 어때요? 날씨에 따라 옷 스타일을 추천해주세요', additional_kwargs={}, response_metadata={}, id='1b6b574b-9cbe-442b-978e-b6c074ae8d3a'),
  ③ AIMessage(content='', additional_kwargs={'refusal': None}, response_metadata={'token_usage': {'completion_tokens': 151, 'prompt_tokens': 151, 'total_tokens': 366, 'completion_tokens_details': {'accepted_prediction_tokens': 0, 'audio_tokens': 0, 'reasoning_tokens': 192, 'rejected_prediction_tokens': 0}, 'prompt_tokens_details': {'audio_tokens': 0, 'cache_read': 0, 'cache_write': 0}, 'model_provider': 'openai', 'model_name': 'gpt-5-nano-2025-08-07', 'system_fingerprint': None, 'id': 'chatcmpl-DhYpNn9wXeEAjvIWz2Dn28kUTvgZP', 'service_tier': 'default', 'finish_reason': 'tool_calls', 'logprobs': None}, 'invalid_tool_calls': [], usage_metadata={'input_tokens': 11da-7302-98ce-5dc35eff4fc9-0', tool_calls=[{'name': 'get_weather', 'args': {'location': '서울'}, 'id': 'call_aLazve0dREX03b5wPHqsSegr', 'type': 'tool_call'}], invalid_tool_calls=[], usage_metadata={'input_tokens': 215, 'total_tokens': 366, 'input_token_details': {'audio': 0, 'cache_read': 0}, 'output_token_details': {'audio': 192}}),
  ⑤ ToolMessage(content='오늘 서울 날씨는 비가 옵니다.', name='get_weather', id='1c0015fb-1313-49e8-93a3-06f481cc8f0f', tool_call_id='call_aLazve0dREX03b5wPHqsSegr'),
  ⑦ AIMessage(content='오늘 서울은 비가 와요. 우산과 방수 아이템을 챙기시는 게 좋습니다.\n\n비 오는 날에 어울리는 코디 아이디어\n- 출근/등교용\n  - 겹옷: 방수 트렌치 코트나 빗방울을 잘 막는 방수 자켓\n  - 이너: 얇은 니트나 셔츠 위에 가벼운 니트를 레이어링\n  - 하의: 슬림 핏 팬츠나 워싱이 밝은 청바지\n  - 신발: 방수 로퍼나 고무가죽 혼합 부츠, 가능하면 바닥이 미끄럽지 않은 편\n  - 악세서리: 우산과 방수 백팩 커버로 소지품 보호\n\n- 캐주얼/나들이용\n  - 겹옷: 방수 후드 자켓이나 경량 비옷\n  - 이너: 맨투맨 또는 폴리스\n  - 하의: 데님이나 면바지\n  - 신발: 방수 스니커나 고무부츠\n  - 포인트: 밝은 색 포인트 아이템으로 분위기 살리기\n\n- 포멀한 만남/외출용(날씨가 조금 쌀쌀할 때)\n  - 겹옷: 방수 트렌치 + 블레이저 조합\n  - 이너: 셔츠와 가벼운 니트 혹은 니트 카디건\n  - 하의: 다크 컬러 슬림핏 팬츠\n  - 신발: 방수 로퍼나 가죽 부츠(신발에 방수 스프레이를 미리 뿌려두면 좋습니다)\n\n비가 올 때의 실용 팁\n- 신발은 방수 또는 비에 젖어도 빨리 마르는 재질 권장. 가능하면 고무 부츠나 방수 스니커로 교체.\n- 바람이 있다면 얇은 이너로 체온 손실을 막고, 겹옷은 방수 기능이 있는 것으로 선택.\n- 우산은 바람에 잘 찢어지지 않는 튼튼한 타입으로.\n\n오늘 일정이나 선호하시는 스타일이 있다면 말씀해 주세요. 직장/학교, 예산, 선호 색상에 맞춰 더 구체적인 코디를 추천해 드릴게요.', additional_kwargs={'refusal': None}, response_metadata={'token_usage':
```



Tool 사용 > (2)사칙 연산

```
from langchain.tools import tool

@tool
def add(a: int, b: int) -> int:
    """`a`와 `b` 더셈.

    Args:
        a: First int
        b: Second int
    """
    return a + b

@tool
def subtract(a: int, b: int) -> int:
    """`a`에서 `b`를 빼기.

    Args:
        a: First int
        b: Second int
    """
    return a - b
```

```
@tool
def multiply(a: int, b: int) -> int:
    """`a`와 `b` 곱셈.

    Args:
        a: First int
        b: Second int
    """
    return a * b

@tool
def divide(a: int, b: int) -> float:
    """`a`와 `b` 나눗셈.

    Args:
        a: First int
        b: Second int
    """
    return a / b
```

```
tools = [add, subtract, multiply, divide]
```

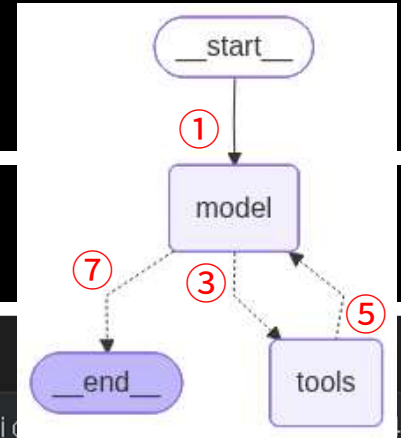
Tool 사용 > (2)사칙 연산

```
agent = create_agent(
    model = model,
    tools = tools
)
```

```
result = agent.invoke(
    {"messages": [{"role": "user", "content": "30 + 5 * 2 은 뭔가요?"}]},
)
```

1 result

```
{'messages': [HumanMessage(content='30 + 5 * 2 은 뭔가요?', additional_kwargs={}, response_metadata={}, id='1303dc47e1d6'),
  ⑦ AIMessage(content='40입니다. 연산 순서대로 보면 곱셈부터 먼저 수행해 5×2=10이고, 그다음 30+10=40이 됩니다.', additional_kwargs=
{'refusal': None}, response_metadata={'token_usage': {'completion_tokens': 366, 'prompt_tokens': 287, 'total_tokens': 653,
```

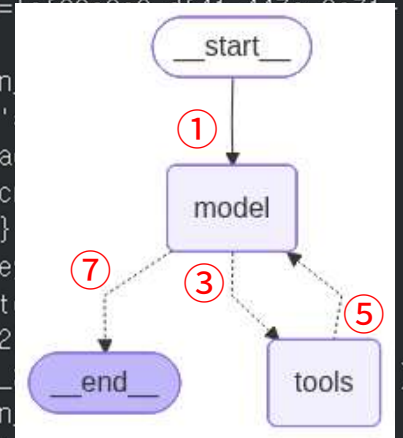


```
agent = create_agent(
    model,
    tools,
    system_prompt="당신은 사칙연산할 때 무조건 도구를 호출하세요"
)
```

```
result = agent.invoke(
    {"messages": [{"role": "user", "content": "42 + 3 * 23은 뭔가요?"}]},
)
```


1 result

```
{
  "messages": [
    HumanMessage(content='42 + 3 * 23은 뭔가요?', additional_kwargs={}, response_metadata={}, id='09bca9bdbae5'),
    AIMessage(content='', additional_kwargs={'refusal': None}, response_metadata={'token_usage': {'completion_tokens': 1, 'prompt_tokens': 307, 'total_tokens': 845, 'completion_tokens_details': {'accepted_prediction_tokens': 0, 'reasoning_tokens': 512, 'rejected_prediction_tokens': 0}, 'prompt_tokens_details': {'audio_tokens': 0, 'cached_tokens': 0}}, 'model_provider': 'openai', 'model_name': 'gpt-5-nano-2025-08-07', 'system_fingerprint': None, 'id': 'chatcmpl-DhX19pfG4G0mdD5yMnr8qkvXINCII', 'service_tier': 'default', 'finish_reason': 'tool_calls', 'logprobs': None}, id='lc_run--019e4499-ddcc-72f1-a4ee-2cece1e8a091-0', tool_calls=[{'name': 'multiply', 'args': {'a': 3, 'b': 23}, 'id': 'call_2ye...', 'type': 'tool_call'}], invalid_tool_calls=[], usage_metadata={'input_tokens': 307, 'output_tokens': 538, 'total_tokens': 845, 'input_token_details': {'audio': 0, 'cache_read': 0}, 'output_token_details': {'audio': 0, 'reasoning': 512}}),
    ToolMessage(content='69', name='multiply', id='1360c485-40dd-4728-b980-f31318d606d4', tool_call_id='call_2ye...'),
    AIMessage(content='', additional_kwargs={'refusal': None}, response_metadata={'token_usage': {'completion_tokens': 1, 'prompt_tokens': 338, 'total_tokens': 358, 'completion_tokens_details': {'accepted_prediction_tokens': 0, 'audio_tokens': 0, 'reasoning_tokens': 0, 'rejected_prediction_tokens': 0}, 'prompt_tokens_details': {'audio_tokens': 0, 'cached_tokens': 0}}, 'model_provider': 'openai', 'model_name': 'gpt-5-nano-2025-08-07', 'system_fingerprint': None, 'id': 'chatcmpl-DhX1DBaB8CHrIJCroKqkqHgVYZei', 'service_tier': 'default', 'finish_reason': 'tool_calls', 'logprobs': None}, id='lc_run--019e4499-eff3-71c0-8212-89068f864121-0', tool_calls=[{'name': 'add', 'args': {'a': 42, 'b': 69}, 'id': 'call_uMIYCUYRf6Xmft1TMxLLPsER', 'type': 'tool_call'}], invalid_tool_calls=[], usage_metadata={'input_tokens': 338, 'output_tokens': 20, 'total_tokens': 358, 'input_token_details': {'audio': 0, 'cache_read': 0}, 'output_token_details': {'audio': 0, 'reasoning': 0}}),
    ToolMessage(content='111', name='add', id='2ce1b87c-9241-4494-8351-306b8cb7ea0f', tool_call_id='call_uMIYCUYRf6Xmft1TMxLLPsER'),
    AIMessage(content='답: 111\n\n설명: 먼저 3 * 23 = 69이고, 그다음 42 + 69 = 111입니다. 따라서 42 + 3 * 23은 111입니다. (곱셈이 덧셈보다 먼저 수행됩니다.)', additional_kwargs={'refusal': None}, response_metadata={'token_usage': {'completion_tokens': 454, 'prompt_tokens': 369, 'total_tokens': 823, 'completion_tokens_details': {'accepted_prediction_tokens': 0, 'audio_tokens': 0, 'reasoning_tokens': 384, 'rejected_prediction_tokens': 0}, 'prompt_tokens_details': {'audio_tokens': 0, 'cached_tokens': 0}}, 'model_provider': 'openai', 'model_name': 'gpt-5-nano-2025-08-07', 'system_fingerprint': None, 'id': 'chatcmpl-DhX1E97KMISzwMm8dkwozFOW0p9nd', 'service_tier': 'default', 'finish_reason': 'stop', 'logprobs': None}, id='lc_run--019e4499-f2b4-7802-b99a-03c44566d4b0-0', tool_calls=[], invalid_tool_calls=[], usage_metadata={'input_tokens': 369, 'output_tokens': 454, 'total_tokens': 823, 'input_token_details': {'audio': 0, 'cache_read': 0}, 'output_token_details': {'audio': 0, 'reasoning': 384}})]
  }
}
```



Tool 사용 > (3)알라딘

- 알라딘 키 발급 (회원 가입 필요) : <https://blog.aladin.co.kr/openapi/popup/6695306>

☆ OpenAPI 이용 안내 | API 이용 안내  댓글(0)

<https://blog.aladin.co.kr/openapi/5353304> 주소복사 OpenAPI | 2012-01-13 16:16

1. OpenAPI 이용 조건

- 개인이 비영리적인 목적으로 사용하는 경우 별도 조건 없이 사용 가능합니다.
- 공공도서관 및 공공기관의 경우 아래 조건을 충족하는 경우 이용하실 수 있습니다.
 - 정보제공이 알라딘에서 되고 있다는 사실을 이용자들이 인지할 수 있도록 명시
 - 해당 상품이 알라딘 소개페이지로 연결 되어 이용자들이 알라딘에서 구매가 가능해야 함
 - 위 2가지 조건을 충족한다면 알라딘에 가입하신 이메일, OpenAPI Key, 위 조건 충족을 확인 할 수 있는 화면캡처 또는 사이트 주소나 앱 이름을 openapi@aladin.co.kr로 보내주시기 바랍니다.
 - 도서관 구축 개발/사업체의 경우, 업체명의 계정으로 OpenAPI 프리미엄서비스를 이용할 수 없으며, 도서관 법인 명의로 알라딘 회원가입 후 OpenAPI프리미엄을 이용할 수 있습니다.
- 도서 판매, 도서 정보를 기반으로 하는 법인/기업/단체 서비스, 영리 목적의 서비스인 경우 이용이 불가능합니다.

2. OPEN API 이용 절차 (OpenAPI Key 발급)

- 알라딘 회원 가입 후 OpenAPI를 사용할 URL정보를 등록해주세요.

▶ API 키 발급 및 URL 등록하기

- 오픈 API 인증 키

알라딘 OpenAPI

TTB 리포트 판매통계

- 기간별 통계
- 판매 상품 통계

알라딘 Open API 안내

- 검색 API
- 상품 API
- API 사용예제
- API 이용 안내
- 이용약관

알라딘 메뉴

- 나의계정
- 회원정보 변경

OpenAPI 사용 URL 예 : www.mydomain.com

http://humanv.com

추가

URL 주소를 등록해 주세요. (사이트 또는 앱 다운로드 링크)

확인 할 수 있는 URL이 없는 경우 신청자의 SNS / 블로그 등의 주소를 입력해주세요.

알라딘 OpenAPI 이용약관

1. 목적

알라딘OpenAPI서비스 이용약관(이하 '본 약관'이라 합니다)은 <주>알라딘커뮤니케이션(이하 '회사'라 합니다)과 이용자 간에 회사가 제공하는 OpenAPI(이하 API) 서비스의 제공 및 이용과 관련한 회사와 이용자의 권리, 의무 및 책임에 관한 제반 사항과 기타 필요한 사항을 구체적으로 규정함을 목적으로 합니다.

☐ 이용약관에 동의합니다.

등록된 URL 목록

URL	Open API 인증키	관리

Tool 사용 > (3)알라딘

```
from langchain.tools import tool
from typing import List, Dict, Any

import requests

@tool
def fetch_aladin_bestseller_topN(top_n:int) -> List[Dict[str, Any]]:
    """
    알라딘 베스트셀러 목록을 조회하고 Top N개(기본 1)를 반환합니다.

    Args:
        top_n: Top 개수 (값이 없을때는 1로하며, 10을 초과하는 경우 10으로 함)
    """
    url = "http://www.aladin.co.kr/ttb/api/ItemList.aspx"
    params = {
        "ttbkey": "key.....", # Key 입력
        "Query Type": "Bestseller",
        "MaxResults": top_n,
        "start": 1,
        "SearchTarget": "Book",
        "output": "js",
        "Version": "20131101"
    }
    resp = requests.get(url, params=params)
    resp.raise_for_status()
    data = resp.json()
    items = data.get("item", [])
    top10 = items[:10]
    return top10
```

Tool 사용 > (3)알라딘

4. Basic Agent

```
agent = create_agent(
    model,
    [fetch_aladin_bestseller_topN],
)
```

```
response = agent.invoke(
    {"messages": [{"role": "user", "content": "현재 알라딘에서 베스트셀러 Top 4가 뭐  
야?"}],
)

response['messages'][-1].content
```

'다음이 현재 알라딘 베스트셀러 Top 4입니다. #n#n1) 일리아스 좋아하세요?
- 어느 덕후와 교수의 고전 교환독서 #n - 저자: 하길(석민주), 이준석 #n
- 출판사: 창비 #n - 판매가: 17,100원 (정가 19,000원) #n#n2) 프로젝트 헤
일메리 #n - 저자: 앤디 위어; 옮김: 강동혁 #n - 출판사: 알에이치코리아
(RHK) #n - 판매가: 19,800원 (정가 22,000원) #n#n3) 안녕이라 그랬어 #n
- 저자: 김애란 #n - 출판사: 문학동네 #n - 판매가: 15,120원 (정가 16,8
00원) #n#n4) 도쿄 에일리언즈 12 (트리플 특장판) #n - 저자: NAOE; 옮김:
박소현 #n - 출판사: 서울미디어코믹스(서울문화사) #n - 판매가: 5,400원
(정가 6,000원) #n#n필요하시다면 각 책의 바로 가기 링크도 함께 드릴게
요.'

베스트셀러

국내도서 전자책 외국도서 알라딘굿즈 온라인종교 음반 블루레이

주간 베스트 지금 베스트 어제 베스트 특가 베스트 북플 베스트 신간 베스트 스테디셀러 월간 >

종합 2026년 5월 3주

최근 1주 동안 가장 많은 고객들이 구매한 국내도서 순위입니다.

1위 51위 101위 151위 201위 251위 301위 351위 401위 451위

전체선택 장바구니 담기 보관함 담기 마이리스트 담기

1. **일리아스** 5월 특별 선물, 넉넉한 보양책 · 가장 속 읽힐래 (이벤트 도서 포함 국내도서 5만 원 이상)
[국내도서] 일리아스 좋아하세요? - 어느 덕후와 교수의 고전 교환독서
하길(석민주), 이준석(지은이) | 창비 | 2026년 5월
19,000원 → **17,100원** (10%할인), 마일리지 950원
★ 9.5 (44) | 세일즈포인트: 22,180
양탄자배송 내일 아침 7시 출근전 배송 지역변경

2. **프로젝트 헤일메리** 종합 1위 6주 프로젝트 헤일메리 금속 키워, 홀로그램 양장노트, 텀블러(앤디 위어 우주 3부작 포함 국내도서 3만 원 이상 구매 시 선택)
[국내도서] 프로젝트 헤일메리 | 앤디 위어 우주 3부작
앤디 위어(지은이), 강동혁(옮김) | 알에이치코리아(RHK) | 2021년 5월
22,000원 → **19,800원** (10%할인), 마일리지 1,100원
★ 9.4 (207) | 세일즈포인트: 430,562
양탄자배송 내일 아침 7시 출근전 배송 지역변경
이 책의 전자책: 13,860원 보기가기

3. **안녕이라 그랬어** 종합 Top10 27주 피크닉 매트 (오늘의 한국문학 2종 이상)
[국내도서] 안녕이라 그랬어 **Choice**
김애란(지은이) | 문학동네 | 2025년 6월
16,800원 → **15,120원** (10%할인), 마일리지 840원
★ 9.5 (141) | 세일즈포인트: 299,052
양탄자배송 내일 아침 7시 출근전 배송 지역변경
이 책의 전자책: 10,620원 보기가기

12만원 캐시백
삼성카드 6개월 미이용 고객 대상

Tool 사용 > (4)챗봇

```
@tool
def get_current_time() -> str:
    """현재 시간을 반환합니다."""

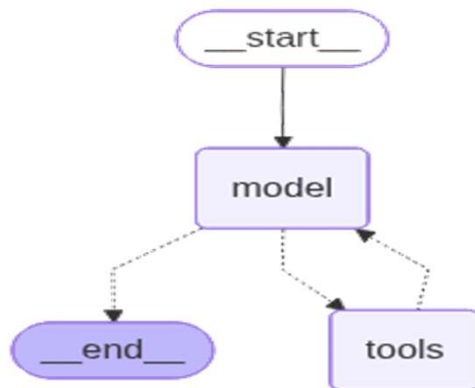
    # KST는 UTC+9
    kst = timezone(timedelta(hours=9))
    now_kst = datetime.datetime.now(kst)
    return now_kst.strftime("%Y년 %m월 %d일 %H시 %M분")
```

```
@tool
def calculate(expression: str) -> float:
    """수학 계산을 수행합니다.

    Args:
        expression: 계산할 수식 (예: "2 + 3 * 4")
    """
    try:
        result = eval(expression)
        return f"{expression} = {result}"
    except Exception as e:
        return f"계산 오류: {e}"
```

```
@tool
def get_weather(city: str) -> str:
    """도시의 날씨를 조회합니다. 도시명은 한글로 해줘요

    Args:
        city: 도시 이름 (예: '서울', '부산', '제주')
    """
    # 실제로는 날씨 API 호출
    # 여기서는 예시 데이터 반환
    weather_data = {
        "서울": "맑음 ☀ 22°C",
        "부산": "흐림 ☁ 20°C",
        "제주": "비 🌧 18°C"
    }
    return weather_data.get(city, f"{city}의 날씨 정보를 찾을 수 없습니다.")
```



```
# 도구 목록
tools = [get_current_time, calculate, get_weather]
```

```
agent = create_agent(
    model=model,
    tools = tools
)
```

Tool 사용 > (4) 챗봇



```

1 # 대화형 질의
2 print("질문을 입력하세요 (종료: exit or quit)")
3 while True:
4     question = input("\n질문: ")
5     if question.lower() in ['exit', 'quit', '종료', '끝']:
6         print("종료합니다.")
7         break
8
9     response = agent.invoke({"messages": [{"role": "user", "content": question}]}))
10
11     # print(f"답변: {response}")
12     print(f"답변: {response['messages'][-1].content}")

```

↑ ↓ ✎ 🗑 ⋮

... 질문을 입력하세요 (종료: exit or quit)

질문: 지금 몇시예요?

답변: 현재 시각은 2026년 5월 27일 15시 20분입니다.

질문: 제주 날씨 알려줘요

답변: 제주 현재 날씨: 비가 오고 있고 기온은 18° C입니다 🌧

추가로 시간대별 예보나 주간 예보가 필요하시면 알려주세요.

질문: Agent를 50자 내로 정의한다면?

답변: 목표 달성을 위해 환경에서 자율적으로 행동하는 소프트웨어 에이전트.

질문:

@tool
def get_current_time() -> str:

@tool
def get_weather(city: str) -> str:

단기 메모리

다중 턴 대화 시 대화 맥락을 고려한 Tool 호출을 위해 메모리 관리 필요

Memory

● 단기 메모리 (Short-term memory) : Checkpointer를 통해 대화 스레드(세션) 저장

- 하나의 대화 스레드(세션) 안에서만 맥락이 이어지는 기억
- 하나의 '채팅방' 안에서는 앞서 했던 말을 기억하고 문맥에 맞는 답변을 해주는 것이 단기 메모리를 사용하는 것임
- 채팅방을 나가거나 새로 고침 하면 이 기억은 보통 초기화 됨

checkpointer (기억 보관소):

에이전트가 사용자와 나눈 대화 기록(상태, State)을 실제로 저장하고, 필요할 때 다시 꺼내오는 물리적/논리적인 저장소

thread_id (기억의 열쇠):

수많은 사용자의 수많은 대화 기록 중, '지금 답변해야 할 사용자의 대화 기록이 어떤 것인지' 식별하는 고유한 키 값

- ✓ 에이전트 호출 시 같은 `thread_id`를 넘겨주면 ➡ "아, 아까 그 사람이구나!" 하고 기존 대화 맥락을 이어서 답변
- ✓ 에이전트 호출 시 다른 `thread_id`를 넘겨주면 ➡ "처음 뵙겠습니다!" 하고 완전히 새로운 대화를 시작

● 장기 메모리 (Long-term memory) : Store를 통해 장기 메모리 저장

- 세션(채팅방)을 넘나들며 영구적으로 유지되어야 하는 핵심 기억
- 사용자의 직업, 선호하는 말투, 알레르기 정보 같은 개인 프로필이 여기에 해당 됨
- ChatGPT의 '개인 맞춤 설정'에 직업을 입력해 두면, 어떤 채팅방을 새로 열든 그 정보를 바탕으로 답변하는 것이 장기 메모리의 대표적인 예 임

단기 메모리

```
from langchain.agents import create_agent
from langgraph.checkpoint.memory import InMemorySaver
```

```
agent = create_agent(
    model,
    tools,
    checkpointer = InMemorySaver(),
)
```

```
response = agent.invoke(
    {"messages": [
        {"role": "user",
         "content": "안녕하세요. 저는 Jumany입니다."}
    ]},
    {"configurable": {"thread_id": "thread_1"}},
)
```

```
response = agent.invoke(
    {"messages": [
        {"role": "user",
         "content": "안녕하세요. 제 이름이 뭐죠?"}
    ]},
    {"configurable": {"thread_id": "thread_1"}},
)
```

--- Message 1 : human ---

안녕하세요. 저는 Jumany입니다.

--- Message 2 : ai ---

반갑습니다, Jumany님! 무엇을 도와드릴까요?

예시:

- 날씨 확인: 도시 이름을 한글로 알려주시면 됩니다(예: 서울, 부산).
- 수학 계산: 수식을 입력해 주세요(예: $2 + 3 * 4$).
- 현재 시간: 원하시면 지금 시각을 알려드립니다.
- 그 외 정보 검색, 간단한 번역 등도 도와드려요.

어떤 걸 도와드릴까요?

--- Message 3 : human ---

안녕하세요. 제 이름이 뭐죠?

--- Message 4 : ai ---

네, Jumany님. 당신의 이름은 Jumany입니다. 다른 도움이 필요하신가요?

단기 메모리

```
response = agent.invoke(  
    {"messages": [  
        {"role": "user",  
         "content": "안녕하세요. 제 이름이 뭐죠?"}  
    ]  
    },  
    {"configurable": {"thread_id": "thread_2"}},  
)
```

```
--- Message 1 : human ---  
안녕하세요. 제 이름이 뭐죠?
```

```
--- Message 2 : ai ---  
안녕하세요! 이 대화에서만 사용할 제게는 아직 당신의 이름이 전달되지 않았어요. 이름을 알려주시면 이 대화 동안 그 이름으로 부를게요. 원하시면 “손님” 이라고 불러드릴 수
```

구조화된 답변

Structured Outputs

Agent의 실행 결과를 자연어가 아닌, 특정 포맷의 구조화된 데이터로 파싱 가능

예시) Agent가 메일의 의도, 감정 상태, 요약, 다음 수행 업무 등을 분석 후 구조에 맞춰 데이터 생성 → 구조화된 답변 생성



구조화된 답변

① Tool 정의

```
from langchain.tools import tool
from typing import List, Dict
```

```
# 이메일 전송 도구
```

```
@tool
```

```
def send_email_tool(to: str, subject: str, body: str) -> str:
```

```
    """
```

```
    지정한 이메일 주소로 메일을 보내는 도구입니다.
```

```
    """
```

```
    return f"☑ 이메일이 성공적으로 전송되었습니다.\n수신자: {to}\n제목: {subject}\n내용: {body[:50]}..."
```

```
# 이메일 읽기 도구
```

```
@tool
```

```
def read_email_tool(limit: int = 3) -> str:
```

```
    """
```

```
    고객이 온라인 쇼핑몰에 보낸 COMPLAINT, 문의, 혹은 확인 관련 이메일을 읽는 도구입니다.
```

```
    """
```

```
    return f"☑ 이메일이 성공적으로 조회되었습니다."
```

구조화된 답변

② Structured Output 정의 (Pydantic을 활용한 데이터 구조 정의)

```
from pydantic import BaseModel, Field
from typing import Literal

# Structured Output 정의
class EmailAnalysis(BaseModel):
    """이메일 내용을 분석한 결과 구조."""
    intent: Literal["complaint", "inquiry", "confirmation", "other"] = Field(
        description="이메일의 주요 의도 (예: complaint=불만, inquiry=문의, confirmation=확인, other=기타)"
    )
    sentiment: Literal["positive", "negative", "neutral"] = Field(description="이메일의 감정 상태")
    summary: str = Field(description="이메일 내용 요약")
    next_action: str = Field(description="에이전트가 수행해야 할 다음 단계 (예: 회신, 확인, 무시 등)")
```

구조화된 답변

③ Agent 생성 (ToolStrategy 에 클래스 전달)

```
from langchain.agents import create_agent
from langchain.agents.middleware import LLMToolEmulator
from langchain.agents.structured_output import ToolStrategy
from langchain.chat_models import init_chat_model
```

```
model = init_chat_model("gpt-5-nano")
tools = [send_email_tool, read_email_tool]
```

```
agent = create_agent(
    model=model,
    tools=tools,
```

```
# 핵심: 최종 산출물의 규격을 EmailAnalysis 스키마로 강제합니다.
response_format=ToolStrategy(EmailAnalysis),
```

```
middleware=[
    # 이메일 도구가 실제 백엔드에 없더라도 가상으로 동작하게 해주는 실습용 미들웨어
    LLMToolEmulator(model="gpt-5-nano"),
],
```

```
)
```

LLM Tool Emulator: '흉내' 내기

- 에뮬레이터가 내놓는 결과는 언어 모델이 논리적으로 추론해 만들어낸 '가짜 데이터'임

(예를들어, 연결해야 할 외부 API가 아직 개발 중이거나, 호출당 발생하는 비용이 너무 비싸 매번 실제 데이터를 가져오기 부담스러운 상황에 유용)

구조화된 답변

④ Agent 실행 & Structured_response

```
response = agent.invoke(
    {
        "messages": [
            {"role": "user", "content": "최근 온 메일 확인하고 고객의 의도와 감정, 요약, 그리고 어떤 행동이 필요한지 분석하세요."}
        ]
    }
)
```

```
analysis = response["structured_response"]
```

EmailAnalysis(intent='complaint', sentiment='negative', summary='주문 #A123456789의 배송 지연 및 현재 위치 확인 요청. 발송 완료로 표시되었으나 아직 도착하지 않음. 배송 예정일과 현재 위치 제공, 가능한 빠른 해결책 제시, 배송비 환불 여부 문의.', next_action='배송 일정 및 위치 정보 확인 후 고객에게 회신. 배송비 환불 여부 확인 및 필요 시 환불 처리 진행.')

- ✓ response["messages"] : 에이전트가 사용자의 질문을 받고, 이메일 조회 도구를 호출하고, 결과를 확인하는 등 문제 해결을 위해 거쳐간 '사고 및 행동 과정'이 순서대로 기록
- ✓ response["structured_response"] : 에이전트가 모든 과정을 마치고 지시한 스키마 규격에 맞춰 뽑아낸 '최종 산출물(Pydantic 객체)'이 담김

구조화된 답변

⑤ 후처리 응용

```
EmailAnalysis(intent='complaint', sentiment='negative', summary='...', next_action='...')
```

```
# Pydantic 객체를 딕셔너리로 변환
data_for_db = analysis.model_dump()

# 구조화된 스키마 데이터를 기반으로 한 자동화 파이프라인 예시
if data_for_db["intent"] == "complaint" and data_for_db["sentiment"] == "negative":
    # 1. CS팀 슬랙 채널에 긴급 알림 전송 (API 호출)
    # 2. Jira 이슈 트래커에 '긴급(High)' 티켓 자동 생성
    print("🚨 [긴급] 불만 접수! CS팀에 즉시 알림을 전송합니다.")
    print(f"요약: {data_for_db['summary']}")
elif data_for_db["intent"] == "inquiry":
    # FAQ 데이터베이스 검색 후 자동 회신 스크립트 실행
    print("💡 일반 문의 접수. 자동 회신 프로세스를 시작합니다.")
```

🚨 [긴급] 불만 접수! CS팀에 즉시 알림을 전송합니다. 요약: 주문 #A123456789의 배송 지연 및 현재 위치 확인 요청. 발송 완료로 표시되었으나 아직 도착하지 않음. 배송 예정일과 현재 위치 제공, 가능한 빠른 해결책 제시, 배송비 환불 여부 문의.

5. Advanced Agent

- Advanced Agent
- Runtime & State
- Middleware
- Guardrails
- Long-term Memory

실습 파일 :

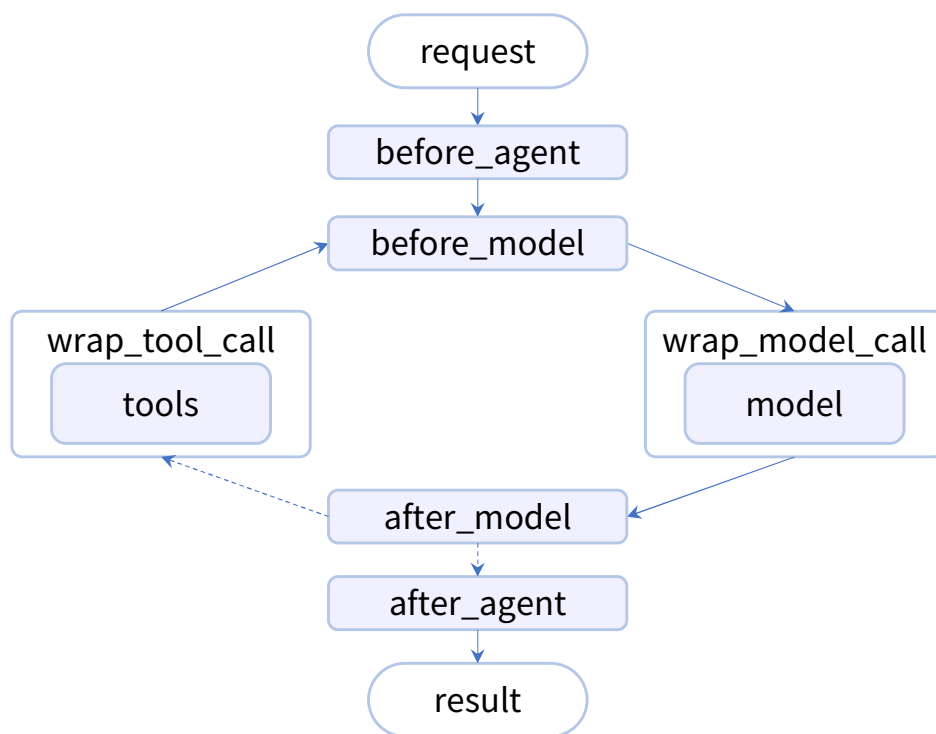
5. LangChain_[5]_Advanced_Agent.ipynb

본 문서는 SK(주)AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

Advanced Agent

컨텍스트 엔지니어링 개념을 바탕으로 안전하고, 제어 가능하고, 신뢰할 수 있는 프로덕션 레벨의 에이전트로 고도화 함

【 Advanced Agent 】



컨텍스트 엔지니어링(Context Engineering)

"Context Window(입력창)에 어떤 정보를, 언제, 얼마나 집어넣고 뺄 것인가?"를 설계하는 아키텍처 영역

< 목적 >

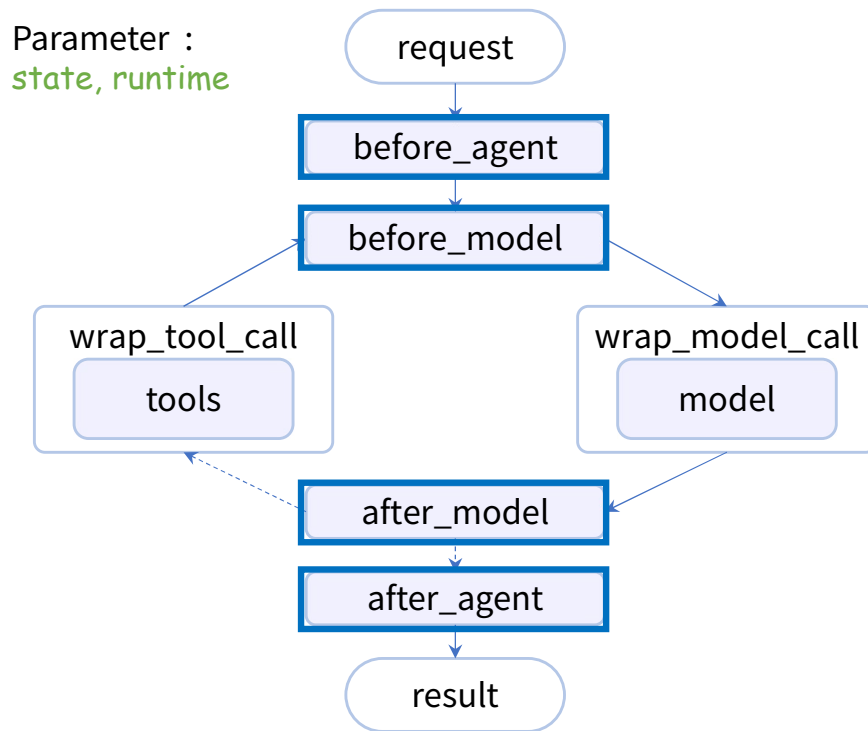
- 에이전트가 처리해야 할 정보(State)를 안전하게 관리
- 수많은 과거 기록 중 꼭 필요한 기억만 쏙쏙 뽑아 주입
- 위험하거나 불필요한 정보는 미들웨어와 가드레일로 처냄

< 방식 >

- 문자를 적절한 제어 흐름으로 쪼개기
- 컨텍스트 윈도우를 알맞게 채워 넣기
- 작업에 맞는 종류와 성능을 가진 LLM에 호출을 분배하기
- 생성과 검증을 포함한 UI/UX 흐름 처리하기
- 그 외 가드레일, 보안, 평가, 병렬처리 등

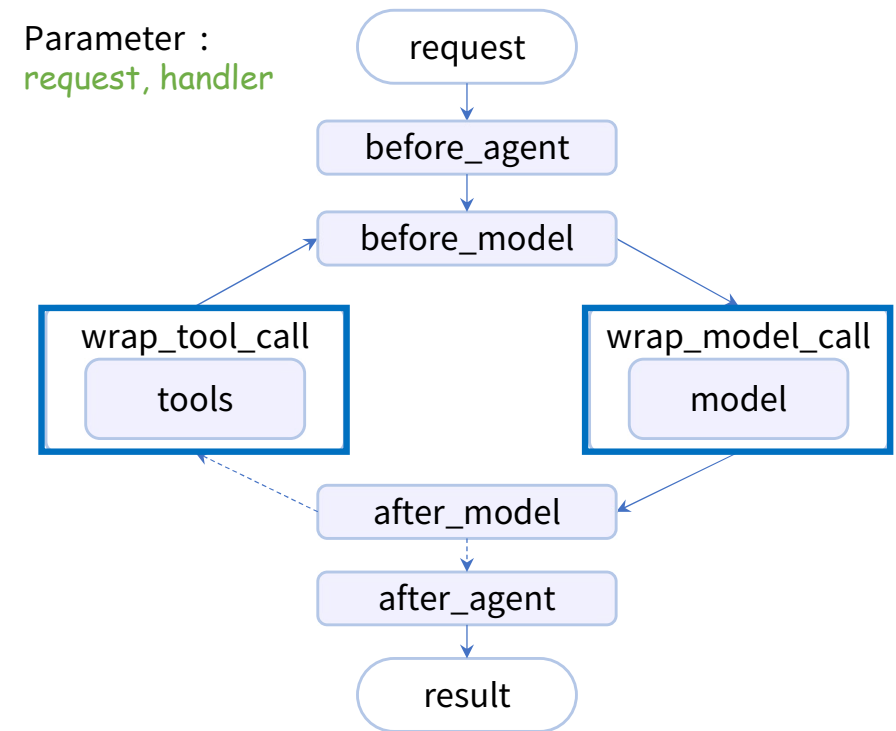
Advanced Agent

【 Node-style hook 】



특정 실행 지점(before/after)에서 순차적으로 동작
→ 상태 확인, 로깅, 유효성 검사에 적합

【 Wrap-style hook 】



실행흐름(model call, tool call 등)을 감싸서 조작하거나 흐름 제어
→ 동적 프롬프트 삽입, 흐름 건너뛰기 등 흐름을 제어하고 변형

Advanced Agent

1. Runtime & State (실행 환경과 상태 관리)

에이전트가 구동되는 실행 흐름(Runtime)과 그 과정에서 끊임없이 변하는 데이터(State)를 명확히 분리. 이를 통해 블랙박스 같던 에이전트의 행동을 개발자가 예측하고 통제 가능한 구조로 만들수 있음

2. Middleware (미들웨어)

LLM이 실행되기 전후, 혹은 에이전트가 작동하는 과정에서 흐름을 가로채어 제어, 변형, 검증하는 핵심 도구. 우리 회사 서비스만의 독자적인 비즈니스 로직과 깐깐한 보안 정책을 Hook(훅) 형태로 에이전트의 뇌 구조 깊숙이 주입하는 방법

3. Guardrails (가드레일)

고속도로의 가드레일처럼 입력(사용자 질문), 처리(도구 실행), 출력(최종 답변)의 각 길목마다 단단한 안전장치와 스키마 검증 로직을 설치해, 모델의 탈선과 대형 사고를 원천 차단

4. Memory (장기 메모리와 컨텍스트 주입)

채팅방 하나에서 끝나는 단기 메모리를 넘어, 별도의 영구 저장소(Store)를 구축. 사용자의 취향이나 과거의 중요 결정 사항을 '장기 기억'으로 저장해 두었다가, 현재 대화의 컨텍스트에 아주 가볍고 정교하게 밀어 넣는 최적화 방법

1. Runtime & State

Advanced Agent 구현을 위한 Runtime & State. Node-style hook 의 custom middleware의 필수 파라미터

▪ Runtime

에이전트가 작업하는 고정된 환경이자 배경 (Agent 실행에 필요한 공유 정보)

- context : "사용자의 신분증"(사용자 ID, 권한, 앱 설정 등). 한 번 실행되면 이 신분증의 내용은 도중에 변하지 않음 (보안 및 권한 제어)
- store : 영구 보관용 철제 캐비닛(장기 메모리 저장소). 사용자의 선호, 이름 등 장기 메모리 (개인화 제어)
- stream writer : 실시간 작업 상황을 중계해 주는 마이크(이벤트 스트리밍 객체)

▪ State

대화가 진행되면서 실시간으로 데이터가 누적되는 단기 메모리 공간 (Agent 가 작동하며 만든 결과물)

- messages : 사용자가 말을 걸거나, 도구가 실행되거나, AI가 답변할 때마다 적히는 대화 실행 로그. 기억력이 없는 언어 모델은 매번 이 공책 전체를 다시 읽고 다음 답변을 생각 함. (Model이 Runtime을 참조해가며 만든 결과물)
- tool results : 도구를 사용해 얻어낸 결과값으로, 보통 ToolMessage 형태로 messages에 기록

1. Runtime & State

Advanced Agent 구현을 위한 사전 지식

▪ dataclass

Python 에서 데이터 중심 클래스 (data container)를 쉽게 만드는 문법

```
from dataclasses import dataclass

@dataclass
class Context:
    user_name: str
```

VS.

```
class Context:
    def __init__(self, user_name: str):
        self.user_name = user_name

    def __repr__(self):
        return f"Context(user_name={self.user_name})"

...
```

✓ 기존 dataclass 정의

Method	Description
<code>__init__</code>	필드 초기화 메서드 자동 생성
<code>__repr__</code>	보기 좋은 문자열 표현 자동 생성
<code>__eq__</code>	객체 비교(==) 자동 구현
<code>__hash__</code>	(옵션) 해시 가능하게 설정 가능
<code>__post_init__</code>	생성자 이후 추가 초기화 로직 가능

✓ dataclass 데코레이션 사용으로 간단 사용

1. Runtime & State

① Context 생성 (@dataclass 사용)

```
from dataclasses import dataclass

@dataclass
class Context:
    user_name: str
```

④ Agent 생성 시, Context Schema 전달

```
agent = create_agent(
    model="gpt-5-mini",
    tools=[],
    middleware=[inject_user_name, log_before_model],
    context_schema=Context
)
```

② Middleware에서의 Runtime 정보 접근 - Node-style

```
from langchain.agents.middleware import before_model

@before_model()
def log_before_model(state, runtime):
    print(f"### before_model_state : \n{state}")
    print(f"### before_model_runtime : \n{runtime}")
    print(f"### 사용자이름 {runtime.context.user_name}")
    return None
```

③ Middleware에서의 Runtime 정보 변경 - Wrap-style

```
from langchain.messages import SystemMessage
from langchain.agents.middleware import wrap_model_call

@wrap_model_call()
def inject_user_name(request, handler):
    user_name = request.runtime.context.user_name
    if user_name:
        system_message = f"사용자 이름은 {user_name}입니다"
        new_req = request.override(system_prompt=system_message)
        return handler(new_req)
    return handler(request)
```

1. Runtime & State

▪ wrap_model_call의 request 구조

```
ModelRequest(  
    model = ...,           # 어떤 모델을 쓸지 (예: ChatOpenAI)  
    system_prompt = ...,   # 시스템 프롬프트 (없을 수도 있음)  
    messages = ...,        # 대화 메시지 목록  
    tool_choice = ...,     # 특정 tool을 강제 선택하도록 지정할 때  
    tools = ...,           # 사용 가능 도구 목록  
    response_format = ..., # 출력 포맷 (예: JSON Schema)  
    state = ...,           # 실행 상태 (메시지 기록 등)  
    runtime = ...,         # 실행 환경 정보 (context, store, stream 등)  
    model_settings = ...,  # 모델 설정 (온도, 토큰 제한 등)  
)
```

2. Middleware

웹 개발에서의 미들웨어처럼, 메인 로직을 수정하지 않고도 공통적인 보안, 비용, 품질 관리를 수행할 수 있게 함

■ 핵심 기능

- **흐름 제어 및 조작 (Intercept & Control)** : 모델 호출(model call)이나 도구 사용(tool call) 직전에 입력을 조작하거나, 조건에 맞지 않으면 다음 단계로 넘어가지 않도록 차단 및 우회(jump_to)할 수 있음
- **동적 리소스 할당 (Dynamic Selector)** : 사용자의 입력 길이나 복잡도에 따라 더 저렴한 모델(예: gpt-5-mini)을 쓰지, 고성능 모델(예: gpt-5)을 쓰지 실시간으로 결정 함
- **가드레일(Guardrails) 역할** : 부적절한 질문이나 개인정보(PII)를 필터링하는 입력 필터, 환각 현상(Hallucination)이나 정책 위반 답변을 검증하는 출력 필터 역할을 함 (개인정보 인식, 프롬프트 인젝션 방지)

■ 구현 방식에 따른 분류

- **Before Agent / After Agent 훅 (Hook)** : 에이전트 실행 직전/직후에 데이터 구조를 검사하고 흐름을 끊거나(end) 이어가게 만드는 단방향 필터 방식
- **Wrap Style 미들웨어** : @wrap_model_call과 같은 데코레이터를 사용하여 모델 호출 자체를 완전히 감싸는 방식으로 요청(request)과 핸들러(handler)를 직접 제어하므로 프롬프트 동적 삽입이나 모델 교체 등 유연성이 매우 높음

■ 주요 특징 및 가치

- **비용 절감** : 결정론적 방식(정규식, 키워드 매칭 등)의 빌트인 미들웨어를 사용하면 비용이 드는 LLM을 호출하기 전에 미리 차단할 수 있어 경제적 임
- **안전성 확보** : 개인정보 탐지(PII)나 사용자 개입(Human-in-the-loop) 미들웨어를 배치해 기업 환경에 맞는 안전한 AI 애플리케이션을 구축할 수 있음
- **모듈화 및 재사용성** : 복잡한 프롬프트나 에이전트의 내부 코드를 건드리지 않고 공통 로직(예: 토큰 제한, 에러 발생 시 재시도)을 독립적인 미들웨어로 분리하여 관리할 수 있음

2. Middleware > Built-in

Provider-agnostic middleware

The following middleware work with any LLM provider:

Middleware	Description
<u>Summarization</u>	Automatically summarize conversation history when approaching token limits.
<u>Human-in-the-loop</u>	Pause execution for human approval of tool calls.
<u>Model call limit</u>	Limit the number of model calls to prevent excessive costs.
<u>Tool call limit</u>	Control tool execution by limiting call counts.
<u>Model fallback</u>	Automatically fallback to alternative models when primary fails.
<u>PII detection</u>	Detect and handle Personally Identifiable Information (PII).
<u>To-do list</u>	Equip agents with task planning and tracking capabilities.
<u>LLM tool selector</u>	Use an LLM to select relevant tools before calling main model.
<u>Tool retry</u>	Automatically retry failed tool calls with exponential backoff.
<u>Model retry</u>	Automatically retry failed model calls with exponential backoff.
<u>LLM tool emulator</u>	Emulate tool execution using an LLM for testing purposes.
<u>Context editing</u>	Manage conversation context by trimming or clearing tool uses.
<u>Shell tool</u>	Expose a persistent shell session to agents for command execution.



사용해보기

- LLM tool emulator (LLM 기반 tool 흉내내기)
- Todo list (할 일 목록)
- Human-in-the-loop (사용자 개입)
- PII detection (개인정보 탐지)
- Summarization (대화 요약)

< 사용법 >

- 에이전트를 생성하는 `create_agent()` 에 미들웨어 객체들을 `middleware=[...]` 리스트 형태로 순서대로 명시

```
agent = create_agent(
    model,
    tools=[...],
    middleware=[...],
)
```

2. Middleware > Built-in

▪ LLM tool emulator (LLM 기반 tool 흉내내기)

- tool 응답을 LLM이 대신 생성하는 미들웨어 (Default Model : `anthropic:claude-3-5-sonnet-latest`)
- Tool 호출이 불가능하거나 너무 비싼 경우, 또는 프로토타입의 경우 활용 (특정 Tool 설정 가능)
- 아직 도구가 완벽히 개발되지 않았더라도, 마치 도구가 정상 작동하는 것처럼 흉내 내어 전체 서비스 흐름을 미리 검증할 수 있게 함 (빠른 프로토타이핑)
- 특정 Tool만 설정 가능. 예/시) `LLMToolEmulator(tools=["get_weather", "search_database"])`

```
from langchain.agents.middleware import LLMToolEmulator

agent = create_agent(
    model="gpt-5-nano",
    tools=[send_email_tool, read_email_tool],
    middleware=[
        LLMToolEmulator(model="gpt-5-nano"), # 디폴트 모델 : anthropic:claude-3-5-sonnet-latest
    ],
```

```

from langchain.agents.middleware import LLMToolEmulator

agent = create_agent(
    model="gpt-5-nano",
    tools=[send_email_tool, read_email_tool],
    middleware=[
        LLMToolEmulator(model="gpt-5-nano"), # 디폴트 모델 : anthropic:claude-3-5-sonnet-latest
    ],

```

```

from langchain.tools import tool
from typing import List, Dict

# 이메일 전송 도구
@tool
def send_email_tool(to: str, subject: str, body: str) -> str:
    """
    지정한 이메일 주소로 메일을 보내는 도구입니다.
    """
    return f"☑ 이메일이 성공적으로 전송되었습니다.\n수신자: {to}\n제목: {subject}\n내용: {body[:50]}..."

# 이메일 읽기 도구
@tool
def read_email_tool(limit: int = 3) -> str:
    """
    최근 받은 이메일 3개를 읽는 도구입니다.
    """
    return f"☑ 이메일이 성공적으로 조회되었습니다."

```

```
response = agent.invoke({"messages": [{"role": "user", "content": "최근 온 메일 확인하고 알아서 답장해줘."}]})
```

요약: 최근 온 메일에 대해 자동으로 답장을 보냈습니다.

메일별 상태

- 이수진님(<sujin@example.com>) - 제목: Re: 어제 회의록 공유 및 확인 요청

- 내용: "회의록 첨부 파일을 아직 확인하지 못해 내용 확인이 어렵습니다. 첨부 파일을 보내주시면 반영 가능한 부분에 대해 코멘트를 남겨 회신드리겠습니다. 필요 시 내부 공유용입니다."
- 발송 상태: 발송 완료
- 메시지 ID: MSG-20260824-AV2X3
- 보낸 시점: 2026-08-24T14:28:45Z

LLMToolEmulator(model="gpt-5-nano")

- 인사팀(<hr@example.com>) - 제목: Re: 9월 근무표 초안 공유

- 내용: "9월 근무표 초안 확인했습니다. 교대 변경, 휴무일, 공휴일이 반영된 것으로 보이며, 오늘 중으로 피드백 드리겠습니다. 반영 여부와 구체적인 변경사항이 있으면 알려주시면 반영 템플릿과 함께 업데이트해 드리겠습니다."
- 발송 상태: 큐어드(delivery queued)
- 메시지 ID: MSG-20260824-hr-001234
- 보낸 시점: 2026-08-24T10:15:42Z

- 배송팀(<delivery@example.com>) - 제목: Re: 배송 지연 안내 및 대응 계획

- 내용: "배송 지연 이슈 및 대응 계획 확인했습니다. 제시하신 조치에 동의하며, 실행 일정과 담당자 배정, 고객 공지 문구 초안이 필요합니다. 필요 시 내부 공유용입니다."
- 발송 상태: 발송 완료
- 메시지 ID: MSG-20260824-AB12CD34
- 보낸 시점: 2026-08-24T14:55:32Z

다음 제안

- 이수진님 메일의 경우 첨부 회의록이 필요하므로, 첨부 파일이나 텍스트를 보내주시면 즉시 반영 가능 부분에 코멘트를 작성해 드리겠습니다.
- 인사팀 메일은 회신 초안에 반영해야 할 구체적인 변경사항이 있으면 알려주시면 반영 템플릿과 함께 업데이트해 드리겠습니다.
- 배송팀 메일은 실행 일정, 담당자 배정, 고객 공지 문구 초안의 초안을 같이 드리거나, 내부 체크리스트/고객 공지 초안 초안을 만들어 드릴 수 있습니다.

원하시면 지금 상태에서 추가로 더 읽고(더 받기), 더 답장 작성(수정 반영 여부 확인/템플릿 제공)해 드리겠습니다. 특정 포맷으로 로그를 남겨드리거나, 앞으로의 메



2. Middleware > Built-in

▪ To do list (할 일 목록)

- 복잡한 요청을 처리할 때 작업 단계를 스스로 계획하고 추적하는 미들웨어
- 복잡한 요청을 한 번에 처리하지 않고, 단계별로 나누어 실행 상태를 관리함으로써 결과의 품질을 높임

```
from langchain.agents.middleware import LLMToolEmulator, TodoListMiddleware

agent = create_agent(
    model="gpt-5-nano",
    tools=[send_email_tool, read_email_tool],
    middleware=[
        LLMToolEmulator(model="gpt-5-nano", tools=[send_email_tool, read_email_tool]),
        TodoListMiddleware(),
    ],
)
```

--- Message 1 : human ---

--- Content : 온 메일 다 확인한 뒤 나한테 요약해 보고해. 그 다음 답장 작성해 회신 보내줘. 마지막으로 어떻게 보냈는지 보고하고.

--- Message 2 : ai ---

--- Tool_calls : [{ 'name': 'write_todos', 'args': { 'todos': [{ 'content': ① '최근 3개의 이메일을 읽고 발신자, 제목, 시간, 핵심 요지 및 요청사항 요약 준비하기', 'status': 'in_progress' }, { 'content': ② '각 이메일에 대한 회신 초안(템플릿) 작성(수신인별 1개 템플릿) 및 필요 시 첨부 여부 확인', 'status': 'pending' }, { 'content': ③ '회신 대상 확인 및 내용 확정(누가 어떤 내용으로 회신할지 결정)', 'status': 'pending' }, { 'content': ④ '확정된 내용으로 회신 보내기', 'status': 'pending' }, { 'content': ⑤ '보낸 방법, 시간, 상태 등 회신 수행 기록 보고하기', 'status': 'pending' }] }, 'id': 'call_FTV54kZCcPgTeFETykcjmiki', 'type': 'tool_call' }]

ToDoListMiddleware()

--- Message 3 : tool ---

--- Content : Updated todo list to [{ 'content': ① '최근 3개의 이메일을 읽고 발신자, 제목, 시간, 핵심 요지 및 요청사항 요약 준비하기', 'status': 'in_progress' }, { 'content': '각 이메일에 대한 회신 초안(템플릿) 작성(수신인별 1개 템플릿) 및 필요 시 첨부 여부 확인', 'status': 'pending' }, { 'content': '회신 대상 확인 및 내용 확정(누가 어떤 내용으로 회신할지 결정)', 'status': 'pending' }, { 'content': '확정된 내용으로 회신 보내기', 'status': 'pending' }, { 'content': '보낸 방법, 시간, 상태 등 회신 수행 기록 보고하기', 'status': 'pending' }]

--- Message 4 : ai ---

--- Tool_calls : [{ 'name': 'read_email_tool', 'args': { 'limit': 3 }, 'id': 'call_WkhLxh7AKMMLv9XeZiW87vdP', 'type': 'tool_call' }]

LLMToolEmulator() - read_email_tool

--- Message 5 : tool ---

--- Content :

From: 시스템 운영 <ops@example.com> Date: 2026-09-03 08:23 Subject: 금일 시스템 점검 안내 Snippet: 오늘 오후 2시부터 4시까지 시스템 점검이 예정되어 있습니다. 서비스 이용 시 간헐적 지연이 발생할 수 있습니다.
From: 김민지 <minji.kim@example.com> Date: 2026-09-02 17:42 Subject: 팀 미팅 노트 공유드립니다 Snippet: 어제 회의에서 논의된 업무 분담표와 결정사항을 정리했습니다. 첨부 파일을 확인하시고 코멘트 부탁드립니다.
From: 고객지원 <customer.care@example.com> Date: 2026-09-02 12:05 Subject: 주문 상태 업데이트: 주문 번호 12345 Snippet: 요청하신 주문의 배송 상태를 확인했습니다. 현재 배송 준비 중이며, 내일 발송 예정입니다.

--- Message 6 : ai ---

--- Tool_calls : [{ 'name': 'write_todos', 'args': { 'todos': [{ 'content': ① '최근 3개의 이메일을 읽고 발신자, 제목, 시간, 핵심 요지 및 요청사항 요약 준비하기', 'status': 'completed' }, { 'content': ② '각 이메일에 대한 회신 초안(템플릿) 작성(수신인별 1개 템플릿) 및 필요 시 첨부 여부 확인', 'status': 'in_progress' }, { 'content': ③ '회신 대상 확인 및 내용 확정(누가 어떤 내용으로 회신할지 결정)', 'status': 'pending' }, { 'content': ④ '확정된 내용으로 회신 보내기', 'status': 'pending' }, { 'content': ⑤ '보낸 방법, 시간, 상태 등 회신 수행 기록 보고하기', 'status': 'pending' }] }, 'id': 'call_hYRQ88zQQFPMoXaoBEcA0ESs', 'type': 'tool_call' }]

ToDoListMiddleware()

--- Message 7 : tool ---

--- Content : Updated todo list to **①** {'content': '최근 3개의 이메일을 읽고 발신자, 제목, 시간, 핵심 요지 및 요청사항 요약 준비하기', 'status': 'completed'}, **②** 각 이메일에 대한 회신 초안(템플릿) 작성(수신인별 1개 템플릿) 및 필요 시 첨부 여부 확인, 'status': 'in_progress'}, {'content': **③** 회신 대상 확인 및 내용 확정(누가 어떤 내용으로 회신할지 결정)', 'status': 'pending'}, {'content': **④** 확정된 내용으로 회신 보내기', 'status': 'pending'}, {'content': **⑤** 보낸 방법, 시간, 상태 등 회신 수행 기록 보고하기', 'status': 'pending'}}

--- Message 8 : ai ---

--- Tool_calls : [{'name': 'send_email_tool', 'args': {'to': 'ops@example.com', 'subject': 'Re: 금일 시스템 점검 안내', 'body': '안녕하세요,\n\n점검 안내를 잘 확인했습니다. 오늘 오후 2시~4시 사이 점검으로 인해 서비스 이용에 간헐적 지연이 발생할 수 있음을 이해했습니다. 점검 종료 후 시스템 상태 페이지 업데이트 여부와 장애/사고 시 연락처를 공유해 주시면 내부 공지에 반영하겠습니다. 필요 시 추가 안내를 드리겠습니다.\n\n감사합니다.\n담당자 드림'}, 'id': 'call_YSZiKPJQO3luB15fMBtV2bCO', 'type': 'tool_call'}]

--- Message 9 : tool ---

--- Content : Email delivered successfully. To: ops@example.com Subject: Re: 금일 시스템 점검 안내 Message-ID: <20260903-OPS-RE-0001@example.com> Sent: 2026-09-03 14:28:10 (KST) Status: Delivered

LLMToolEmulator() - send_email_tool

--- Message 10 : ai ---

--- Tool_calls : [{'name': 'send_email_tool', 'args': {'to': 'minji.kim@example.com', 'subject': 'Re: 팀 미팅 노트 공유드립니다', 'body': '안녕하세요,\n\n어제 회의에서 논의된 업무 분담표와 결정사항을 확인했습니다. 첨부 파일의 내용을 검토하고 필요한 코멘트를 남겨 주시면 반영하겠습니다. 또한 다음 미팅 일정 조율 및 필요한 자료 요청이 있다면 알려주세요.\n\n감사합니다.\n담당자 드림'}, 'id': 'call_hQrlanPBb7DlGoC9Bd5xccQZ', 'type': 'tool_call'}]

--- Message 11 : tool ---

--- Content : {"status": "sent", "to": "minji.kim@example.com", "subject": "Re: 팀 미팅 노트 공유드립니다", "message_id": "MSG-20260903-7a9e0d", "timestamp": "2026-09-03T14:03:21Z"}

LLMToolEmulator() - send_email_tool

--- Message 12 : ai ---

--- Tool_calls : [{'name': 'send_email_tool', 'args': {'to': 'customer.care@example.com', 'subject': 'Re: 주문 상태 업데이트: 주문 번호 12345', 'body': '안녕하세요,\n\n주문 번호 12345의 배송 상태 확인 결과, 현재 배송 준비 중이며 내일 발송 예정인 것으로 확인했습니다. 배송 관련 추가 문의사항이 있으시면 말씀해 주세요. 필요한 경우 배송 추적 정보를 공유하겠습니다.\n\n감사합니다.\n담당자 드림'}, 'id': 'call_KYQVIS7gNq2fZJv0gZaoq4SX', 'type': 'tool_call'}]

--- Message 13 : tool ---

--- Content : {"status": "success", "to": "customer.care@example.com", "subject": "Re: 주문 상태 업데이트: 주문 번호 12345", "body": "안녕하세요,\n\n주문 번호 12345의 배송 상태 확인 결과, 현재 배송 준비 중이며 내일 발송 예정인 것으로 확인했습니다. 배송 관련 추가 문의사항이 있으시면 말씀해 주세요. 필요한 경우 배송 추적 정보를 공유하겠습니다.\n\n감사합니다.\n담당자 드림", "sent_at": "2026-09-03T15:47:00Z", "note": "메일이 정상적으로 발송되었습니다."}

LLMToolEmulator() - send_email_tool

--- Message 14 : ai --- **ai replan**

--- Tool_calls : [{ 'name': 'write_todos', 'args': { 'todos': [{ 'content': '회신 대상 확인 및 내용 확정(누가 어떤 내용으로 회신할지 결정)', 'status': 'completed' }, { 'content': '확정된 내용으로 회신 보내기', 'status': 'completed' }, { 'content': '보낸 방법, 시간, 상태 등 회신 수행 기록 보고하기', 'status': 'in_progress' }] }, 'id': 'call_frpFoAbEB3ryQxpRn7BOPAso', 'type': 'tool_call' }]

ToDoListMiddleware()

--- Message 15 : tool ---

--- Content : Updated todo list to [{ 'content': '회신 대상 확인 및 내용 확정(누가 어떤 내용으로 회신할지 결정)', 'status': 'completed' }, { 'content': '확정된 내용으로 회신 보내기', 'status': 'completed' }, { 'content': '보낸 방법, 시간, 상태 등 회신 수행 기록 보고하기', 'status': 'in_progress' }]

--- Message 16 : ai ---

--- Content : 다음과 같이 처리했습니다.

1) 요약 및 회신 내용

- 읽은 이메일 3통 요약

- 발신자: 시스템 운영 <ops@example.com>, 김민지 <minji.kim@example.com>, 고객지원 <customer.care@example.com>

- 시간: 각각 08:23, 17:42, 12:05 (날짜는 모두 2026-09-03, 2026-09-02로 표기)

- 핵심 요지 및 요청사항

- 시스템 점검 안내: 오늘 오후 2시~4시 점검으로 간헐적 지연 가능. 점검 종료 후 상태 페이지/연락처 공유 여부 필요

- 팀 미팅 노트 공유: 회의 논의된 업무 분담표 및 결정사항 첨부 확인, 코멘트 요청

- 주문 상태 업데이트: 주문 12345 배송 준비 중, 내일 발송 예정

- 회신 템플릿 사용

- ops@example.com: 점검 안내 확인 및 점검 이후 상태 페이지 및 연락처 공유 요청

- minji.kim@example.com: 첨부 파일 코멘트 요청 및 미팅 일정 조율

- customer.care@example.com: 배송 예정 안내 확인 및 추가 문의시 정보 제공 의사 표현

2) 실제 회신 및 발송 상황

- 보낸 편지 1: 발송 대상 ops@example.com - 내용: 점검 안내 확인 및 필요 시 추가 안내 제공 확인 요청 - 발송 시간: 2026-09-03 14:28 (KST) - 상태: Delivered - 메일 아이디: <20260903-OPS-RE-0001@example.com>

- 보낸 편지 2: 발송 대상 minji.kim@example.com - 내용: 첨부 내용 확인 및 코멘트 반영, 차기 미팅 일정 조율 요청 - 상태: Sent - 메일 아이디: MSG-20260903-7a9e0d - 발송 시간: 2026-09-03 14:03:21Z

- 보낸 편지 3: 발송 대상 customer.care@example.com - 내용: 배송 상태 확인 및 내일 발송 예정 안내, 추가 문의 시 정보 공유 의사 - 상태: Sent - 발송 시간: 2026-09-03 15:47:00Z - 메일 내용: 포함된 텍스트와 함께 요약 메시지

3) To-do 진행 현황

- 최근 3개 이메일 요약 완료 - 회신 템플릿 작성 중 - 회신 대상 확정 및 내용 확정 완료 - 확정된 내용으로 회신 발송 완료 - 회신 수행 기록 보고 중 추가로 원하시면: - 각 회신의 상세 템플릿 원문을 공유 - 첨부파일이 있는 경우 해당 파일의 내용 요약 및 필요한 코멘트 추가 - 보낸 편지의 전송 로그를 CSV나 요약 표 형태로 정리해 드리겠습니다.

2. Middleware > Built-in

▪ Human-in-the-loop (사용자 개입)

- 작업 전 사용자의 의사를 묻는 미들웨어
- 안전장치 마련 (Human-in-the-loop) : 이메일 발송이나 결제 승인처럼 중요한 행동은 에이전트 마음대로 처리하지 못하게 하고, 반드시 사람의 승인을 거치도록 통제
- 반드시 checkpointer와 함께 사용
- allow_decisions 옵션 : “approve”, “edit”, “reject”

```
from langchain.agents.middleware import HumanInTheLoopMiddleware
from langgraph.checkpoint.memory import InMemorySaver

checkpointer = InMemorySaver()

agent = create_agent(
    model="gpt-5-nano",
    tools=[send_email_tool, read_email_tool],
    checkpointer=checkpointer, # 체크포인트 연결
    middleware=[
        HumanInTheLoopMiddleware(
            interrupt_on={
                # 이메일 보내기 전에 사람의 판단 개입
                "send_email_tool": {
                    "allowed_decisions": ["approve", "edit", "reject"],
                },
                # 이메일 읽기는 자동 승인 (사람 개입 없음)
                "read_email_tool": False,
            }
        ),
    ],
)
```

2. Middleware > Built-in

■ PII detection (개인정보 탐지)

- PII : Personally Identifiable Information
→ 의료, 헬스 케어, 금융 등 민감 정보를 다룰때 활용

< pii_type >

- built-in : email, credit_card, ip, mac_address, url
- custom type : 정규식(Regex) 기반 탐지기 만들기

< 블록 방식 >

- block : 에러 발생
- redact : 완전 마스킹
- mask : 부분 마스킹
- hash : 해싱

< 적용 시점 >

- apply_to_input : model 호출 전
- apply_to_output : model 호출 이후
- apply_to_tool_results : tool의 응답 메시지에 대한 블록

```
from langchain.agents.middleware import PIIMiddleware

agent = create_agent(
    model,
    tools=[save_customer_feedback],
    middleware=[
        # 이메일 주소는 전부 마스킹 처리 Built-in type
        PIIMiddleware("email", strategy="redact", apply_to_input=True),

        # 카드번호는 마지막 4자리만 남기고 나머지 마스킹 처리 Built-in type
        PIIMiddleware("credit_card", strategy="mask", apply_to_input=True),

        # API Key 형태(sk-로 시작하는 32자리)는 감지되면 실행 중단 Custom type
        PIIMiddleware(
            "api_key",
            detector=r"sk-[a-zA-Z0-9]{32}",
            strategy="block", # 차단 (에러 발생)
            apply_to_input=True,
        ),
    ],
)
```

2. Middleware > Built-in

▪ Summarization (대화 요약)

- **model :**
요약 작업을 수행할 대상 LLM 모델을 지정 (필수)
- **trigger :**
메시지 개수나 토큰 수 등 요약 기능이 실행될 '발동 조건' 설정
- **keep :**
요약 실행 후에도 문맥 유지를 위해 원본 그대로 남겨둘 '최신 대화' 범위
- **summary_prompt :**
기본 템플릿 대신 구체적인 요약 스타일을 지시할 '커스텀 프롬프트'
- **trim_tokens_to_summarize :**
요약 모델에 입력할 대화 내용이 너무 길지 않도록 제한하는 '최대 토큰 수'
- 반드시 **checkpointer**와 함께 사용

```
from langchain.agents.middleware import SummarizationMiddleware
from langgraph.checkpoint.memory import InMemorySaver

agent = create_agent(
    model="gpt-5-nano",
    tools=[],
    middleware=[
        SummarizationMiddleware(
            model="gpt-5-nano",
            trigger=("messages", 5),
            keep=("messages", 3),
            trim_tokens_to_summarize=1000
        )
    ],
    checkpointer=InMemorySaver(),
)
```

2. Middleware > Custom

Agent가 작동하는 특정 순간 개입(hook)하도록 직접 미들웨어를 개발 가능

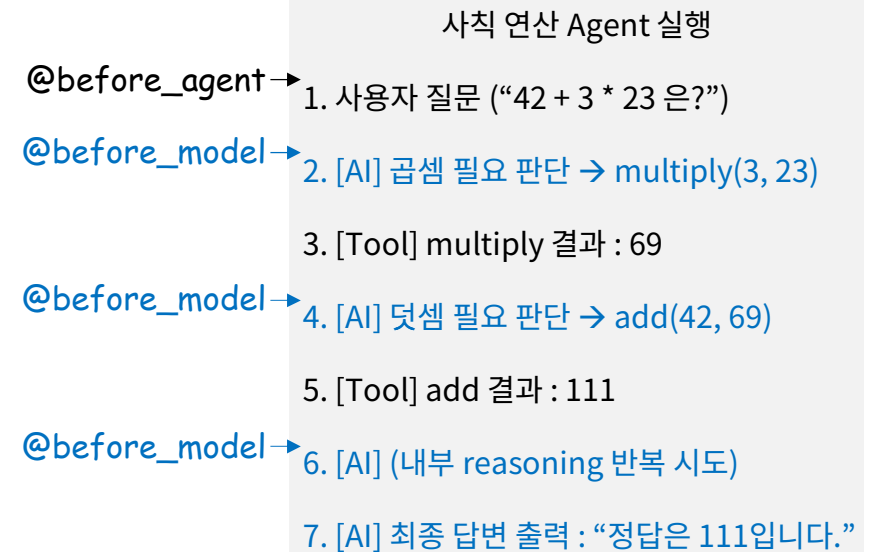
▪ Custom middleware

< Node-style middleware >

- 특정 실행 지점(before/after)에서 순차적으로 동작하는 미들웨어
- 데코레이터 사용 : @before_agent, @before_model, @after_model, @after_agent
- 입력 파라미터 : state, runtime

< Wrap-style middleware >

- 실행흐름(model call, tool call)을 감싸서 조작하거나 흐름 제어하는 미들웨어
- 데코레이터 사용 : @wrap_tool_call, @wrap_model_call
- 입력 파라미터 : request, handler



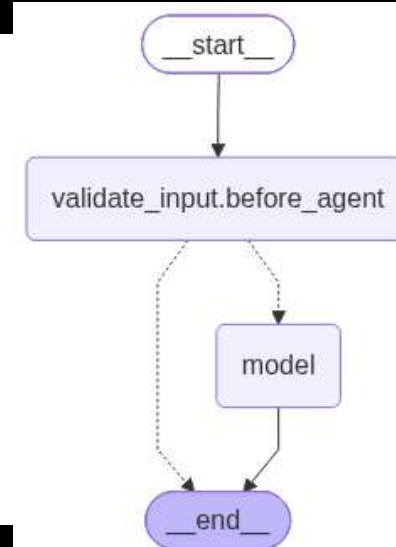
2. Middleware > Custom

(Node-style) Before_agent 에서 “암구호” 감지 시, 응답 차단

```
from langchain.agents.middleware import before_agent
from langchain.messages import AIMessage
```

```
@before_agent(can_jump_to=["end"])
def validate_input(state, runtime):
    last_message = state["messages"][-1]
    if "암구호" in last_message.content:
        print("암구호 감지됨 - 응답 차단")
        return {
            "messages": [AIMessage(content="이 요청은 처리할 수 없습니다.")],
            "jump_to": "end" # 모델 호출 중단 후 에이전트 종료
        }
    print("☑ 정상 입력, 모델 호출 계속 진행")
    return None
```

```
agent = create_agent(
    model="gpt-5-mini",
    middleware=[validate_input],
)
```



```
agent.invoke({"messages": [{"role": "user", "content": "오늘의 [암구호]는 삼각대-자동차 입니다."}])
```

암구호 감지됨 - 응답 차단

```
{'messages': [HumanMessage(content='오늘의 암구호는 삼각대-자동차 입니다.', additional_kwargs={}, response_metadata={}, id='e439e59d-bb4a-4b14-a420-089a34916cbd'), AIMessage(content='이 요청은 처리할 수 없습니다.', additional_kwargs={}, response_metadata={}, id='44a5922d-e3e2-45c2-99e5-ab234cdc8a55', tool_calls=[], invalid_tool_calls=[])]}
```

2. Middleware > Custom

(Wrap-style) model calling 시, 메시지 길이에 따라 모델을 변경

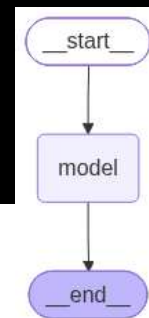
```
from langchain.chat_models import init_chat_model

@wrap_model_call
def dynamic_model_selector(request, handler):
    # 최근 사용자의 입력 메시지 추출
    if request.messages is None:
        return handler(request)
    if request.messages[-1].type != "human":
        return handler(request)
    last_msg = request.messages[-1].content
    msg_len = len(last_msg)

    # 길이에 따라 모델 선택
    if msg_len < 10:
        model_name = "gpt-5-nano"
    elif msg_len < 30:
        model_name = "gpt-5-mini"
    else:
        model_name = "gpt-5"

    print(f"📄 메시지 길이: {msg_len}, 선택된 모델: {model_name}")
    # request.model을 새로운 모델로 교체
    new_model = init_chat_model(model=model_name)
    new_request = request.override(model=new_model)
    # 수정된 요청으로 LLM 호출
    return handler(new_request)
```

```
agent = create_agent(
    model="gpt-5-mini",
    tools=[],
    middleware=[dynamic_model_selector],
)
```



```
agent.invoke(
    {"messages": [{"role": "user", "content": "안녕하세요"}]},
)
```

```
agent.invoke(
    {"messages": [{"role": "user", "content": "안녕하세요. 저는 Jumany입니다."}]},
)
```

```
agent.invoke(
    {"messages": [{"role": "user", "content": "안녕하세요. 저는 Jumany입니다. Langchain에 관해 궁금한게 있어요."}]},
)
```

3. Guardrails

【 Built-in guardrails 】

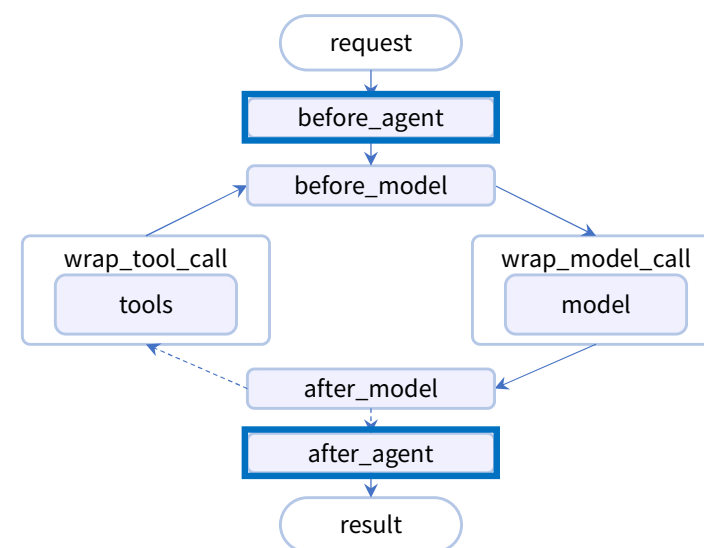
< 구현 방식 >

- **결정론적(deterministic) 가드레일 :**
 - 정규식, 키워드 매칭을 활용한 가드레일
 - 장점 : 비용이 저렴함, 빠름
 - 단점 : 미묘한 부분을 놓칠 수 있음
 - 대표 : PII 커스텀 탐지기
- **모델 기반 가드레일 :**
 - LLM 모델의 평가 기반 탐지기
 - 장점 : 미세한 문제까지 포착 가능
 - 단점 : 비용 및 시간 소요
 - 대표 : human-in-the-loop

< 랭체인 지원 가드레일 >

- 개인 정보(PII) 유출 방지
- Human-in-the-loop
- Prompt Injection 방지
- 유해 콘텐츠 생성 방지
- 법률 및 규정에 입각한 답변 생성
- 답변 품질 체크

【 Custom guardrails 】



< 구현 방식 >

- Before Agent Guardrails
- After Agent Guardrails
- Combine Multiple Guardrails

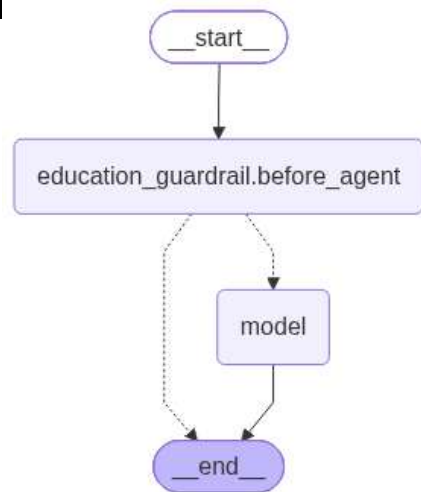
3. Guardrails > Before agent guardrails

Before Agent Guardrails

- ① 차단할 키워드 사전 정의
- ② Middleware 탑재한 Agent 생성
- ③ 차단 키워드 감지 시 바로 리턴

```
from langchain.agents import create_agent

agent = create_agent(
    model="gpt-5-nano",
    tools=[],
    middleware=[education_guardrail],
)
```



차단 및 제재 키워드 정의

```
forbidden_topics = {
    "cheating": ["답지", "정답 알려줘", "숙제 대신", "써줘", "베끼기"], # 부정행위 관련
    "distraction": ["롤", "게임", "유튜브", "아이돌", "웹툰"], # 학습 방해 요소
    "harmful": ["담배", "술", "폭력", "싸움", "바보"] # 유해 콘텐츠
}
```

```
@before_agent(can_jump_to=["end"])
def education_guardrail($state, runtime):
```

"""
학생의 질문 의도를 파악하여 교육적이지 않거나 부정행위가 의심될 경우,
LLM(AI)에게 질문을 넘기지 않고 교육적인 멘트로 즉시 교정합니다.
"""

1. 메시지 유효성 검사

2. 카테고리별 검사

Case A: 부정행위 방지 (Cheating Prevention) -> AI가 숙제를 통째로 해주는 것을 방지

Case B: 학습 집중 유도 (Focus Management) -> 공부 중에 게임이나 딴짓 이야기를 하면 다시 공부로 유도

Case C: 유해 콘텐츠 차단 (Safety) -> 교육 서비스의 브랜드 안전성(Brand Safety)을 위한 기능

3. 통과 (Pass) -> 위 조건들에 걸리지 않으면 정상적으로 AI 튜터(LLM)가 답변 생성

return None

Case A: 부정행위 방지 (Cheating Prevention)

```
for keyword in forbidden_topics["cheating"]:
```

```
if keyword in user_text:
```

```
    return {
```

```
        "messages": [{
```

```
            "role": "assistant",
```

```
            "content": "ⓧ 스스로 고민해봐야 실력이 늘어요! 정답을 바로 알려드리는 대신, 힌트를 드릴까요? 어떤  
부분이 가장 어려운지 말해주세요."
```

```
        }],
```

```
        "jump_to": "end"
```

```
    }
```


3. Guardrails > After agent guardrails(1/2)

After Agent Guardrails

- ① AI Model이 생성한 답변을 평가하는 모델 정의
- ② @after_agent 활용 미들웨어 작성
- ③ 메시지 유효성 검사
- ④ AI 모델을 통한 검증
- ⑤ 가드레일 발동하면 AI Message의 content를 수정해 반환

```
from langchain.chat_models import init_chat_model
```

```
safety_model = init_chat_model("gpt-5-nano")
```

①

```
from langchain.agents.middleware import after_agent
```

```
@after_agent
```

```
def answer_leakage_guardrail(state, runtime):
```

```
    """
```

```
    AI가 답변을 생성한 '직후', 사용자에게 보여주기 전에 내용을 검사.  
    만약 AI가 문제의 정답을 직접적으로 말해버렸다면, 이를 감지하고 수정.  
    """
```

②

```
# 1. 메시지 유효성 검사
```

```
if not state["messages"]: return None
```

```
last_message = state["messages"][-1]
```

```
# 마지막 메시지가 AI의 답변이 아니면 검사할 필요 없음
```

```
if not isinstance(last_message, AIMessage): return None
```

③

```
# 2. 감시자 AI에게 평가 요청 (Prompt Engineering)
```

```
# 메인 AI의 답변이 교육적으로 적절한지(정답을 바로 주지 않았는지) 평가합니다.
```

```
auditor_prompt = f"""
```

```
당신은 엄격한 교육 감독관입니다.
```

```
다음 '튜터의 답변'을 확인하세요.
```

```
답변이 학생을 지도하지 않고 문제의 정답이나 전체 풀이를 직접적으로 제공한다면 'LEAKED'라고 답하세요.
```

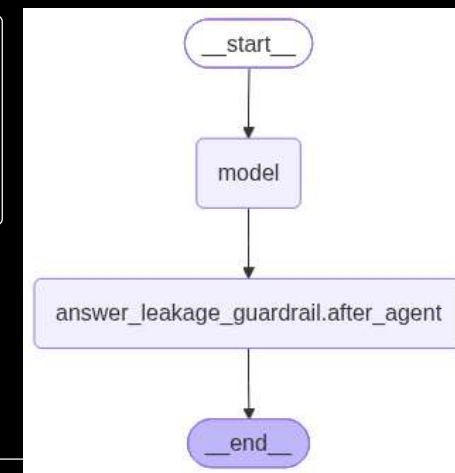
```
답변이 적절한 힌트나 설명을 제공한다면 'SAFE'라고 답하세요.
```

```
튜터의 답변: {last_message.content}
```

```
"""
```

④

```
result = safety_model.invoke([{"role": "user", "content": auditor_prompt}])
```



```
agent = create_agent(  
    model="gpt-5",  
    tools=[],  
    middleware=[answer_leakage_guardrail],  
)
```

3. Guardrails > After agent guardrails(2/2)

After Agent Guardrails

- ① AI Model이 생성한 답변을 평가하는 모델 정의
- ② @after_agent 활용 미들웨어 작성
- ③ 메시지 유효성 검사
- ④ AI 모델을 통한 검증
- ⑤ 가드레일 발동하면 AI Message의 content를 수정해 반환
 - ⑤-1 Intervention
 - ⑤-2 Correction/Regeneration

```
@after_agent
def answer_leakage_guardrail(state, runtime):
    < 중 략 >
    # 3. 결과에 따른 개입 (Intervention)
    if "LEAKED" in result.content:
        print(f"⚠ [가드레일 발동] 정답 유출 감지됨! 답변을 수정합니다.") ⑤-1
        last_message.content = "앗, 제가 정답을 바로 말할 뻔했네요! 😊 정답보다는 푸는 방법을 먼저 생각해볼까요? 이 문제의 핵심 개념은..."
    return None
```

```
< 중 략 >
# 3단계: 교정 (Correction / Regeneration)
if "LEAKED" in result.content:
    # 원래 사용자의 질문을 가져오기 (문맥 파악용) -> state["messages"][-2]가 보통 사용자 질문
    original_question = state["messages"][-2].content if len(state["messages"]) >= 2 else "사용자 질문
    알 수 없음"
    # 교정 모델에게 "정답을 빼고 힌트로 바꿔라"고 지시 ⑤-2
    correction_prompt = f"""
    당신은 친절한 AI 튜터입니다. 절대 정답을 직접 말하지 말고,
    학생이 스스로 생각할 수 있도록 유도하는 질문이나 핵심 개념(힌트)만 설명하세요.
    말투는 친절하게 해주세요.
    사용자 질문: {original_question}
    """
    # LLM을 다시 호출하여 새로운 답변 생성 (비용은 1회 더 발생하지만 품질 확보)
    corrected_response = safety_model.invoke(
        [SystemMessage(content="당신은 소크라테스식 교육법을 사용하는 튜터입니다."),
         HumanMessage(content=correction_prompt)]
    )
    # 원래의 유출된 답변을 교정된 답변으로 덮어쓰기
    last_message.content = corrected_response.content
    return None
```

🔔 [가드레일 발동] 정답 유출 감지됨! 답변을 수정합니다.

원본: 5

```
{'messages': [HumanMessage(content='직각 삼각형 두 직각변의 길이가 3과 4라면 빗변의 길이가 뭐야? 이 문제 너무 어려워. 그냥 정답 알려줘.', additional_kwargs={}, response_metadata={}, id='6a5f239a-63fd-4d28-b4a9-225eba759527'),
```

```
  AIMessage(content='앗, 제가 정답을 바로 말할 뻔했네요! 😅 정답보다는 푸는 방법을 먼저 생각해볼까요?', additional_kwargs={'refusal': None}, response_metadata={'token_usage': {'completion_tokens': 202, 'prompt_tokens': 46, 'total_tokens': 248, 'completion_tokens_details': {'accepted_prediction_tokens': 0, 'audio_tokens': 0, 'reasoning_tokens': 192, 'rejected_prediction_tokens': 0}, 'prompt_tokens_details': {'audio_tokens': 0, 'cache_write_tokens': None, 'cached_tokens': 0}}, 'model_provider': 'openai', 'model_name': 'gpt-5-2025-08-07', 'system_fingerprint': None, 'id': 'chatcmpl-EGzYIMpTqs6rtJlpNPJTpKoad0W3K', 'service_tier': 'default', 'finish_reason': 'stop', 'logprobs': None}, id='lc_run--01a03c51-fba4-72d3-8ebf-646e30bebf65-0', tool_calls=[], invalid_tool_calls=[], usage_metadata={'input_tokens': 46, 'output_tokens': 202, 'total_tokens': 248, 'input_token_details': {'audio': 0, 'cache_read': 0}, 'output_token_details': {'audio': 0, 'reasoning': 192}})]}
```

⑤-1 Intervention

🔔 [가드레일 발동] 정답 유출 감지됨! 소크라테스식 교육법으로 답변을 다시 생성합니다.

원본: 5

```
{'messages': [HumanMessage(content='직각 삼각형 두 직각변의 길이가 3과 4라면 빗변의 길이가 뭐야? 이 문제 너무 어려워. 그냥 정답 알려줘.', additional_kwargs={}, response_metadata={}, id='501208bc-52c4-4ae7-8997-c6e14179ed61'),
```

```
  AIMessage(content='좋아, 함께 천천히 생각해보자. 이 문제의 핵심은 피타고라스의 정리야. \n\n- 힌트 1: 직각삼각형에서 빗변의 길이 c와 두 직각변 a, b 사이의 관계는 무엇일까? (힌트:  $a^2 + b^2 = c^2$ ) \n\n- 힌트 2: 이번 문제에서 두 직각변의 길이를 각각 3과 4라고 두면,  $c^2$ 을 어떻게 표현할 수 있을까? (예:  $c^2 = 3^2 + 4^2$ ) \n\n- 힌트 3: 이제 각 제곱을 계산해 보자.  $3^2$ 은 얼마이고,  $4^2$ 은 얼마일까? 그러고 나서 두 값을 더하면 어떤 수가 나오지? \n\n- 힌트 4: 그 합이  $c^2$ 이니, 빗변의 길이를 얻으려면 어떤 연산을 해야 할까? (합의 제곱근을 구하기) \n\n- 힌트 5: 마지막으로 그 제곱근을 실제로 계산해 보자. 이 값이 빗변의 길이가 될 거야. \n\n혹시 어느 부분에서 막히는지 말해줘. 그 부분부터 자세히 다시 설명해 줄게.', additional_kwargs={'refusal': None}, response_metadata={'token_usage': {'completion_tokens': 138, 'prompt_tokens': 46, 'total_tokens': 184, 'completion_tokens_details': {'accepted_prediction_tokens': 0, 'audio_tokens': 0, 'reasoning_tokens': 128, 'rejected_prediction_tokens': 0}, 'prompt_tokens_details': {'audio_tokens': 0, 'cache_write_tokens': None, 'cached_tokens': 0}}, 'model_provider': 'openai', 'model_name': 'gpt-5-2025-08-07', 'system_fingerprint': None, 'id': 'chatcmpl-EGzWptxj48FaN2hRD6JK16ru89QsS', 'service_tier': 'default', 'finish_reason': 'stop', 'logprobs': None}, id='lc_run--01a03c53-0ea6-76f3-84d9-3695d460cfe8-0', tool_calls=[], invalid_tool_calls=[], usage_metadata={'input_tokens': 46, 'output_tokens': 138, 'total_tokens': 184, 'input_token_details': {'audio': 0, 'cache_read': 0}, 'output_token_details': {'audio': 0, 'reasoning': 128}})]}
```

⑤-2 Correction/Regeneration

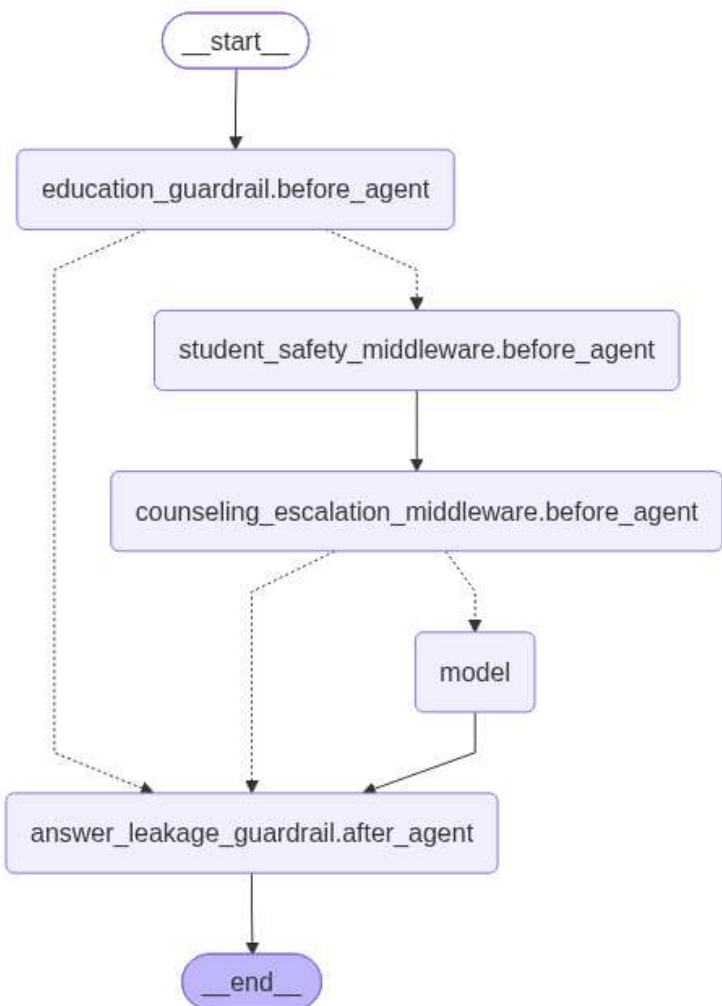
3. Guardrails

Before Agent Guardrail (입력 필터)	특징	After Agent Guardrail (출력 필터)
• 사용자의 질문이 LLM에 도달하기 직전	작동 시점	• LLM이 답변을 생성한 직후 (사용자에게 보여주기 전)
• 비용 절감, 보안(PIII), 정책 위반 차단	주 목적	• 품질 관리, 환각(Hallucination) 방지, 답변 안전성 검증
• 주로 규칙 기반(Keywords, Regex), 가벼운 로직	방식	• 주로 모델 기반(LLM에게 평가 요청), 검증 로직
• 매우 저렴함 (LLM 호출 자체를 막음)	비용	• 추가 비용 발생 (메인 LLM + 감시자 LLM 비용)
• 매우 빠름	속도	• 감시자 모델이 검사하는 시간만큼 느려짐

+

Combine multiple Guardrails

3. Guardrails > Combine multiple guardrails



```
@before_agent(can_jump_to=["end"])
def education_guardrail(state, runtime):
    """
```

```
    학생의 질문 의도를 파악하여 교육적이지 않거나 부정행위가 의심될 경우, LLM(AI)에게 질문
    을 넘기지 않고 교육적인 멘트로 즉시 교정합니다."""
```

```
@before_agent
def student_safety_middleware(state, runtime):
    """
```

```
    학생의 전화번호나 이메일이 감지되면 마스킹 처리하여 안전을 확보
    """
```

```
@before_agent(can_jump_to=["end"])
def counseling_escalation_middleware(state, runtime):
    """
```

```
    [Layer 3] 심리적 위기 상황이나 상담 요청이 감지되면 AI 답변을 멈추고 인간 상담사에게 알
    림을 보냅니다."""
```

```
@after_agent
def answer_leakage_guardrail(state, runtime):
    """
```

```
    AI가 답변을 생성한 '직후', 사용자에게 보여주기 전에 내용을 검사.
    만약 AI가 문제의 정답을 직접적으로 말해버렸다면, 이를 감지하고 수정."""
```


3. Guardrails > Combine multiple guardrails

4중 방어막이 적용된 에이전트

```
agent = create_agent(
    model="gpt-5-nano",
    tools=[],
    middleware=[
        education_guardrail,          # Layer 1: 입력 필터 (규칙 - 딴짓/부정행위) - before_agent
        student_safety_middleware,    # Layer 2: 개인정보 보호 (전화번호 마스킹) - before_agent
        counseling_escalation_middleware, # Layer 3: 상담 이관 (휴먼 에스컬레이션) - before_agent
        answer_leakage_guardrail       # Layer 4: 출력 필터 (모델 기반 교정) - after_agent
    ],
)
```

```
1 agent.invoke({"messages": [{"role": "user", "content": "저 수학 과외 구하고 싶어요. 제 번호 010-1234-5678로 연락 주세요."}]})
```

...  [학생 보호] 개인정보가 감지되어 마스킹 처리했습니다.
원본: 저 수학 과외 구하고 싶어요. 제 번호 010-1234-5678로 연락 주세요.
수정: 저 수학 과외 구하고 싶어요. 제 번호 <PHONE_REDACTED>로 연락 주세요.
{'messages': [HumanMessage(content='저 수학 과외 구하고 싶어요. 제 번호 <PHONE_REDACTED>로 연락 주세요.', additional_kwargs={},

```
1 agent.invoke({"messages": [{"role": "user", "content": "나 요즘 학교에서 왕따 당하는 것 같아서 너무 우울해."}]})
```

...  [상담 이관] 심각한 고민/요청 감지: 왕따
{'messages': [HumanMessage(content='나 요즘 학교에서 왕따 당하는 것 같아서 너무 우울해.', additional_kwargs={}, response_metadata={}, id='862dbeb0-a962-4d8b-a600-8e1cf6050d1b'),
AIMessage(content='학생, 많이 힘들었겠구나. 이 문제는 내가 답변하기보다는 전문 상담 선생님이 직접 듣고 도와주시는 게 좋을 것 같아. #n#n지금 바로 상담 선생님께 연결해 드렸으니 잠시만 기다려 줄래? 🌸 (상담실 연결 중...)', additional_kwargs={}, response_metadata={}, id='85408f78-

4. Long-term Memory

▪ Store

- **store** : 모든 사용자 정보를 저장하는 인메모리 저장소. key-value 쌍으로 Document 객체를 저장
- **store 관리** : (저장) `store.mset(store_key, store_value)`, (조회) `store.mget(store_key)`
- **store_key** : namespace(폴더) + id(파일)
- store_key 목록을 값으로 저장하는 전역 딕셔너리 : `managed_keys_map` (defaultdict)

주요 작업	Use Case 1	Use Case 2
▪ store 생성	Langchain_core.stores의 InMemoryStore 사용하여 생성 (<code>store=InMemoryStore()</code>)	
▪ namespace 정의	namespace = f"{user_id}::{application_name}" # 사용자+App 정보로 폴더와 같은 용도	
▪ store 저장	Store_key : namespace + id("xx01") #하드코딩 Store_value : document 객체 #하드코딩	Store_key : namespace + id(uuid) #자동채번 Store_value : document 객체 #모델이 감지/생성
▪ store 조회	Store_key : managed_keys = []	Store_key : managed_keys_map (defaultdict)

4. Long-term Memory > Use Case 1

Use Case 1

- ① store 생성
 - ①-1 InMemoryStore() 객체 생성
 - ①-2 store key 목록 정의
 - ①-3 namespace key 목록 정의
- ② store key 정의
 - namespace prefix 정의
- ③ store data 저장
- ④ store data 조회
- ⑤ Context 정의 : user, app 정보 입력
- ⑥ @wrap_model_call 정의
 - 장기 메모리 데이터 → 모델 전달
- ⑦ agent 생성
 - 모델, store, context_schema
 - middleware 장착
- ⑧ agent 실행

```
from langchain_core.stores import InMemoryStore
from langchain_core.documents import Document
from collections import defaultdict # To store managed keys per user/app
```

```
# Initialize the store
```

```
store = InMemoryStore() ①-1
```

```
# List to keep track of our keys
```

```
managed_keys = [] ①-2
```

```
namespace_keys = defaultdict(list) ①-3
```

```
user_id = "user_001"
```

```
app_context = "personal_assistant"
```

```
# Define a namespace for the user and application
```

```
namespace_prefix = f"{user_id}::{app_context}::" ②
```


4. Long-term Memory > Use Case 1

Use Case 1

- ① store 생성
- ② store key 정의
 - namespace prefix 정의
- ③ store data 저장
 - ③-1 store_key 정의 : 하드코딩
 - ③-2 store_value 정의 : 하드코딩
 - ③-3 store 저장 (key, value)
 - ③-4 store_key 저장
 - ③-5 namespace key 저장
- ④ store data 조회
- ⑤ Context 정의 : user, app 정보 입력
- ⑥ @wrap_model_call 정의
 - 장기 메모리 데이터 → 모델 전달
- ⑦ agent 생성
 - 모델, store, context_schema
 - middleware 장착
- ⑧ agent 실행

```
# Document 1
key_1 = f"{namespace_prefix}item_001" ③-1

doc_1 = Document(
    page_content="사용자는 게임을 좋아함", ③-2
    metadata={
        "context": "preferences",
        "source": "chat_history"
    }
)

store.mset([(key_1, doc_1)]) ③-3

managed_keys.append(key_1) ③-4
namespace_keys[(user_id, app_context)].append(key_1) ③-5

< 이 하 생 략 >
# Document 2
# Document 3
```

```
# Retrieve all documents using the managed_keys list
all_retrieved_documents = store.mget(managed_keys) ④

print("\nAll retrieved documents:")
for i, doc in enumerate(all_retrieved_documents):
    print(f"\nDocument {i+1} : \nKey : {managed_keys[i]}")
    print(f"Content: {doc.page_content}\nMetadata: {doc.metadata}")
```

4. Long-term Memory > Use Case 1

Use Case 1

- ① store 생성
- ② store key 정의
 - namespace prefix 정의
- ③ store data 저장
- ④ store data 조회
- ⑤ Context 정의 : user, app 정보 입력
- ⑥ @wrap_model_call 정의
 - 장기 메모리 데이터 → 모델 전달
- ⑦ agent 생성
 - 모델, store, context_schema
 - middleware 장착
- ⑧ agent 실행

```
from dataclasses import dataclass
```

```
# 실행 컨텍스트 정의 (누가 실행하는지 식별)
```

```
@dataclass
class Context:
    user_id: str
    app_name: str
```

⑤

```
@wrap_model_call
```

```
def inject_memory(request, handler):
```

```
    current_user = request.runtime.context.user_id
    current_app = request.runtime.context.app_name
    keys_to_retrieve = namespace_keys[(current_user, current_app)]
```

⑥

```
    memory_content = "기록된 사용자 특정 정보 없음"
    extracted_facts = []
```

```
    if keys_to_retrieve:
```

```
        retrieved_documents: list[Document | None] = request.runtime.store.mget(keys_to_retrieve)
```

```
        for doc in retrieved_documents:
```

```
            if doc is not None and isinstance(doc, Document):
                extracted_facts.append(doc.page_content)
```

```
    if extracted_facts:
```

```
        memory_content = "\n- ".join(extracted_facts)
```

```
    system_message = f"\n사용자 관련 장기 메모리 : \n- {memory_content}"
```

```
    new_request = request.override(system_prompt=system_message)
```

```
    return handler(new_request)
```

4. Long-term Memory > Use Case 1

Use Case 1

- ① store 생성
- ② store key 정의
 - namespace prefix 정의
- ③ store data 저장
- ④ store data 조회
- ⑤ Context 정의 : user, app 정보 입력
- ⑥ @wrap_model_call 정의
 - 장기 메모리 데이터 → 모델 전달
- ⑦ agent 생성
 - 모델, store, context_schema
 - middleware 장착
- ⑧ agent 실행

```
from langchain.agents import create_agent
```

```
agent = create_agent(
    model="gpt-5-nano",
    store=store, # store 연결
    context_schema=Context,
    middleware=[inject_memory] # 미들웨어 장착
)
```

⑦

```
response = agent.invoke(
    {"messages": [{"role": "user", "content": "나에 대해 알고있는 정보 알려줘"}]},
    context=Context(user_id="user_001", app_name="personal_assistant")
)
```

⑧

키 목록: ['user_001::personal_assistant::item_001', 'user_001::personal_assistant::item_002', 'user_001::personal_assistant::item_003']
 사용자 user_001를 위한 시스템 메시지:
 사용자 관련 장기 메모리 :
 - 사용자는 게임을 좋아함
 - 사용자는 주말에 코딩을 함
 - 사용자는 초콜릿 아이스크림을 선호함

```
1 response = agent.invoke(
2     {"messages": [{"role": "user", "content": "나에 대해 알고있는 정보 알려줘"}]},
3     context=Context(user_id="user_008", app_name="personal_assistant")
4 )
```

... 키 목록: []
 사용자 user_008를 위한 시스템 메시지:
 사용자 관련 장기 메모리 :
 - 기록된 사용자 특정 정보 없음

4. Long-term Memory > Use Case 2

Use Case 2

① store 생성

①-1 InMemoryStore() 객체 생성

①-2 store key 목록 정의
- defaultdict(list)

② 기본 객체 정의

②- Context : user, app 정보 입력

②- UserInfo : 모델이 추출 할 정보 구조

③ @Tool 정의

③-1 사용자 정보 조회

③-2 사용자 정보 저장

④ @wrap_model_call 정의

- 장기 메모리 데이터 → 모델 전달

⑤ agent 생성

- 모델, store, context_schema

- tools, middleware 장착

⑥ agent 실행

```
from langchain_core.stores import InMemoryStore
from collections import defaultdict # To store managed keys per user/app
```

Store 초기화

```
store = InMemoryStore() ①-1
```

Global dictionary to store managed keys for each user/app namespace

Each key will be (user_id, app_name) and value will be a list of string keys

```
managed_keys_map = defaultdict(list) ①-2
```

```
from dataclasses import dataclass
from typing import TypedDict, Annotated
```

실행 컨텍스트 정의 (누가 실행하는지 식별)

```
@dataclass
class Context: ②-1
```

user_id: str

app_name: str

LLM이 추출해야 할 정보의 구조를 정의

```
class UserInfo(TypedDict):
```

"""사용자의 개인 정보, 민감 정보 및 선호도를 담는 구조체.

모든 필드는 사용자의 입력 문맥에서 정확히 추출되어야 합니다."""

②-2

user_name: Annotated[str, "The user's real name or nickname. Example: '홍길동', 'Alice'"]

age: Annotated[int, "The user's age in years. Must be a positive integer. Example: 29"]

email: Annotated[str, "The user's valid email address. ..."]

preference: Annotated[str, "The user's favorite items or hobbies. Example: '아메리카노', ..."]

life_style: Annotated[str, "The user's daily habits, routines or lifestyle description. ..."]

religion: Annotated[str, "The user's religion. Strict examples: '기독교', '천주교', '불교', ..."]

is_premium_member: Annotated[bool, "Boolean flag indicating whether the user is a premium..."]

4. Long-term Memory > Use Case 2

Use Case 2

- ① store 생성
 - ①-1 InMemoryStore() 객체 생성
 - ①-2 store key 목록 정의
 - defaultdict(list)
- ② 기본 객체 정의
 - ②- Context : user, app정보 입력
 - ②- UserInfo : 모델이 추출 할 정보 구조
- ③ @Tool 정의
 - ③-1 사용자 정보 조회
 - ③-2 사용자 정보 저장
- ④ @wrap_model_call 정의
 - 장기 메모리 데이터 → 모델 전달
- ⑤ agent 생성
 - 모델, store, context_schema
 - tools, middleware 장착
- ⑥ agent 실행

```
@tool
def get_user_info(runtime) -> str:
    """
    현재 사용자의 정보 조회 (시스템 내부용 도구)
    """
```

③-1

```
@tool
def save_user_info(user_info: UserInfo, runtime):
    """
    사용자의 정보를 저장하거나 업데이트
    사용자 정보가 하나라도 있으면 저장하고, 없으면 저장하지 않음
    """
```

③-2

```
From langchain.agents.middleware import wrap_model_call
```

```
@wrap_model_call
Def prompt_update(request, handler):
    """
    system_message = request.system_prompt or ""
    system_message += """사용자 프롬프트에서 사용자 정보가 인지되면, 정보를 저장해줘.
    사용자 정보를 묻는 경우, 장기 메모리에 저장된 내용들을 종합해서 친절하게 답변해줘."""

    request = request.override(system_prompt=system_message)
    return handler(request)
```

④

4. Long-term Memory > Use Case 2

Use Case 2

- ① store 생성
 - ①-1 InMemoryStore() 객체 생성
 - ①-2 store key 목록 정의
 - defaultdict(list)
- ② 기본 객체 정의
 - ②- Context : user, app정보 입력
 - ②- UserInfo : 모델이 추출 할 정보 구조
- ③ @Tool 정의
 - ③-1 사용자 정보 조회
 - ③-2 사용자 정보 저장
- ④ @wrap_model_call 정의
 - 장기 메모리 데이터 → 모델 전달
- ⑤ agent 생성
 - 모델, store, context_schema
 - tools, middleware 장착
- ⑥ agent 실행

```
from langchain.agents import create_agent
```

```
agent = create_agent(
    model="gpt-5-nano",
    tools=[get_user_info, save_user_info],
    store=store, # 에이전트에 store 연결
    middleware=[prompt_update], # 미들웨어 장착
    context_schema=Context
)
```

⑤

```
# 대화형 질의
```

```
print("질문을 입력하세요 (종료: exit or quit)")
```

⑥

```
while True:
```

```
    question = input("\n질문: ")
```

```
    if question.lower() in ['exit', 'quit', '종료', '끝']:
```

```
        print("종료합니다.")
```

```
        break
```

```
    response = agent.invoke({"messages": [{"role": "user", "content": question}],
                             context=Context(user_id="user_001", app_name="personal_assistant"))
```

```
    # print(f"답변: {response}")
```

```
    print(f"답변: {response['messages'][-1].content}")
```

③-1 사용자 정보 조회

```

from langchain.tools import tool
from langchain_core.documents import Document

# Tool 정의: 사용자의 정보를 조회
@tool
def get_user_info(runtime) -> str:
    """
    현재 사용자의 정보 조회 (시스템 내부용 도구)
    """
    user_id = runtime.context.user_id
    app = runtime.context.app_name
    store = runtime.store
    global managed_keys_map    # Access the global managed_keys_map

    # Get all keys for the current user/app from our managed_keys_map
    keys_to_retrieve = managed_keys_map[(user_id, app)]

    if not keys_to_retrieve:
        return "기록된 정보 없음"

    # Use mget to retrieve the documents
    retrieved_documents: list[Document | None] = store.mget(keys_to_retrieve)

    results = []
    for doc in retrieved_documents:
        if doc is not None:
            results.append(doc.page_content)
            for k, v in doc.metadata.items():
                if v:
                    results.append(f" {k}: {v}")

    return "\n".join(results) if results else "데이터 형식 불일치로 읽을 수 없음"

```

③-2 사용자 정보 저장

```
import uuid

@tool
def save_user_info(user_info: UserInfo, runtime):
    """
    사용자의 정보를 저장하거나 업데이트
    사용자 정보가 하나라도 있으면 저장하고, 없으면 저장하지 않음
    """
    user_id = runtime.context.user_id    # 실행 컨텍스트에서 user_id 가져오기
    app = runtime.context.app_name
    store = runtime.store
    global managed_keys_map    # Access the global managed_keys_map

    memory_item_id = str(uuid.uuid4())    # Generate a unique key for the memory item
    store_key = f"{user_id}::{app}::{memory_item_id}"    # Using a consistent key format for InMemoryStore
    # Convert user_info (TypedDict) to a Document object
    page_content_parts = []
    metadata = {}
    result = "변경된 정보가 없습니다."
    for u_key, u_value in user_info.items():
        if u_key in ["user_name", "preference", "life_style", "religion"]:
            if u_value: page_content_parts.append(f"{u_key} : {u_value}")
        else:
            if u_value: metadata[u_key] = u_value
    if page_content_parts or metadata:
        document = Document(
            page_content="; ".join(page_content_parts),
            metadata=metadata    # Store metadata
        )
        store.mset([(store_key, document)])    # Use mset to store the Document
        managed_keys_map[(user_id, app)].append(store_key)    # Add the key to our managed list for this user/app namespace
        result = f"정보가 저장되었습니다. (ID: {store_key})"
    return result
```


4. Long-term Memory

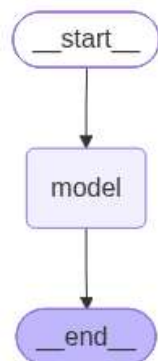
Use Case 1

```
...
All retrieved documents:

Document 1 :
Key : user_001::personal_assistant::item_001
Content: 사용자는 게임을 좋아함
Metadata: {'context': 'preferences', 'source': 'chat_history'}

Document 2 :
Key : user_001::personal_assistant::item_002
Content: 사용자는 주말에 코딩을 함
Metadata: {'context': 'activity', 'source': 'user_input'}

Document 3 :
Key : user_001::personal_assistant::item_003
Content: 사용자는 초콜릿 아이스크림을 선호함
Metadata: {'context': 'preferences', 'source': 'user_survey'}
```



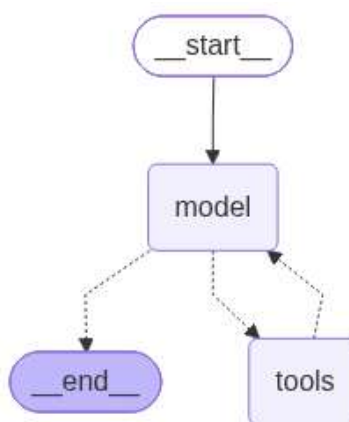
- wrap_model_call 에서
Context에서 받은 사용자정보로
하드코딩으로 이미 저장된 store 를 조회하여
system_message에 주입

Use Case 2

```
All retrieved documents:

Document 1 :
Key : user_008::personal_assistant::15958d78-8817-42d7-b3f6-9cb8d34f07e4
Content: user_name : 지우; preference : 아침 북상용 음악 목록을 이메일로 받기; I
Metadata: {'age': 45, 'email': 'aaa@email.com'}

Document 2 :
Key : user_008::personal_assistant::16a766c7-4b1c-47bb-974f-ae2f3bb75c8e
Content: user_name : 지우; preference : 아침 묵상 시 들으면 좋을 음악 목록; life
Metadata: {'age': 40, 'email': 'aaa@email.com'}
```



- save_user_info() :
대화에서 사용자 정보를 인지하여 저장(uuid key)
- get_user_info() :
사용자 정보를 물을때, 도구 호출

이번 과정에서 배운 내용

LangChain

- LCEL(LangChain Expression Language)과 체인 구성
- 프롬프트 엔지니어링 및 구조화된 출력 (Structured Output)
- 도구 바인딩 (Tool Calling)과 대화 메모리(Memory) 관리

LLM 제어 및 선형적인 AI 애플리케이션 파이프라인 구축

RAG

● LLM과 친해지기

Hugging Face, Google GenAI, Open AI

- 데이터 전처리(ETL) 및 벡터 데이터베이스 적재

● LangChain 기본 컴포넌트

- 고급 검색 기술 (Advanced Retrieval & Re-ranking)

- 컨텍스트 주입 및 할루시네이션(환각) 제어

● Basic Agent

Tool 사용, Short-term Memory, 구조화된 답변

● Advanced Agent

Runtime & State, Middleware, Guardrails, Long-term Memory

식 기반 서비스 개발

LangGraph

- 상태(State) 기반 순환 그래프(Cyclic Graph) 설계

- 영속성(Persistence)과 동적 인터럽트 (Dynamic Interrupt)

- 멀티 에이전트 오케스트레이션 (Multi-Agent System)

는 자율형 멀티 에이전트 구축

참고. 링크

리소스	URL
LangChain 메인 문서	https://docs.langchain.com
Build Overview	https://docs.langchain.com/build-overview
LangChain Python 개요	https://docs.langchain.com/oss/python/langchain/overview
LangChain 설치 가이드	https://docs.langchain.com/oss/python/langchain/install
LangChain 빠른 시작	https://docs.langchain.com/oss/python/langchain/quickstart
Models 가이드	https://docs.langchain.com/oss/python/langchain/models
Tools 가이드	https://docs.langchain.com/oss/python/langchain/tools
Agents 가이드	https://docs.langchain.com/oss/python/langchain/agents
LangGraph 개요	https://docs.langchain.com/langgraph
LangSmith 관찰성	https://docs.langchain.com/langsmith
통합(Integrations) 페이지	https://docs.langchain.com/oss/python/integrations/providers/overview
API 레퍼런스	https://reference.langchain.com/python
LangChain Academy	https://academy.langchain.com
GitHub	https://github.com/langchain-ai/langchain
커뮤니티 포럼	https://community.langchain.com



수고 하셨습니다

본 문서는 SK(주)AX의 콘텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.